

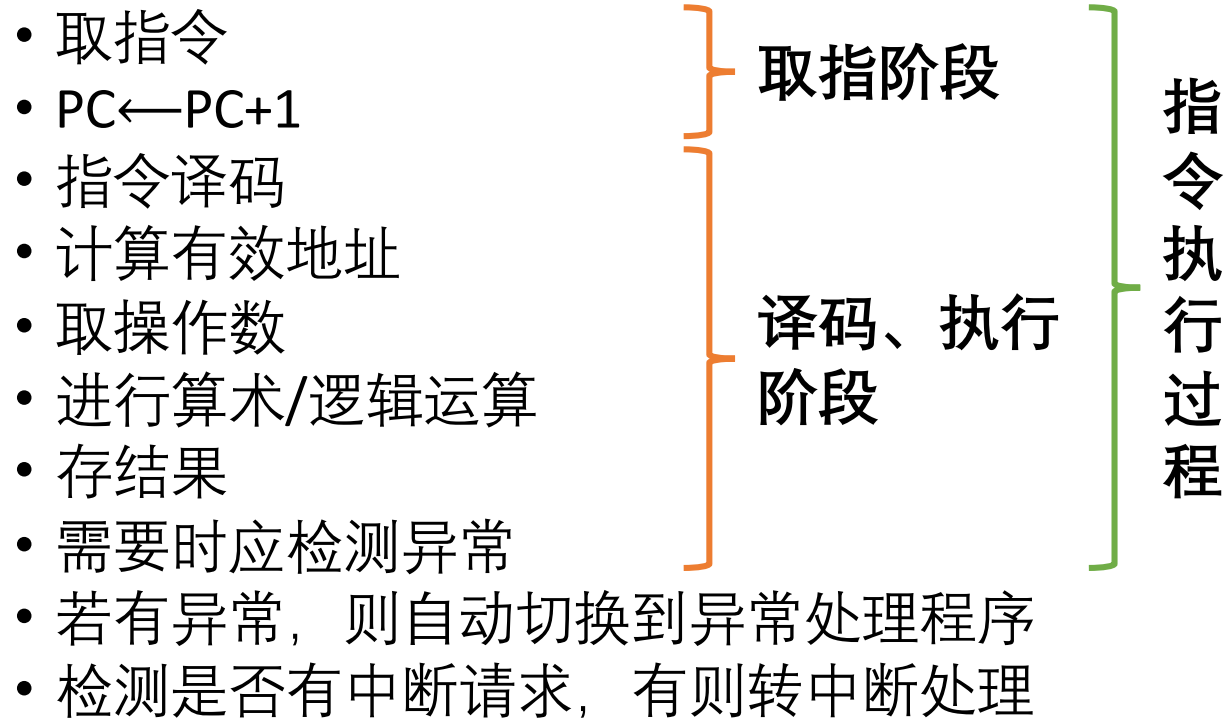
# 第三章 处理器体系结构

第一讲 中央处理器概述

第二讲 单周期处理器设计

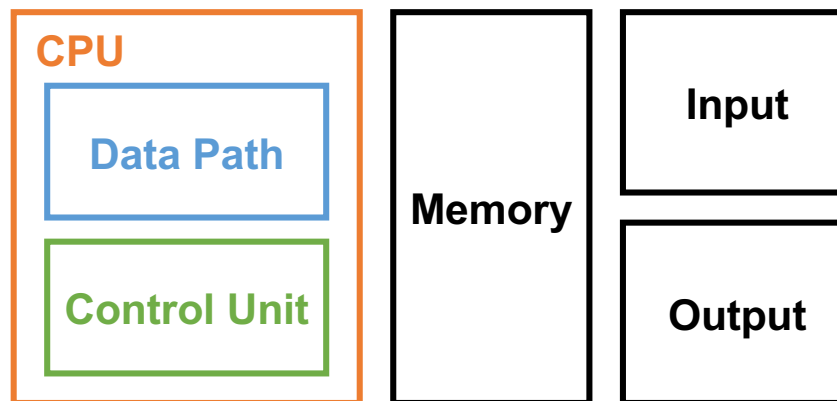
第三讲 流水线处理器设计

# CPU执行程序 and 指令的过程



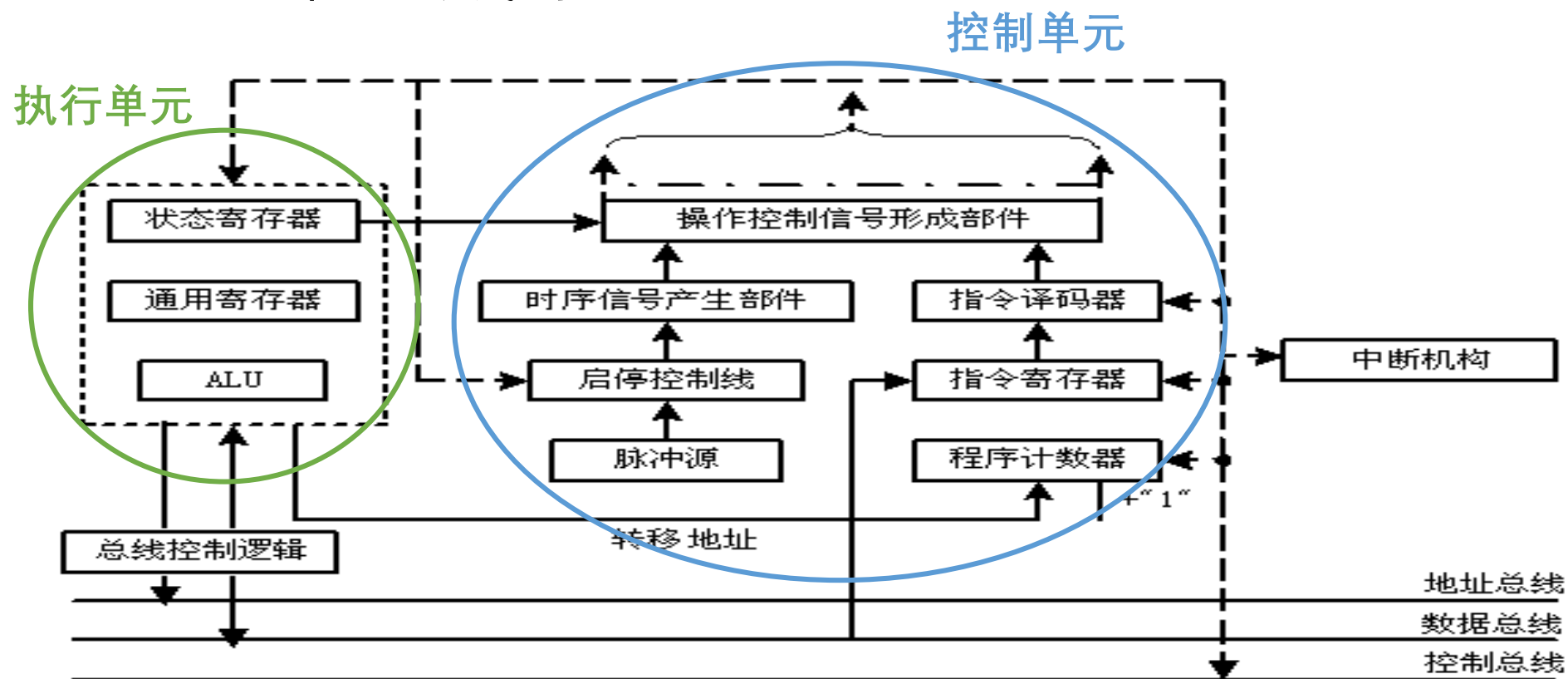
# CPU基本结构

- 计算机的五大组成部分



- 数据通路 (Data Path)
  - 指令执行过程中，数据所经过的路径，包括路径中的部件。它是指令的执行部件。
- 控制单元 (Control Unit)
  - 对指令进行译码，生成指令对应的控制信号，控制数据通路的动作。它能对执行部件发出控制信号，因此是指令的控制部件。

# CPU基本组成原理

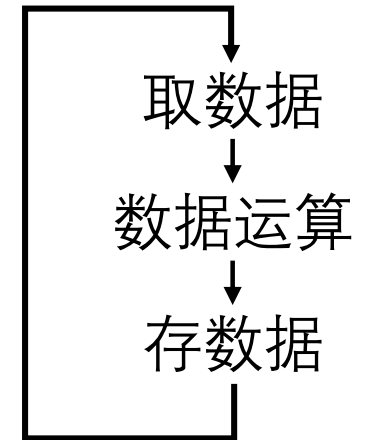


CPU由数据通路（执行单元）和控制器（控制单元）两部分组成

控制单元主要由指令译码单元和控制信号形成单元组成

# 数据通路的基本结构

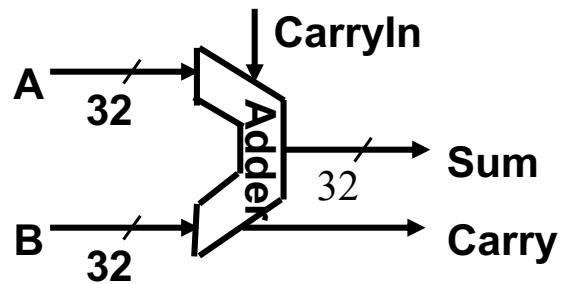
- 数据通路由两类元件组成
  - 组合逻辑元件（也称操作元件）
    - 逻辑门组成的电路，输入决定输出，如加法器、移位器、多路选择器
  - 时序逻辑元件（也称状态元件，存储元件）
    - 依赖时钟信号，有内部状态，状态转移存储信息，如寄存器、计数器
- 元件间的连接方式
  - 总线连接方式
  - 分散连接方式
- 数据通路的构成
  - 由“操作元件”和“存储元件”通过总线方式或分散方式连接而成
- 数据通路的功能
  - 进行数据存储、处理、传送



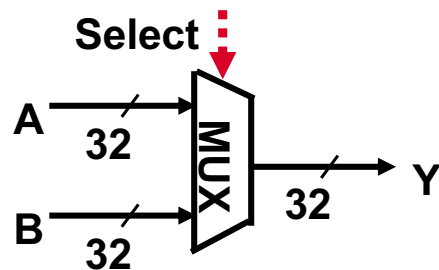
数据通路是由操作元件和存储元件通过总线或分散方式连接而成的进行数据存储、处理、传送的路径

# 操作元件：组合逻辑电路

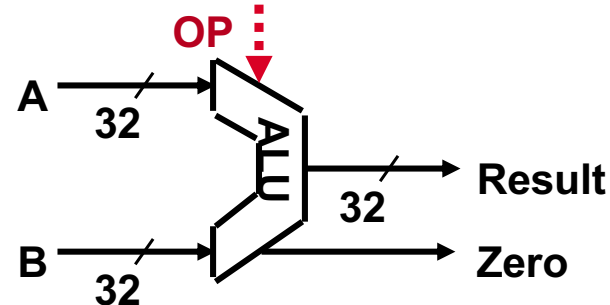
## • 加法器 (Adder)



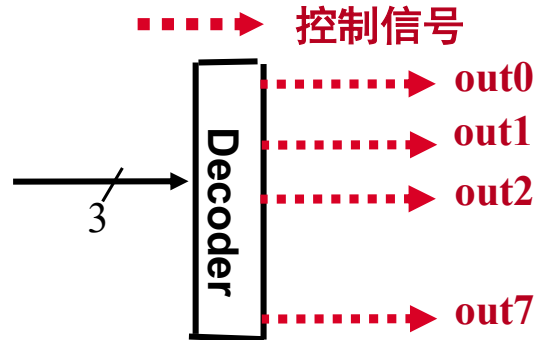
## • 多路选择器 (MUX)



## • 算术逻辑单元 (ALU)



## • 译码器 (Decoder)

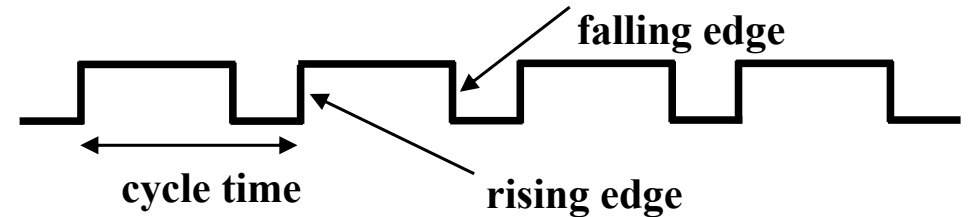


## 操作元件的特点

- 输出只取决于当前的输入。即：若输入一样，则其输出也一样
- 无定时：所有输入到达后，经过一定的逻辑门延时，输出端改变，并保持到下次改变，不需要时钟信号来定时

# 状态元件：时序逻辑电路

- 状态（存储）元件的特点
  - 具有**存储功能**，在时钟控制下输入被写到电路中，直到下个时钟到达
  - **输入端由时钟决定何时被写入，输出端状态随时可以读出**
- 定时方式：规定信号何时写入状态元件或何时从状态元件读出
  - **边沿触发**方式：状态单元中的值只在时钟边沿改变。每个时钟周期改变一次。
    - 上升沿触发：在时钟正跳变时进行读/写。
    - 下降沿触发：在时钟负跳变时进行读/写。



- 最简单的状态元件：D触发器

| CLK（时钟输入） | D（数据输入） | Q（输出） |
|-----------|---------|-------|
| ↑（上升沿）    | 0       | 0     |
| ↑         | 1       | 1     |

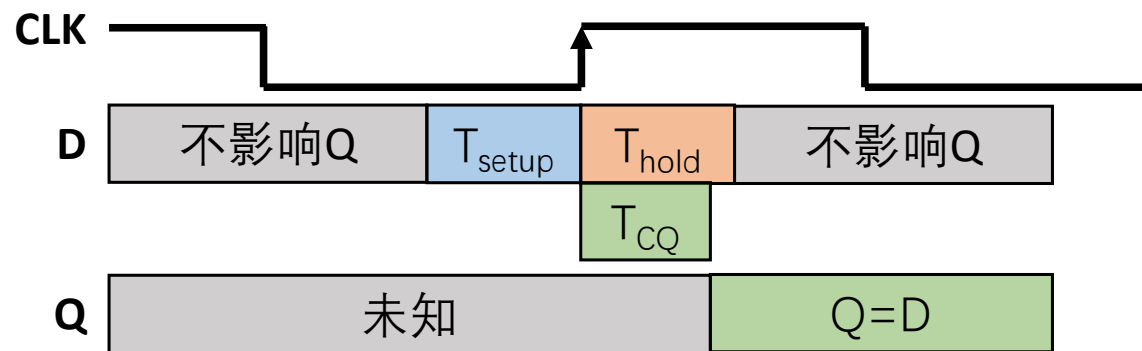
- 寄存器：多个D触发器并联

## 寄存器（组），存储器

# 状态元件的状态改变时间

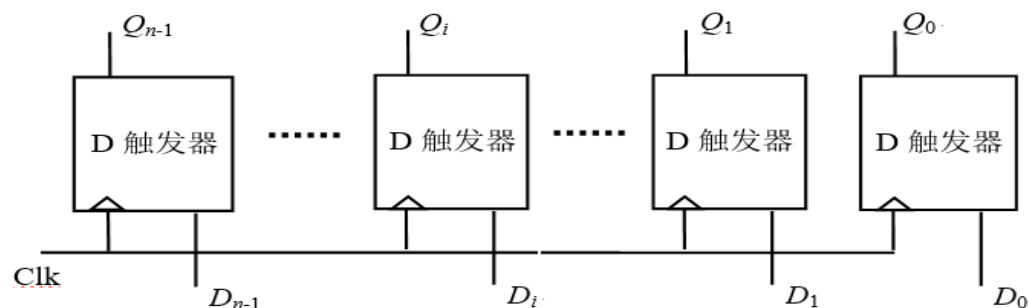
- 建立时间 ( $T_{\text{setup}}$ )
  - 在时钟信号的上升沿或下降沿到来之前，输入信号必须保持稳定的最小时间
- 保持时间 ( $T_{\text{hold}}$ )
  - 输入信号在时钟信号的有效边沿之后必须保持稳定的最小时间
- 锁存延迟 (Clock-to-Q时间,  $T_{\text{CQ}}$ )
  - 收到时钟信号的有效边沿之后，输出信号发生改变所需的时间

状态元件的输入总是在一个时钟边沿到达后的“Clk-to-Q”时间后才被写入元件，此时的输出才反映新的状态值



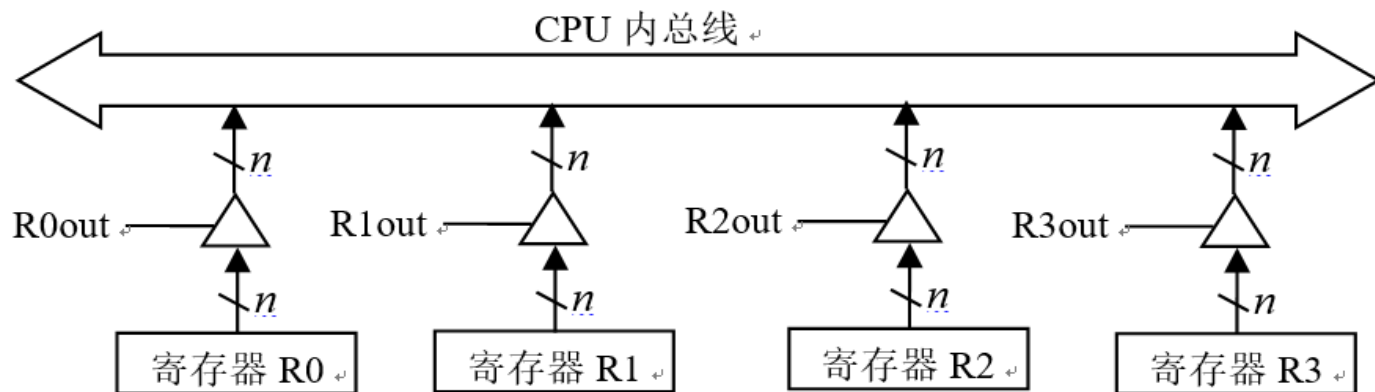
# 寄存器

- 寄存器是用来暂存信息的逻辑部件
- 寄存器可直接由若干个触发器组成



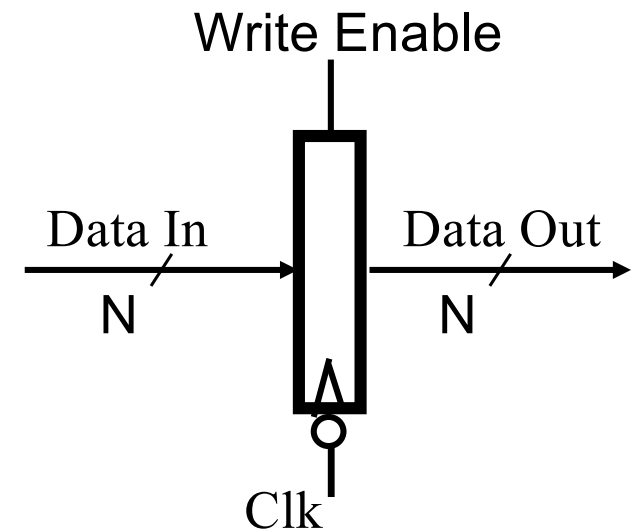
- 寄存器通过三态门和总线互连

任何时刻至多只有一个Rout有效



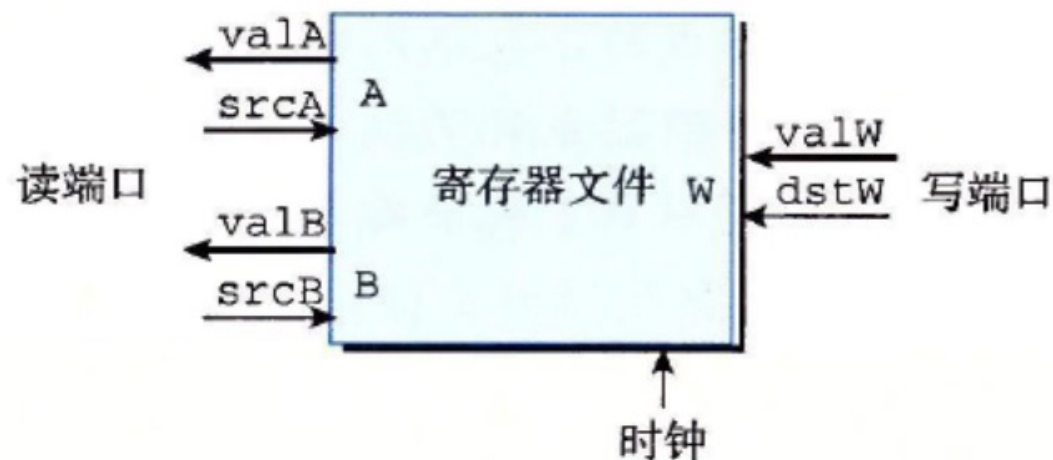
# 寄存器

- 多数时候保持在稳态 (x)，输出等于当前稳态
- 信号沿着寄存器前面的组合逻辑传播，产生新输入 (y) 时
- 若时钟在低电位，则输出仍为x
- 时钟变为高电位后y被存储
- 有一个写使能 (Write Enable, WE) 信号
  - 0: 时钟边沿到来时，输出不变
  - 1: 时钟边沿到来时，输出开始变为输入
- 若每个时钟边沿都写入，则不需WE信号



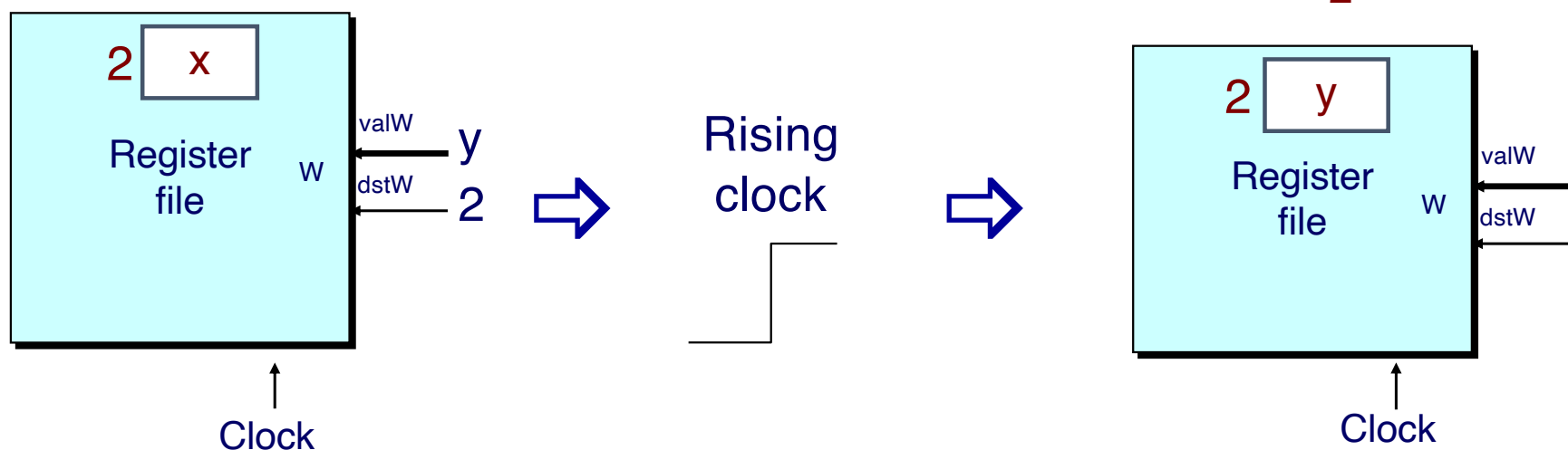
# 寄存器文件

- CPU内部用于暂存指令执行过程中的中间数据，也称通用寄存器组（General Purpose Register set, GPRs）
- 由寄存器组成，每个寄存器有一个编号，CPU可对指定编号的寄存器进行读写
- 两个读端口（A、B）
- 一个写端口（W）
- 读：组合逻辑操作
  - $srcA=ID$ ,  $valA=$ 寄存器ID的值
- 写：时序逻辑操作
  - $dstW=ID$ , 时钟上升（CLK+WE）， $valW$ 写入寄存器ID
  - $dstW=0xF$ , 不写入任何寄存器



# 寄存器文件

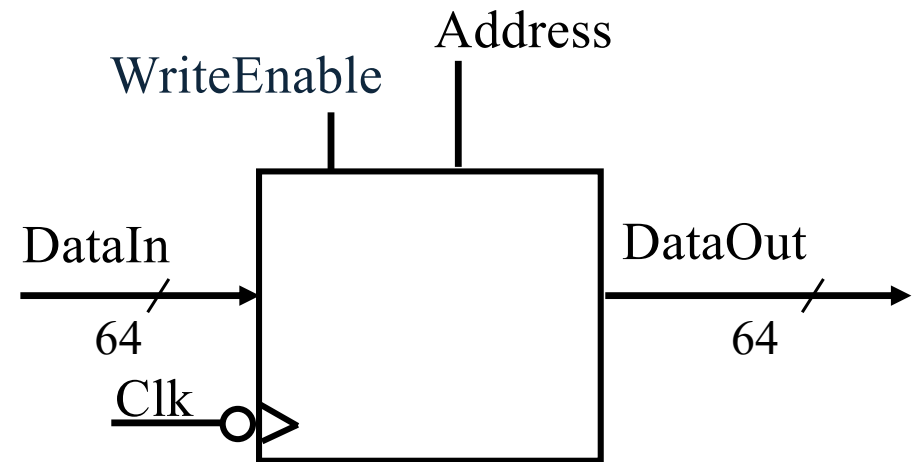
- 读
  - 组合逻辑
  - 根据输入地址（一定延迟后）得到输出数据
- 写
  - 类似寄存器，时钟上升更新



# 理想存储器模型

- 为简化数据通路操作说明，把存储器简化为带时钟信号的理想模型

- DataOut: 64位读出数据
- DataIn: 64位写入数据
- Address: 读写共用一个64位地址
- 读操作（组合逻辑）：地址Address有效后，经一个“取数时间AccessTime”，DataOut上数据有效
- 写操作（时序逻辑）：写使能为1的情况下，时钟Clk边沿到来时，DataIn传来的值开始被写入Address指定的存储单元中



# 早期计算机的三级时序

- 时钟周期 (T周期)

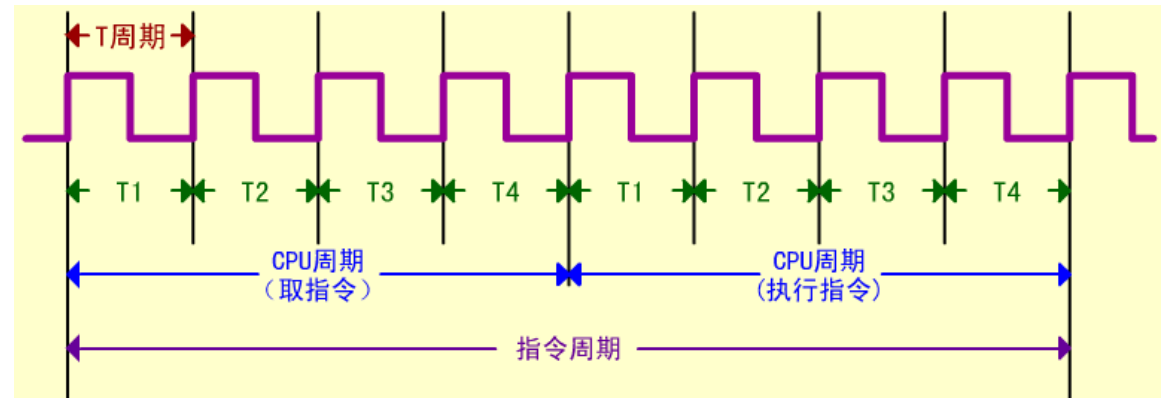
- 处理器内部的基础时间单位
- 所有操作发生在时钟周期的边沿
- $T=1/\text{CPU时钟频率}$

- 机器周期 (CPU周期)

- CPU完成某一操作（如一次指令执行或一次内存访问）所需的时间
- 通常包含多个T周期

- 指令周期

- CPU完成一条指令的完整过程所需的时间
- 每个阶段（取指、译码、执行、访存、写回、更新PC）可能各需要若干T周期



# 早期IAS机设计

## • 六条典型指令组成的一个简单程序

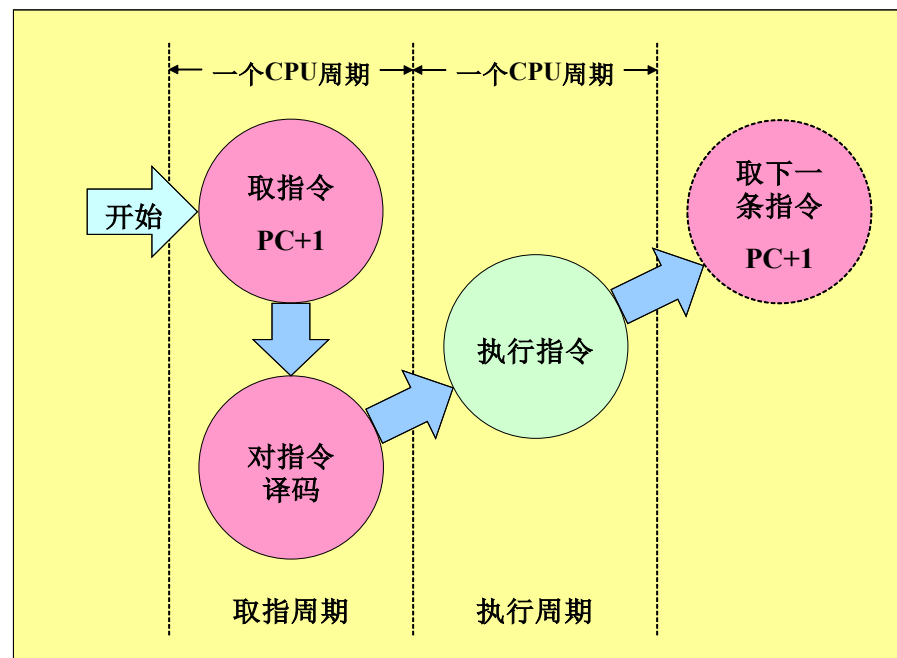
| 指令存储器 | 八进制地址 | 指令助记符        | 说明                                         |
|-------|-------|--------------|--------------------------------------------|
|       | 100   |              | 1. 程序执行前(R0)=00, (R1)=10, (R2)=20, (R3)=30 |
|       | 101   | MOV R0, R1   | 2. MOV执行(R1) → R0                          |
|       | 102   | LAD R1, 6    | 3. LAD从数存6号单元取数(100) → R1                  |
|       | 103   | ADD R1, R2   | 4. ADD执行(R1)+(R2)→R2, 结果(R2)=120           |
|       | 104   | STO R2, (R3) | 5. STO用(R3)间址, (R2)写入数存30号单元               |
|       | 105   | JMP 101      | 6. JMP改变程序执行顺序到101号单元                      |
|       | 106   | AND R1, R3   | 7. AND执行(R1) · (R3) → R3                   |

| 数据存储器 | 八进制地址 | 八进制数据    | 说明                  |
|-------|-------|----------|---------------------|
|       | 5     | 70       |                     |
|       | 6     | 100      | 执行LAD指令后, 6号单元内容不变  |
|       | 7     | 66       |                     |
|       | ...   | ...      |                     |
|       | ...   | ...      |                     |
|       | 30    | 40 → 120 | 执行STO指令后, 数据由40→120 |

# 早期IAS机设计

- **MOV指令的指令周期**

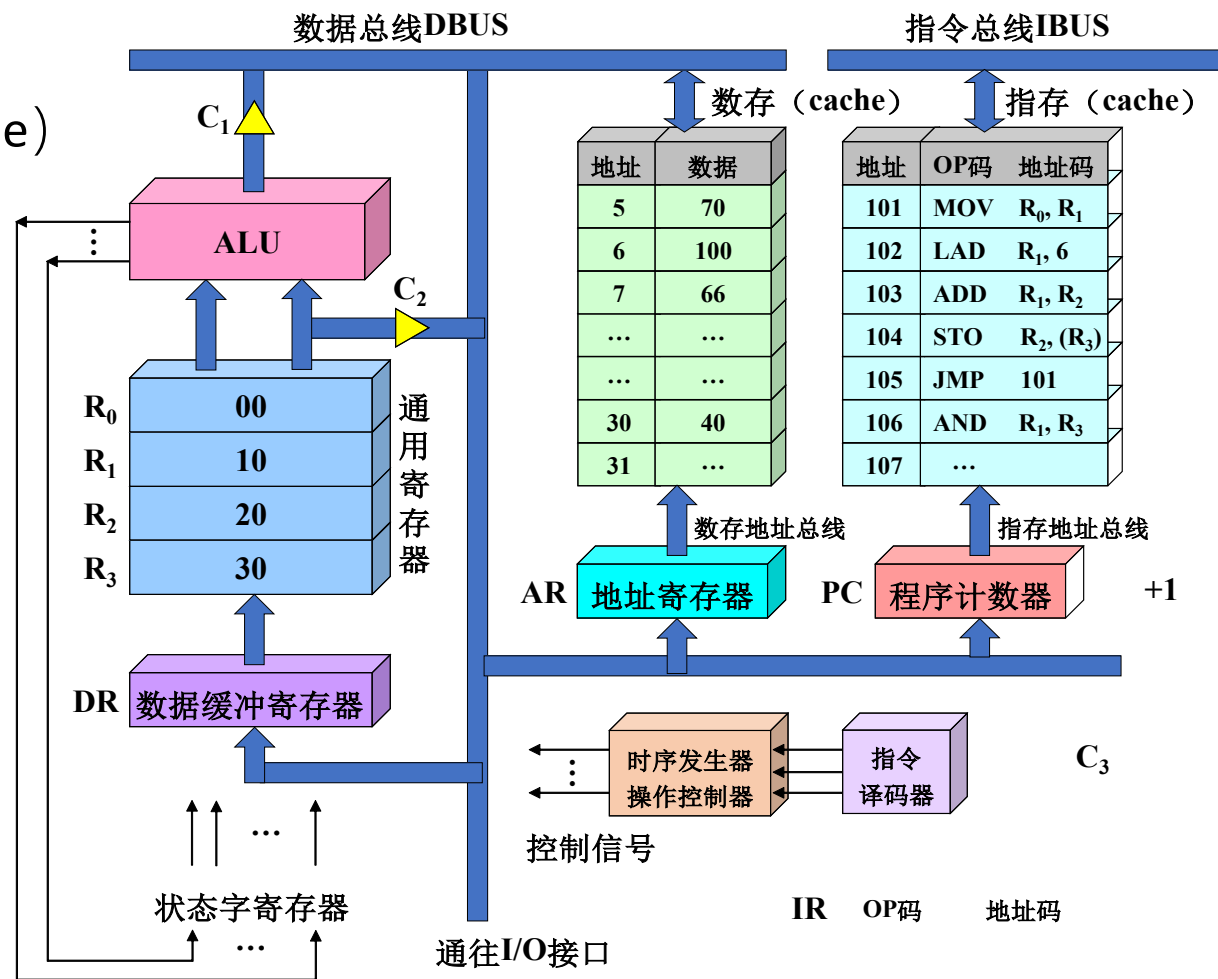
- RR型指令，主要需要两个CPU周期
  - 取指
  - 执行



# 早期IAS机设计

- 假定程序已经装入指存（I-Cache）

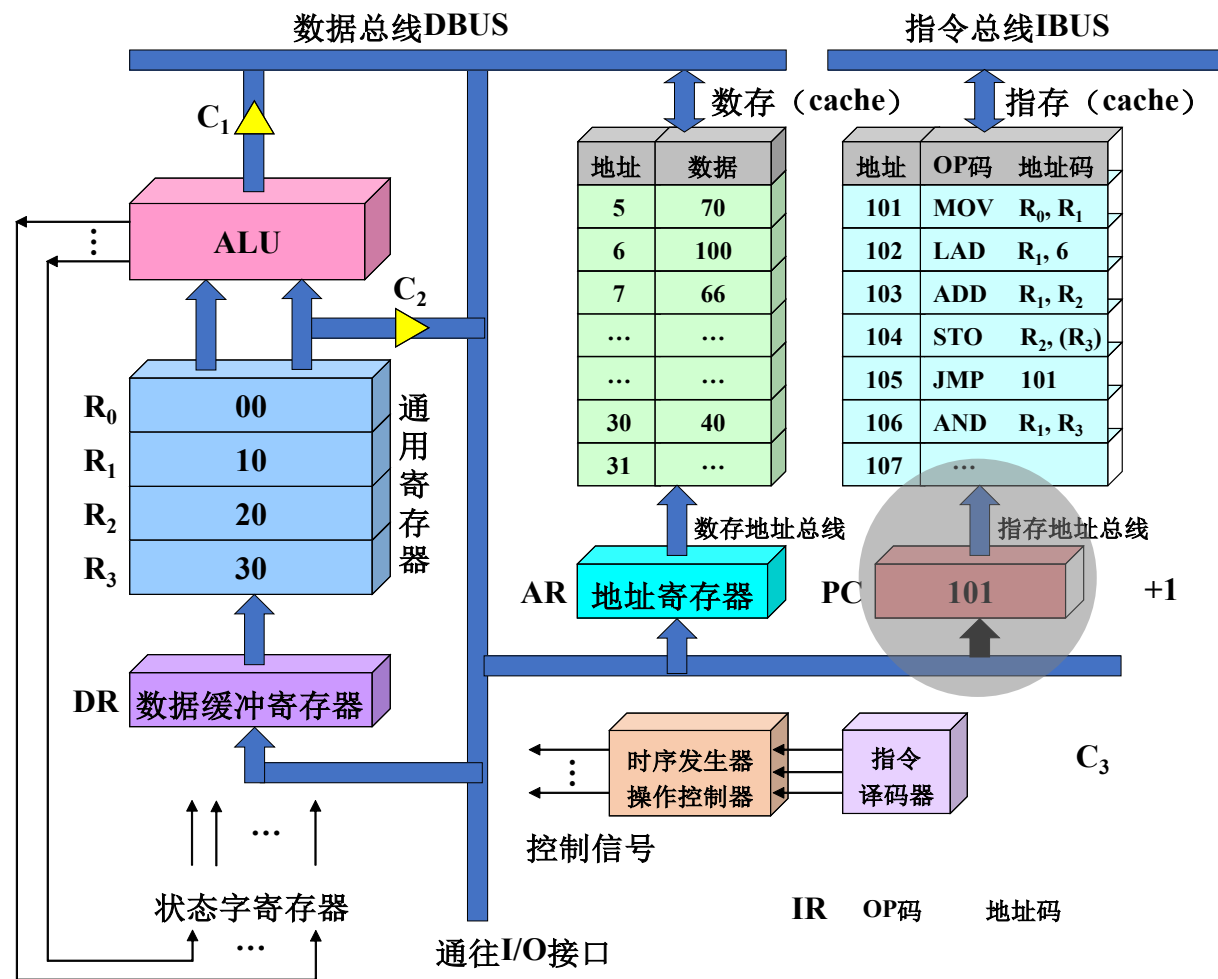
## 1. 取指周期



# 早期IAS机设计

## 1. 取指周期

### 1.1. 程序计数器PC中装入第一条指令地址101（八进制）

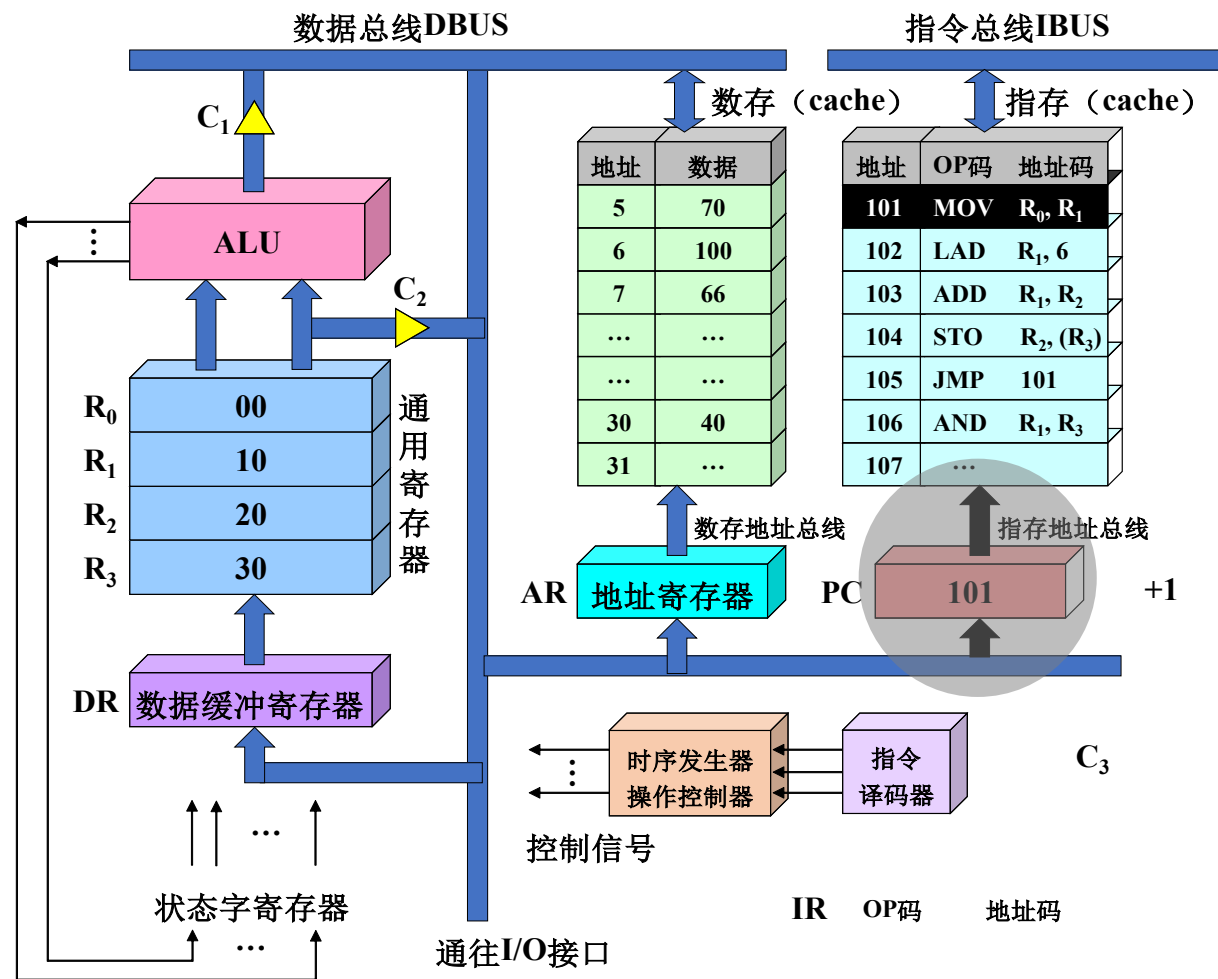


# 早期IAS机设计

## 1. 取指周期

1.1. 程序计数器PC中装入第一条指令地址101（八进制）

1.2. PC的内容被放到指存地址总线ABUS(I)上，启动读命令



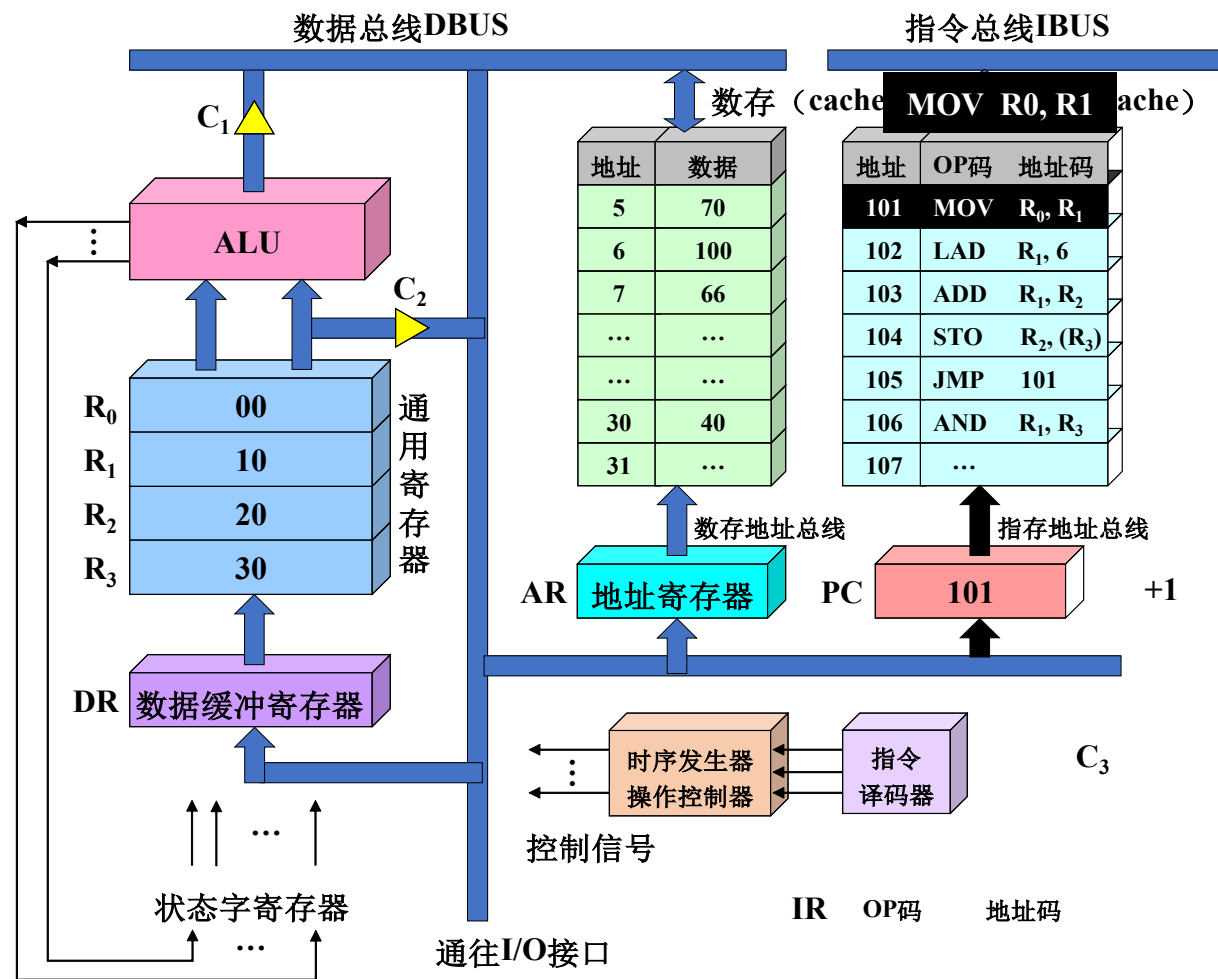
# 早期IAS机设计

## 1. 取指周期

1.1. 程序计数器PC中装入第一条指令地址101（八进制）

1.2. PC的内容被放到指存地址总线ABUS(I)上，启动读命令

1.3. 从101号地址读出指令  
通过指令总线IBUS  
装入指令寄存器IR



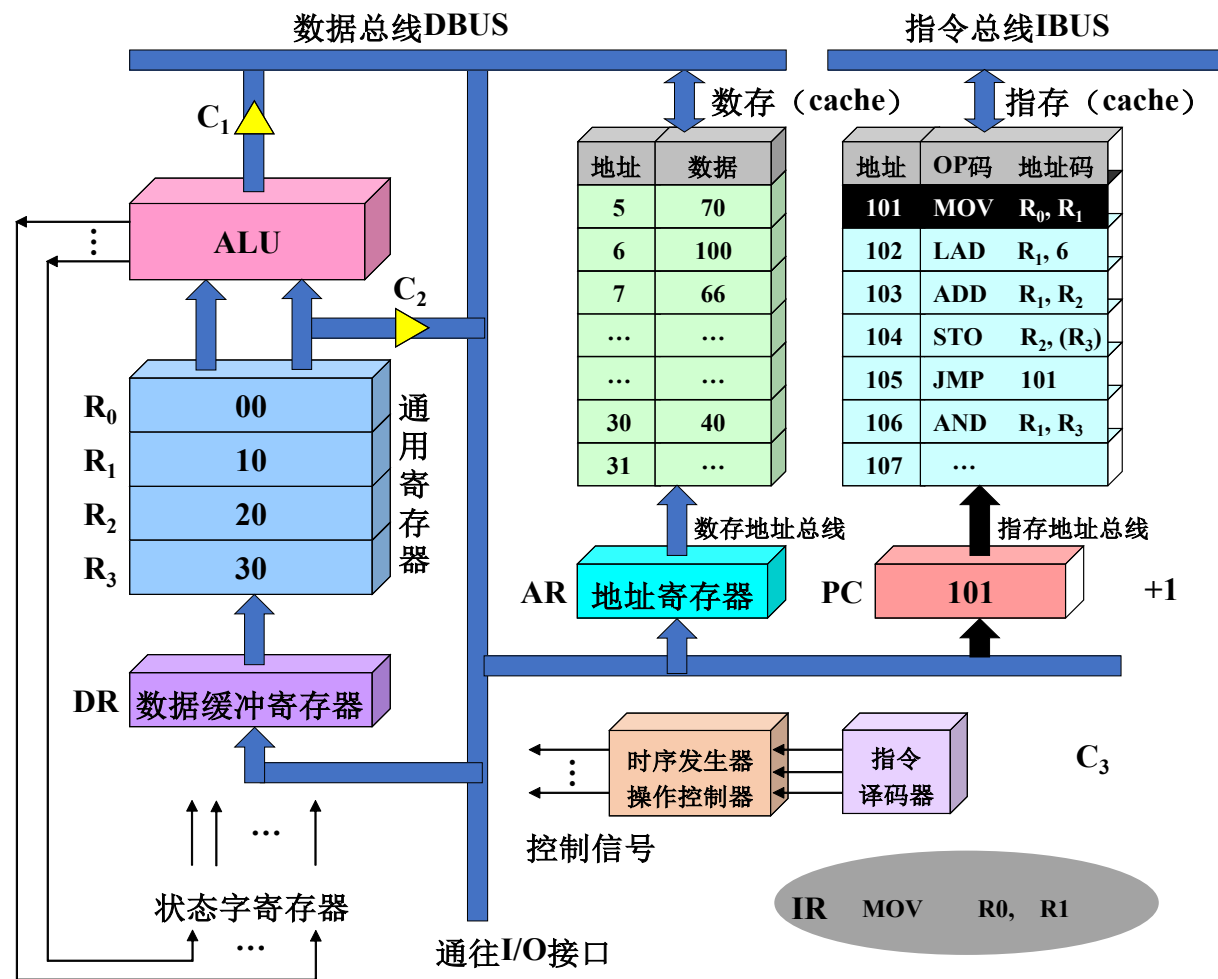
# 早期IAS机设计

## 1. 取指周期

1.1. 程序计数器PC中装入第一条指令地址101（八进制）

1.2. PC的内容被放到指存地址总线ABUS(I)上，启动读命令

1.3. 从101号地址读出指令  
通过指令总线IBUS  
装入指令寄存器IR



# 早期IAS机设计

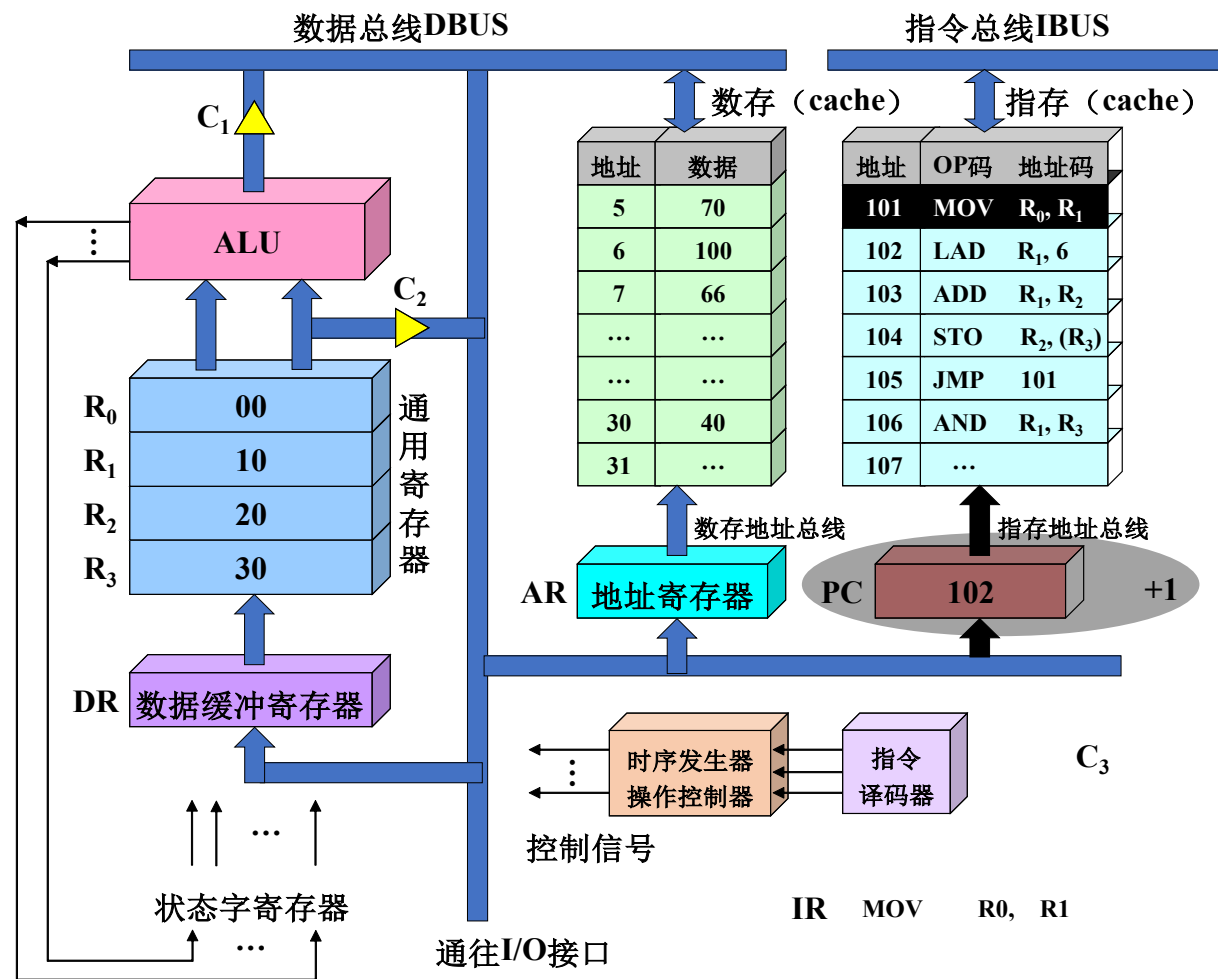
## 1. 取指周期

1.1. 程序计数器PC中装入第一条指令地址101（八进制）

1.2. PC的内容被放到指存地址总线ABUS(I)上，启动读命令

1.3. 从101号地址读出指令  
通过指令总线IBUS  
装入指令寄存器IR

1.4. PC+1，变成102，  
为取下一条指令做准备



# 早期IAS机设计

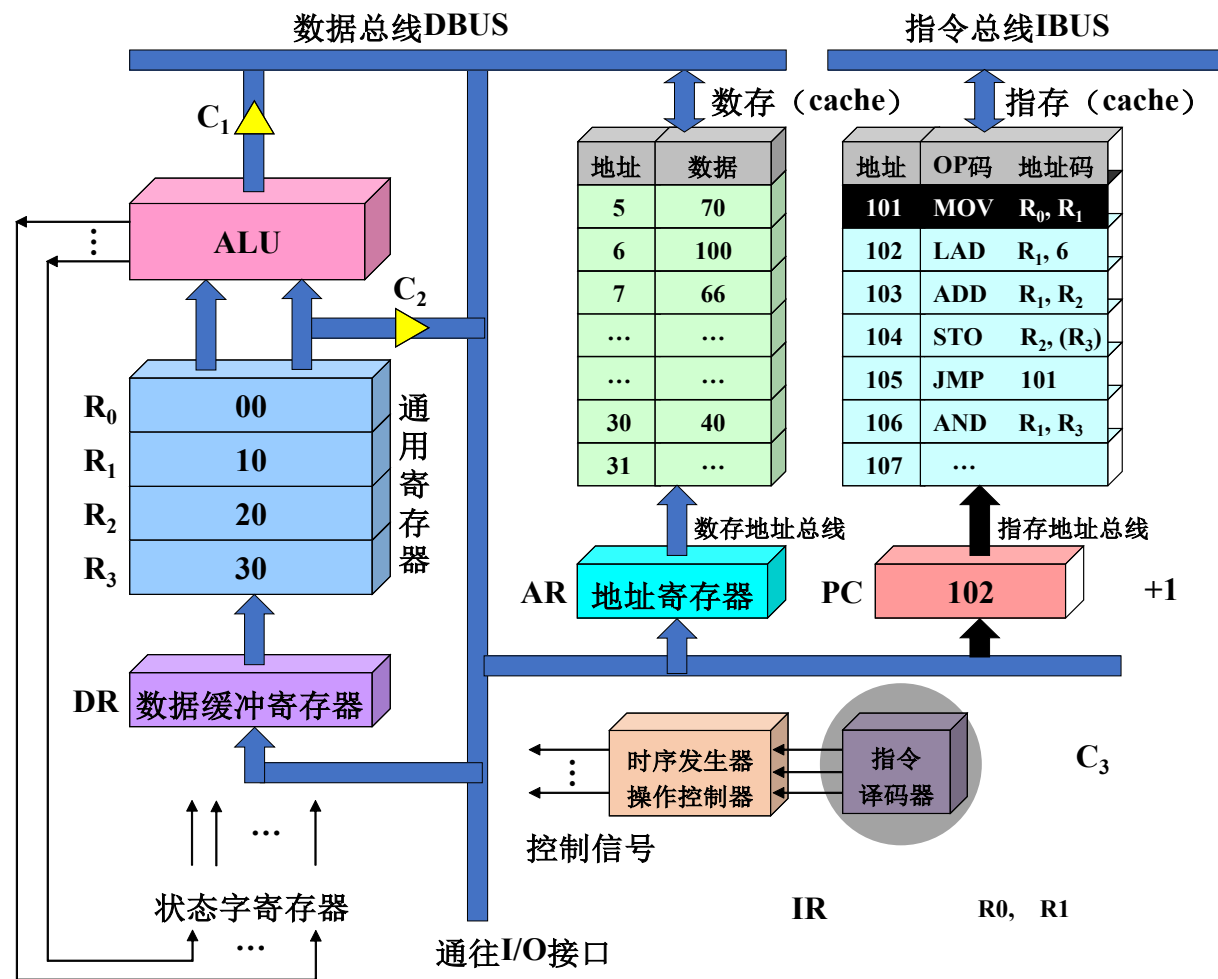
## 1. 取指周期

...

1.4.  $PC+1$ , 变成102,

为取下一条指令做准备

1.5. IR中的操作码 (OP) 被译码



# 早期IAS机设计

## 1. 取指周期

...

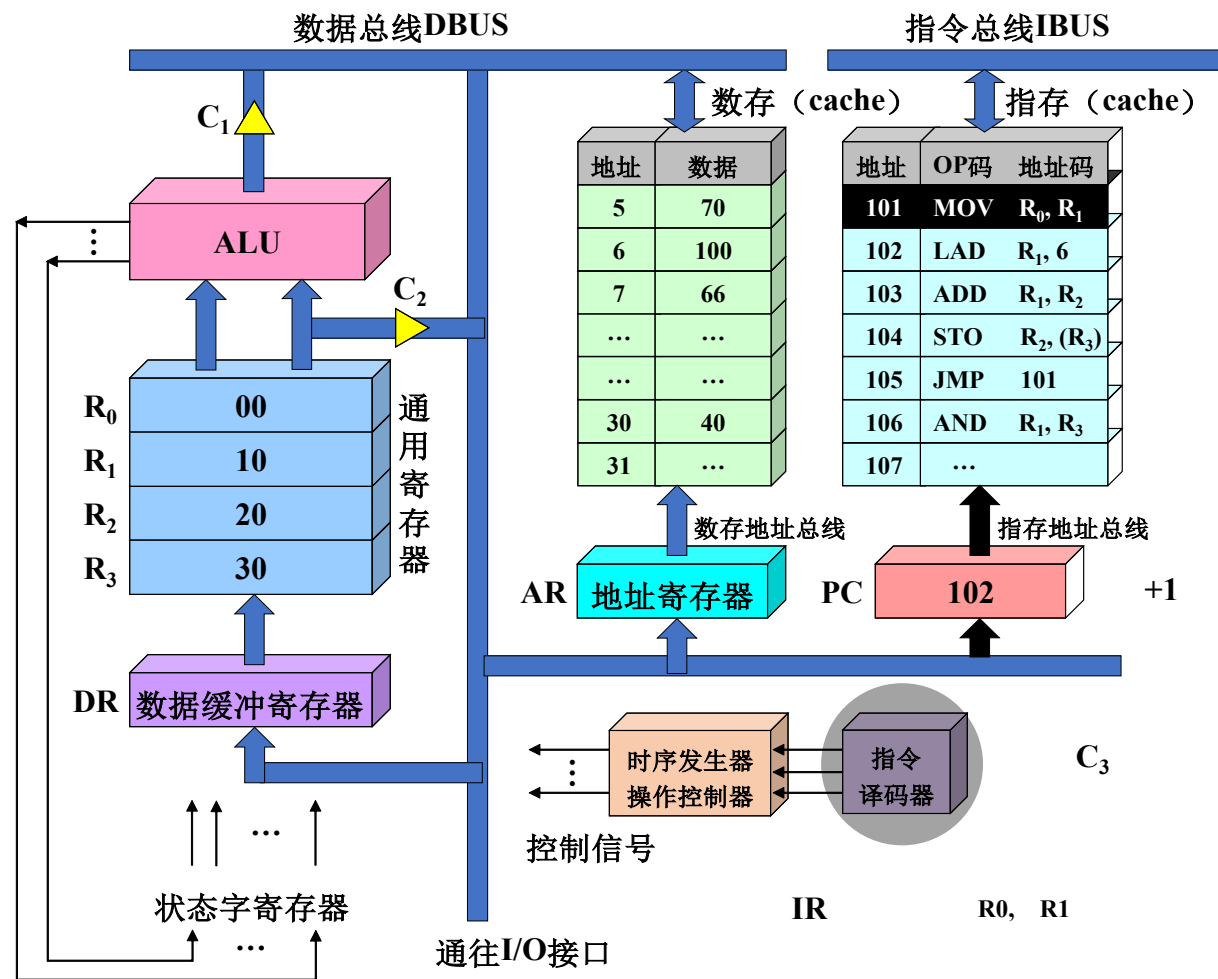
1.4. PC+1, 变成102,

为取下一条指令做准备

1.5. IR中的操作码 (OP) 被译码

1.6. 操作控制器OC识别出MOV指令

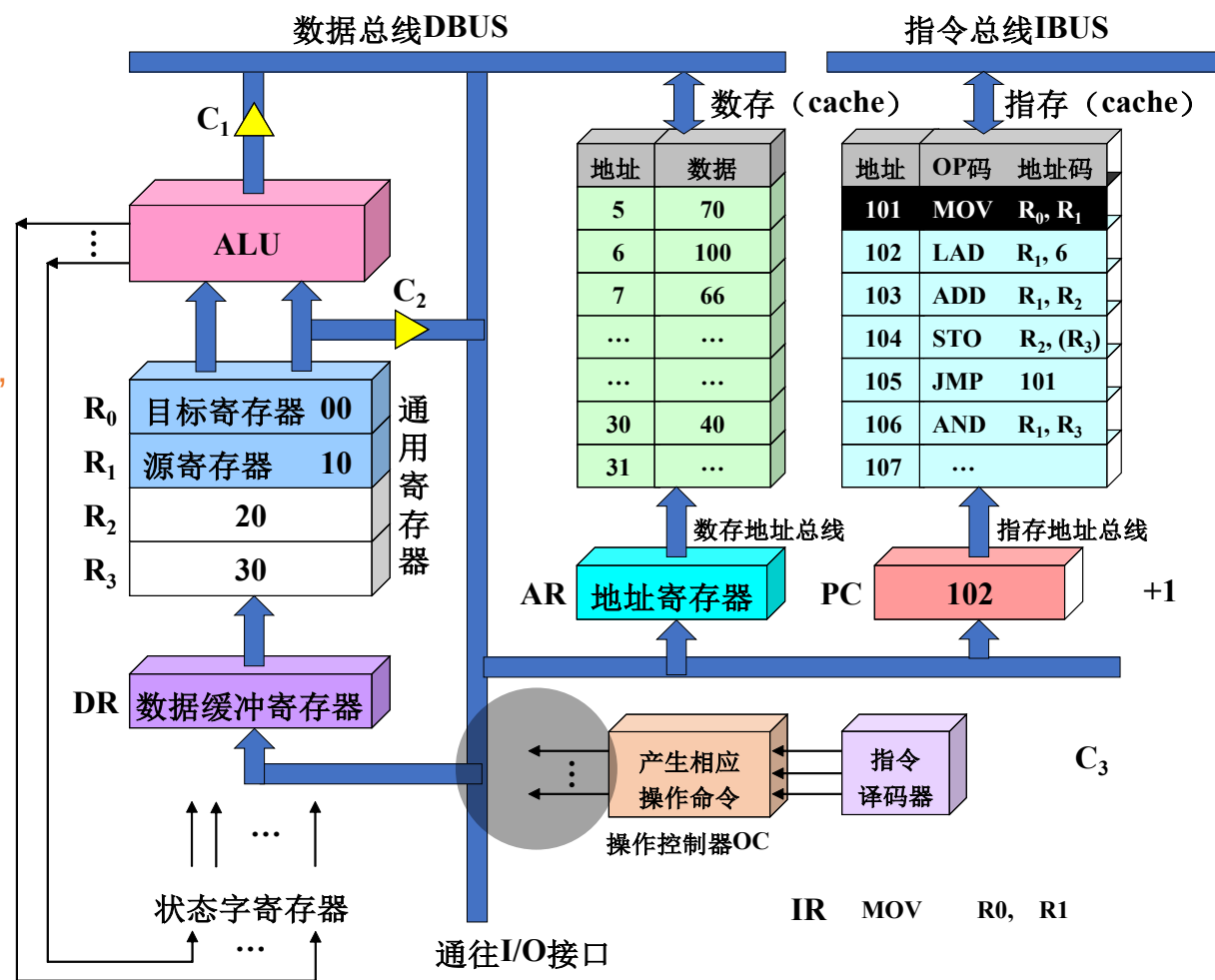
取指周期结束



# 早期IAS机设计

## 2. 执行周期

2.1. 操作控制器OC送出控制信号  
到通用寄存器,  
选择R1（内容为10）作源寄存器,  
选择R0作目标寄存器

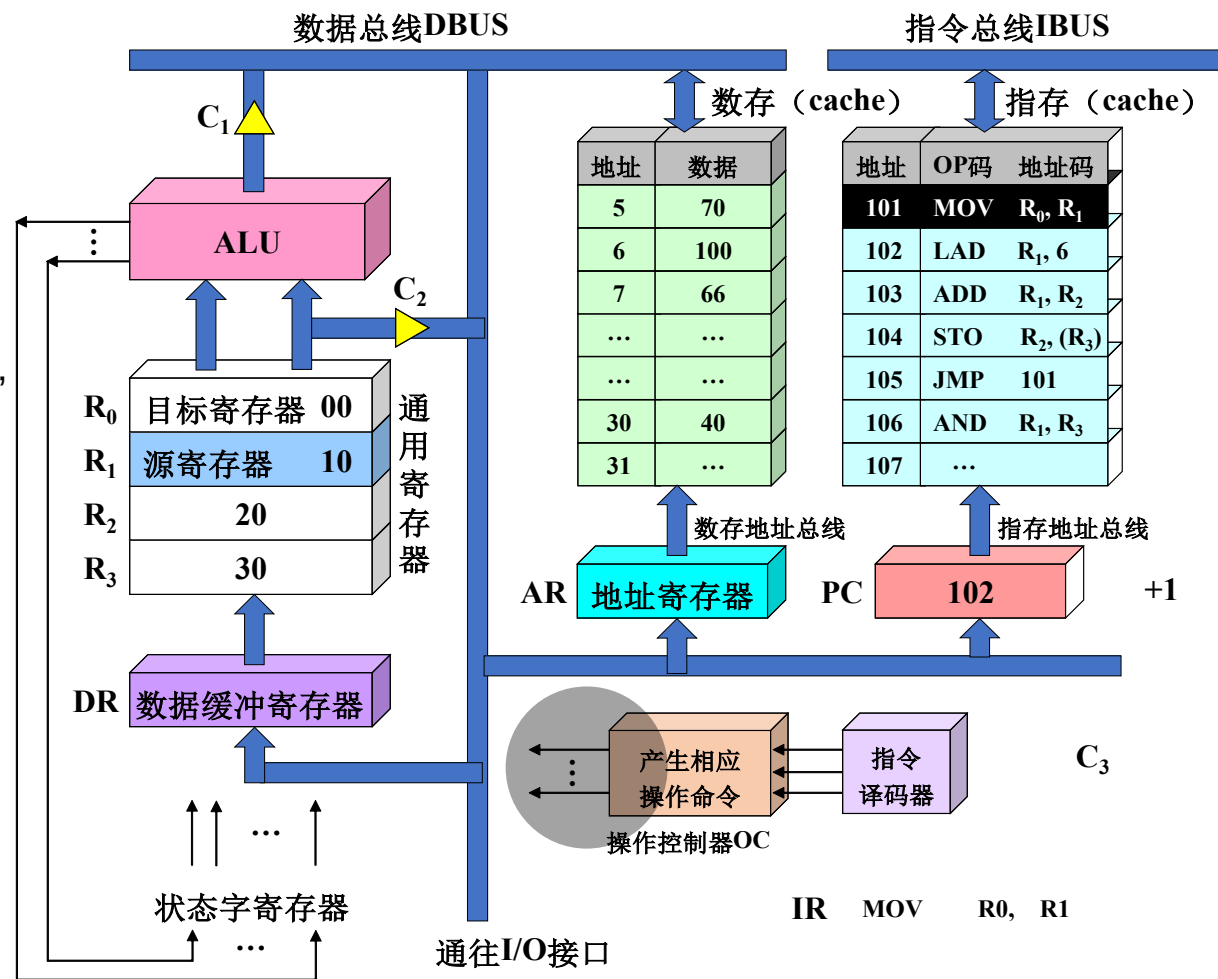


# 早期IAS机设计

## 2. 执行周期

2.1. 操作控制器OC送出控制信号  
到通用寄存器,  
选择R1 (内容为10) 作源寄存器,  
选择R0作目标寄存器

2.2. OC送出控制信号到ALU,  
指定ALU做传送操作



# 早期IAS机设计

## 2. 执行周期

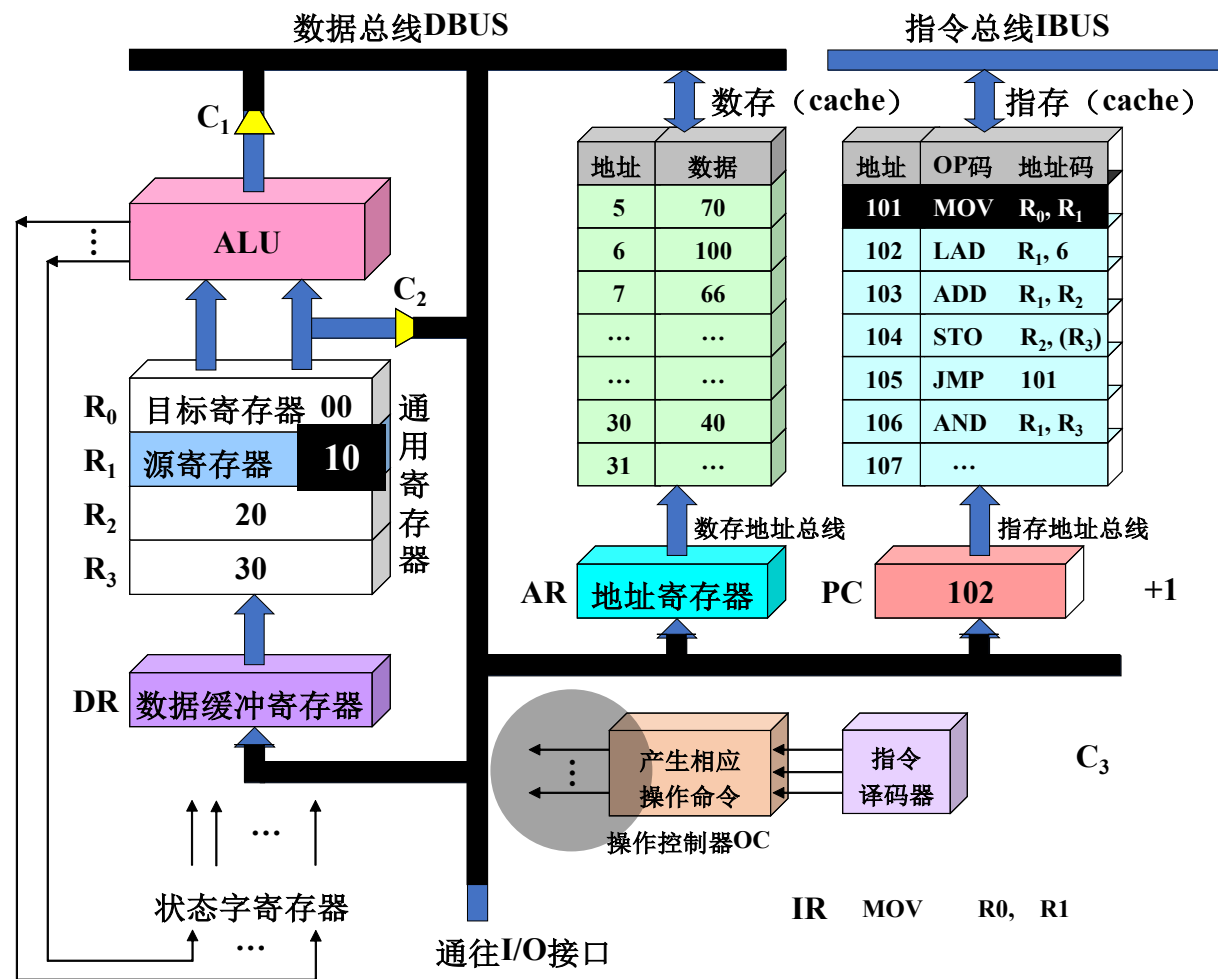
...

2.2. OC送出控制信号到ALU,  
指定ALU做传送操作

2.3. OC送出控制信号,  
打开ALU输出三态门。

将R1寄存器作为ALU输出  
送到数据总线DBUS上。

任意时刻DBUS上只能有一个数据



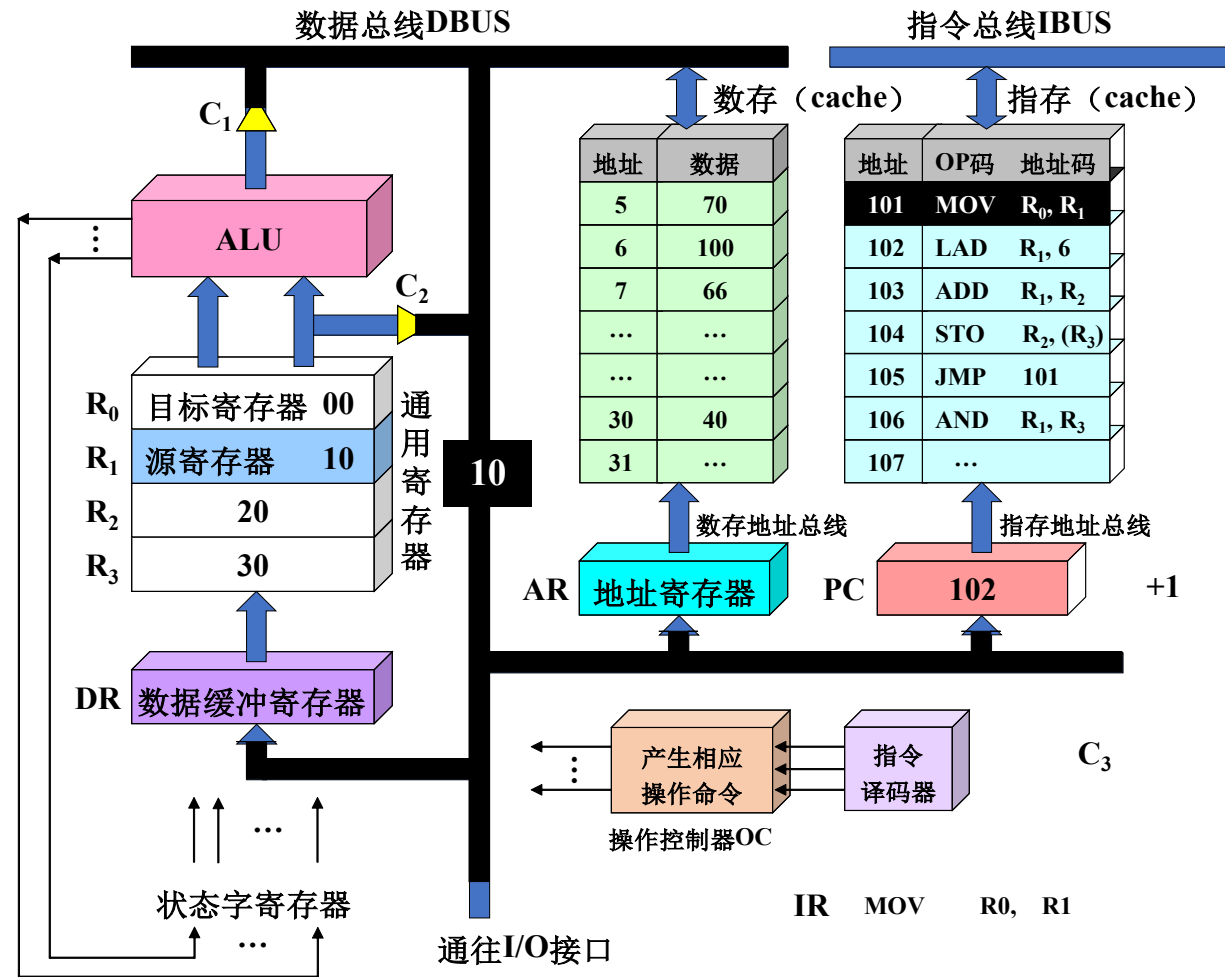
# 早期IAS机设计

## 2. 执行周期

...

2.3. OC送出控制信号，  
打开ALU输出三态门。  
将R1寄存器作为ALU输出  
送到数据总线DBUS上。

2.4. OC送出控制信号，将DBUS上的  
数据送入数据缓冲寄存器DR



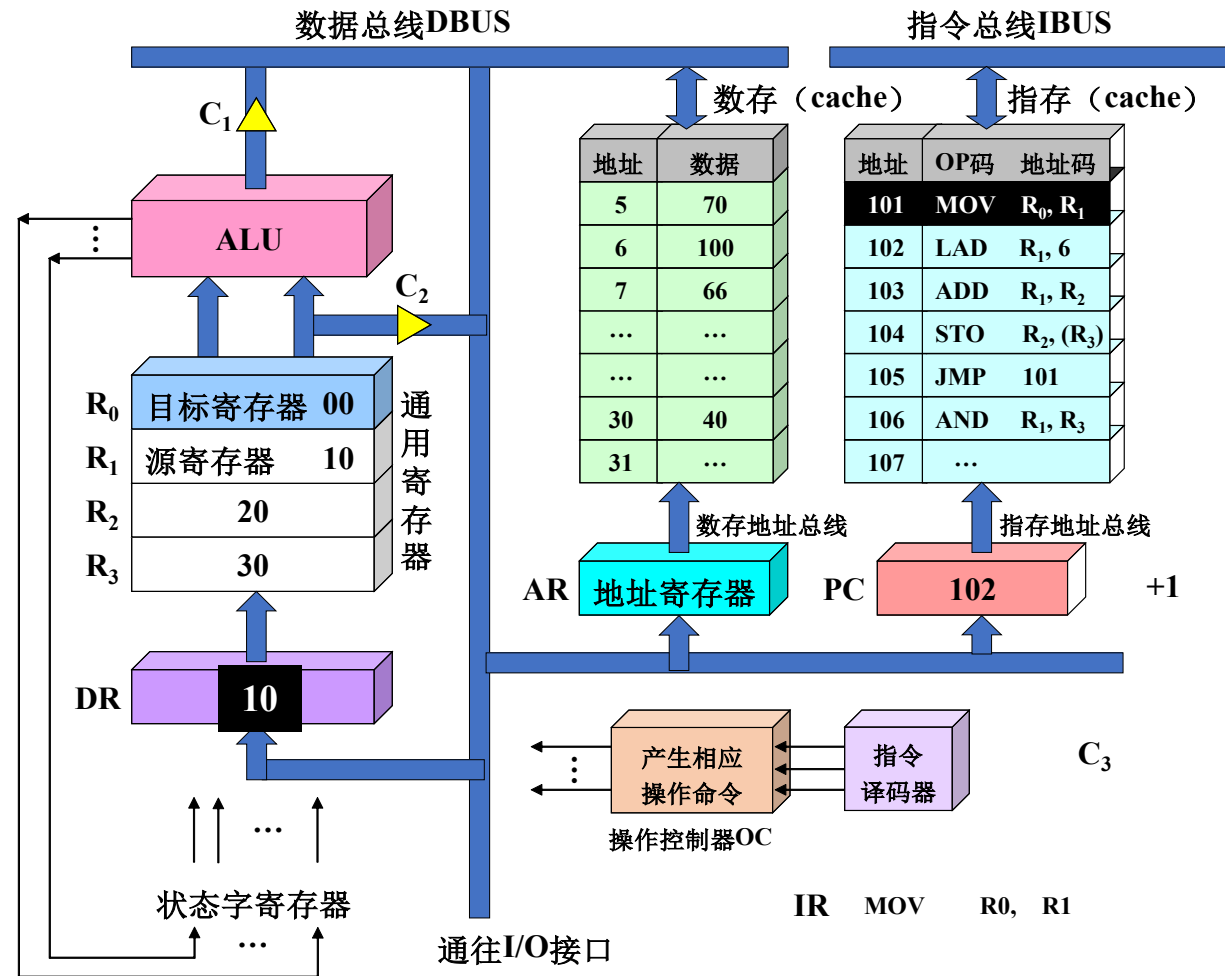
# 早期IAS机设计

## 2. 执行周期

...

2.4. OC送出控制信号，将DBUS上的数据送入数据缓冲寄存器DR

2.5. OC送出控制信号，将DR中的数据10送入目标寄存器R<sub>0</sub>，R<sub>0</sub>的内容由00变成10



# 早期IAS机设计

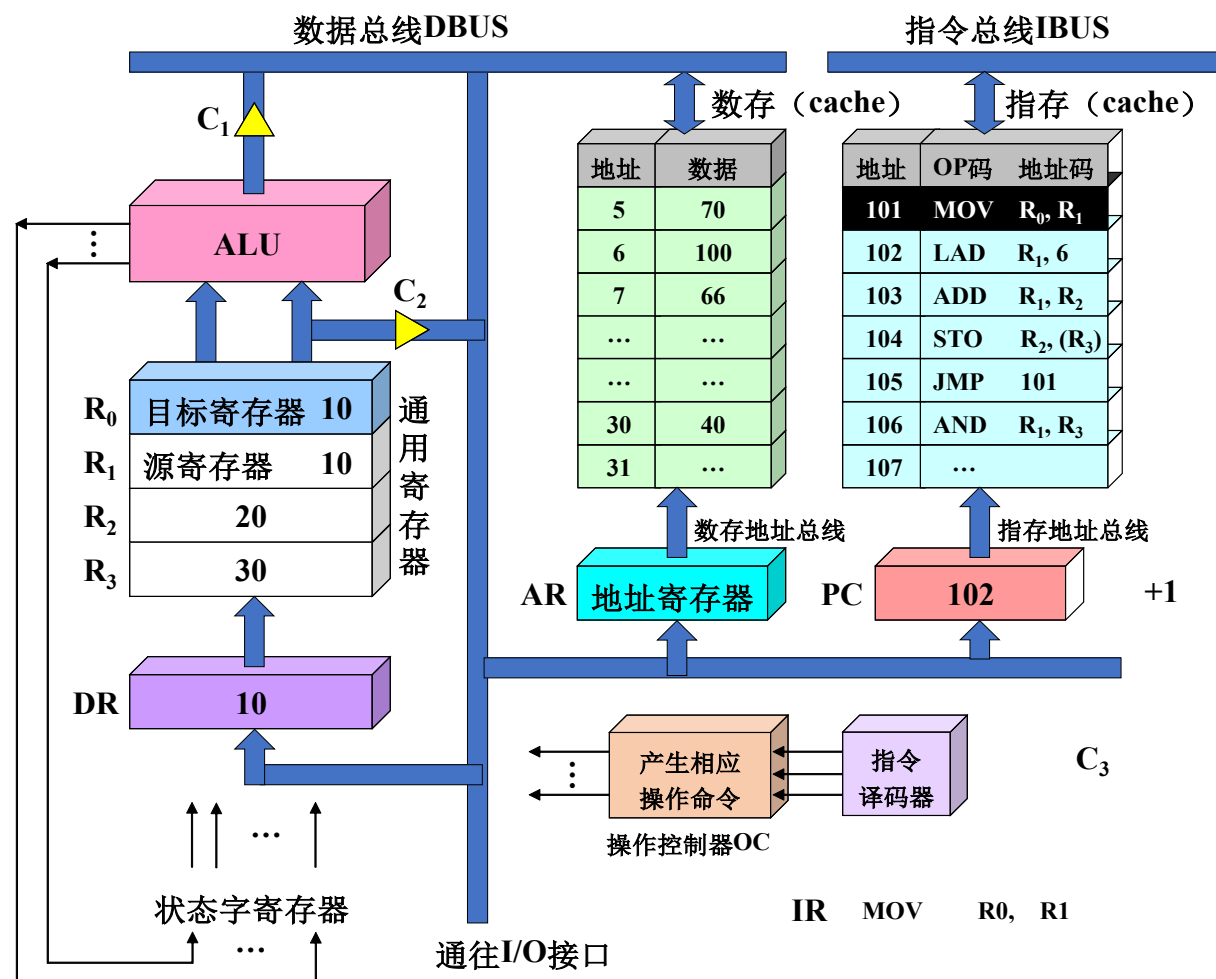
## 2. 执行周期

...

2.4. OC送出控制信号，将DBUS上的数据送入数据缓冲寄存器DR

2.5. OC送出控制信号，将DR中的数据10送入目标寄存器R<sub>0</sub>，R<sub>0</sub>的内容由00变成10

MOV指令执行结束

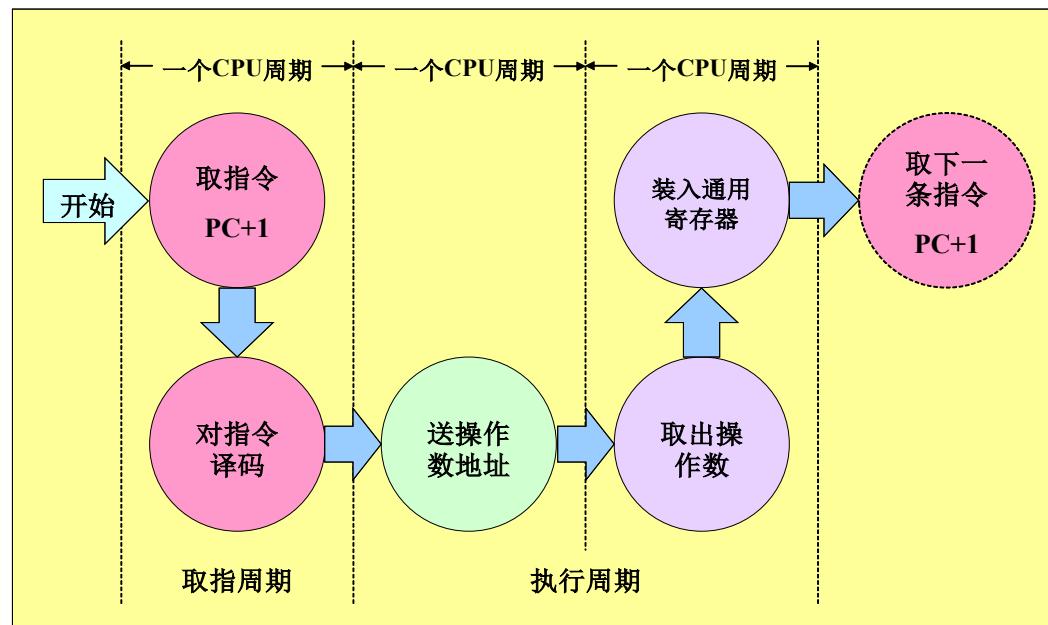


# 早期IAS机设计

- LAD指令的指令周期

- RS型指令，需要3个CPU周期

- 取指
- 计算有效地址
- 取数据装入寄存器

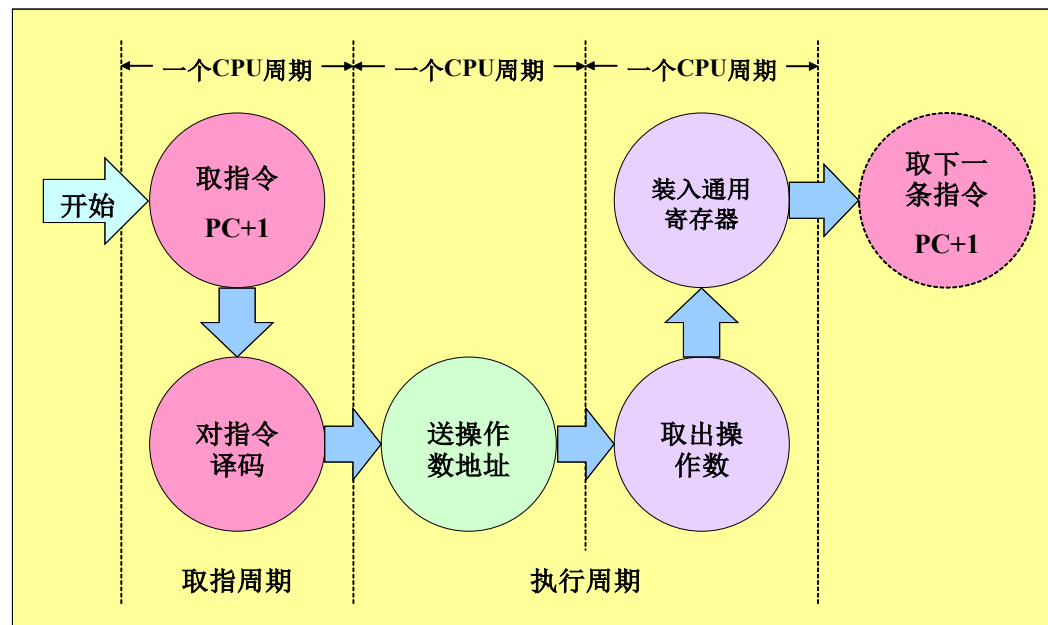


# 早期IAS机设计

- LAD指令的指令周期
  - RS型指令，需要3个CPU周期
  - 取指
  - 执行：计算有效地址
  - 执行：取数据装入寄存器

## 1. 取指周期

与MOV指令取指周期相同，  
只是PC提供的指令地址102，  
按此地址从指存中读出“LAD R1， 6”指令放入IR中。  
然后PC+1，使PC内容变为103，为取下一条ADD指令做准备。

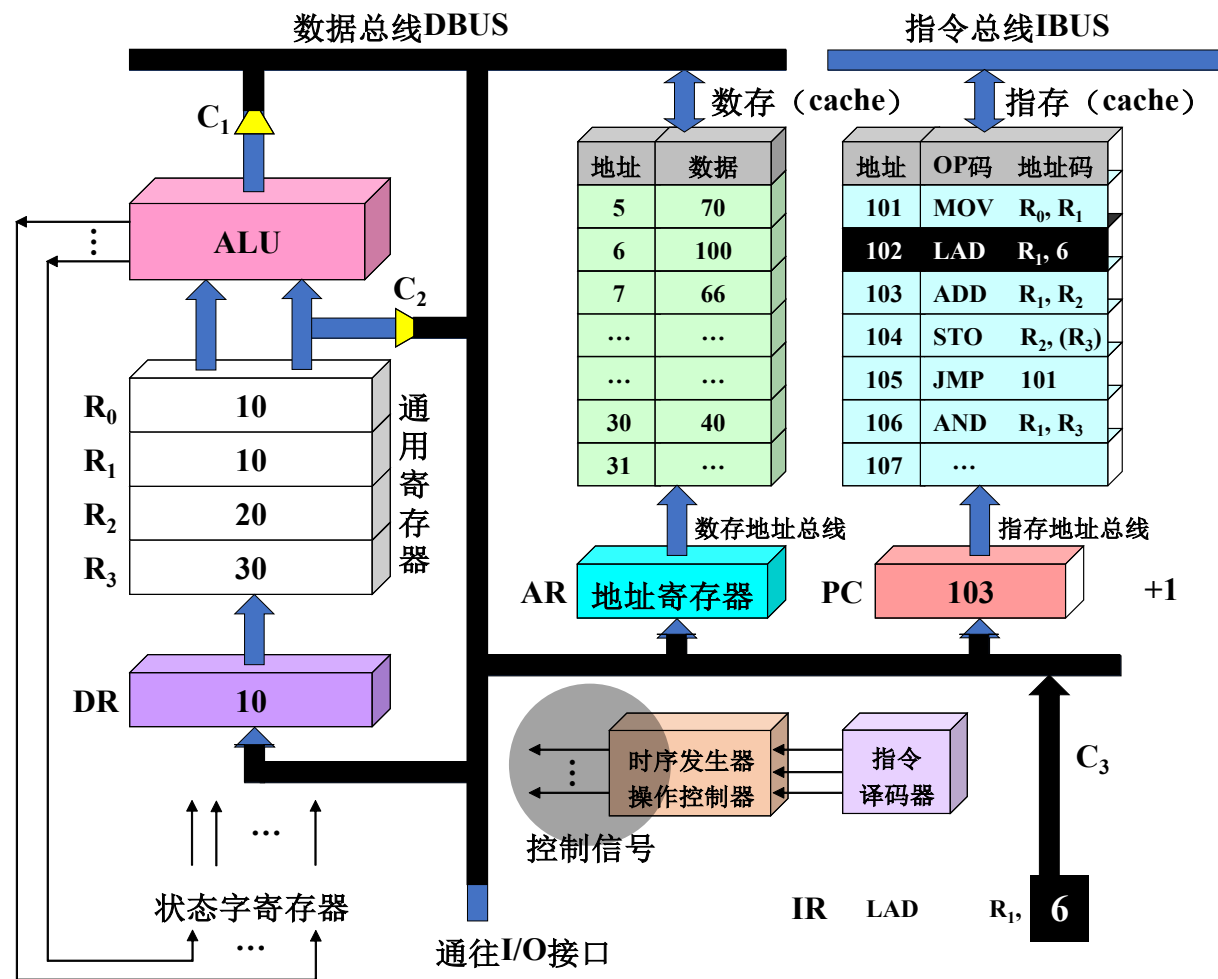


# 早期IAS机设计

## 2. 执行周期

### 2.1. 操作控制器OC发出控制命令 打开IR输出三态门。

将指令中的直接地址码6  
放到数据总线DBUS上。

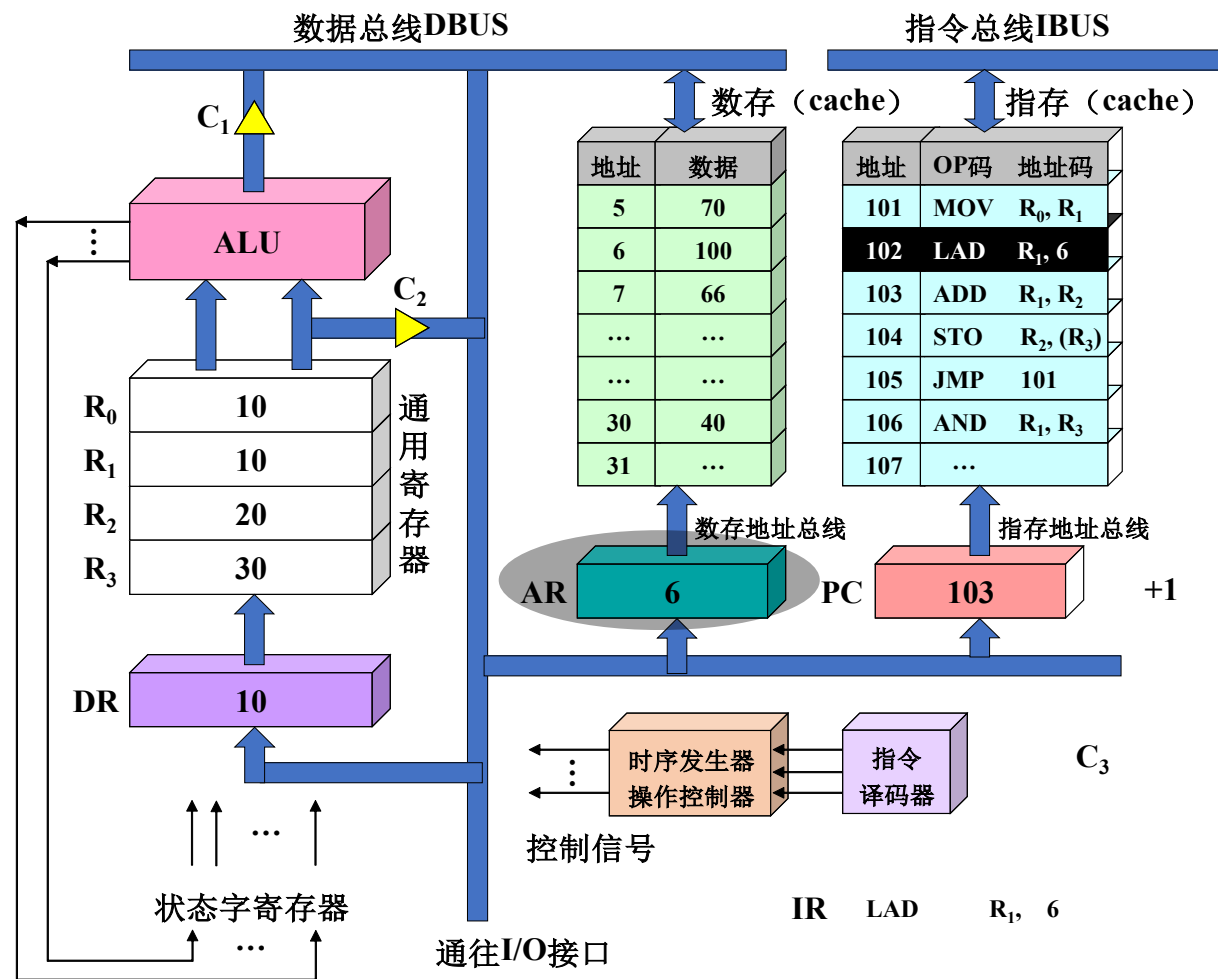


## 2. 执行周期

打开IR输出三态门。

将指令中的直接地址码6  
放到数据总线DBUS上。

## 2.2. OC发出操作命令，将地址码6 装入数据地址寄存器AR



# 早期IAS机设计

## 2. 执行周期

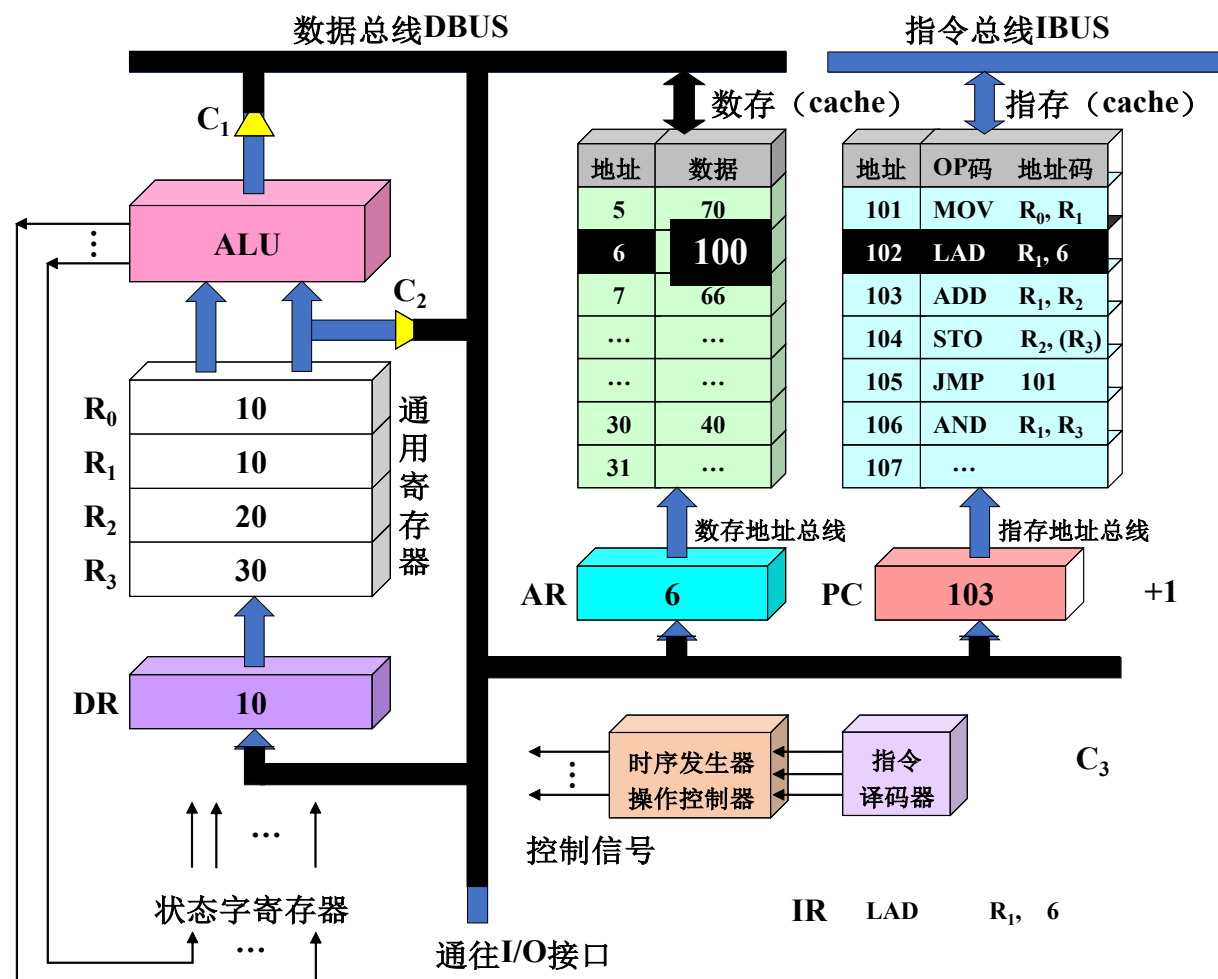
### 2.1. 操作控制器OC发出控制命令

打开IR输出三态门。

将指令中的直接地址码6  
放到数据总线DBUS上。

### 2.2. OC发出操作命令，将地址码6 装入数据地址寄存器AR

### 2.3. OC发出读命令，将数存6号单元 中的数100读出到DBUS



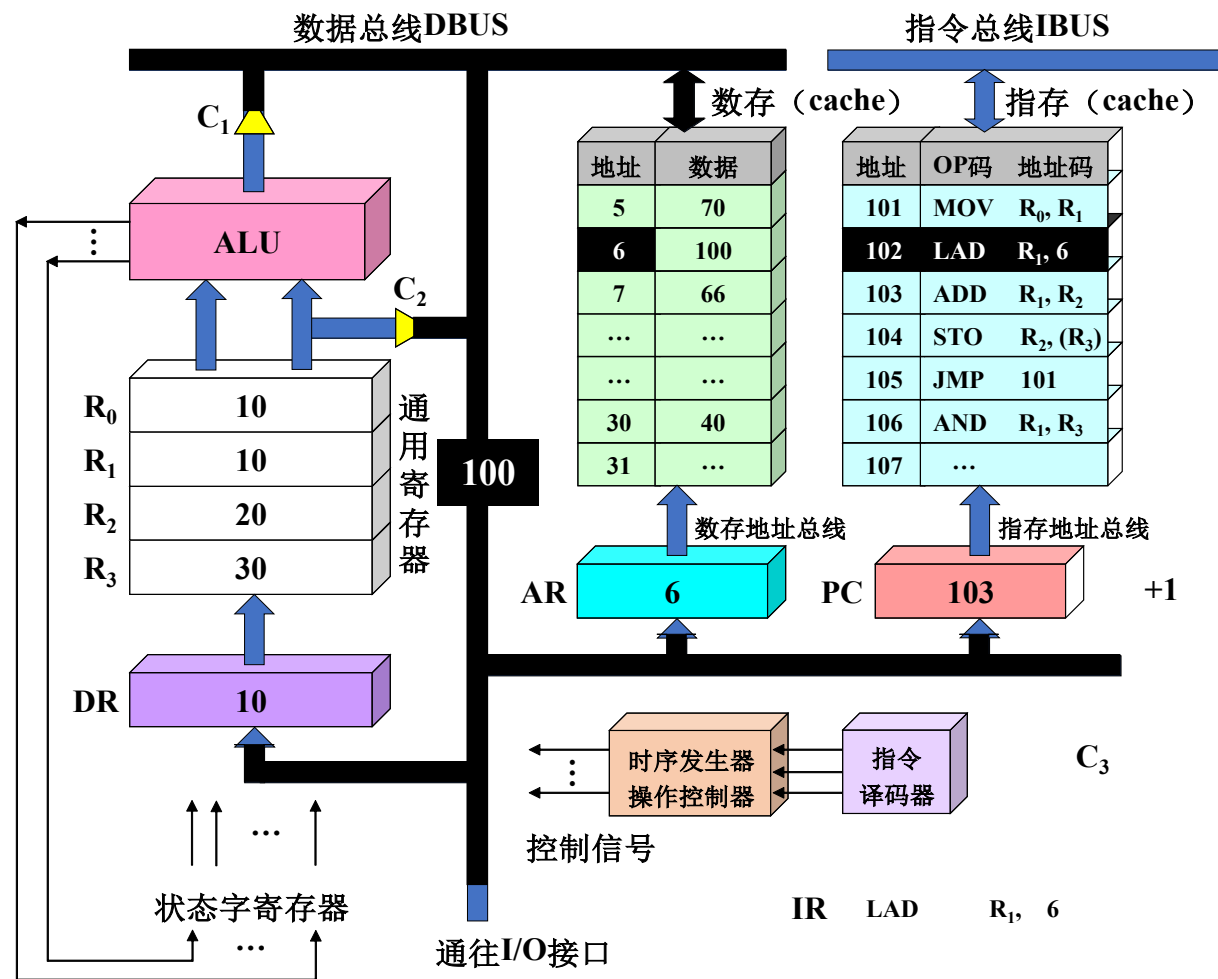
# 早期IAS机设计

## 2. 执行周期

...

2.3. OC发出读命令，将数存6号单元中的数100读出到DBUS

2.4. OC发出命令，将DBUS上的数据100装入数据缓冲寄存器DR



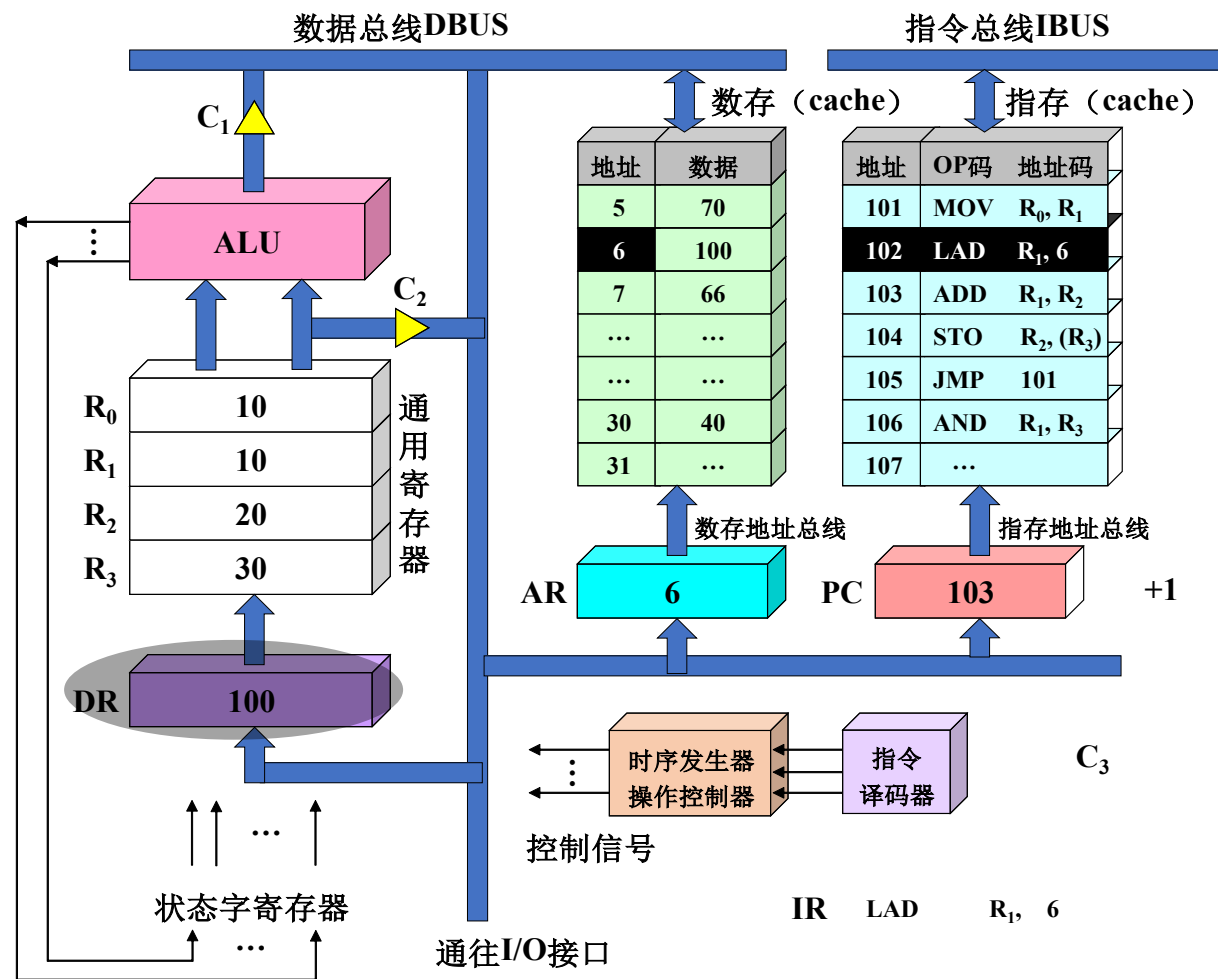
# 早期IAS机设计

## 2. 执行周期

...

2.3. OC发出读命令，将数存6号单元中的数100读出到DBUS

2.4. OC发出命令，将DBUS上的数据100装入数据缓冲寄存器DR



# 早期IAS机设计

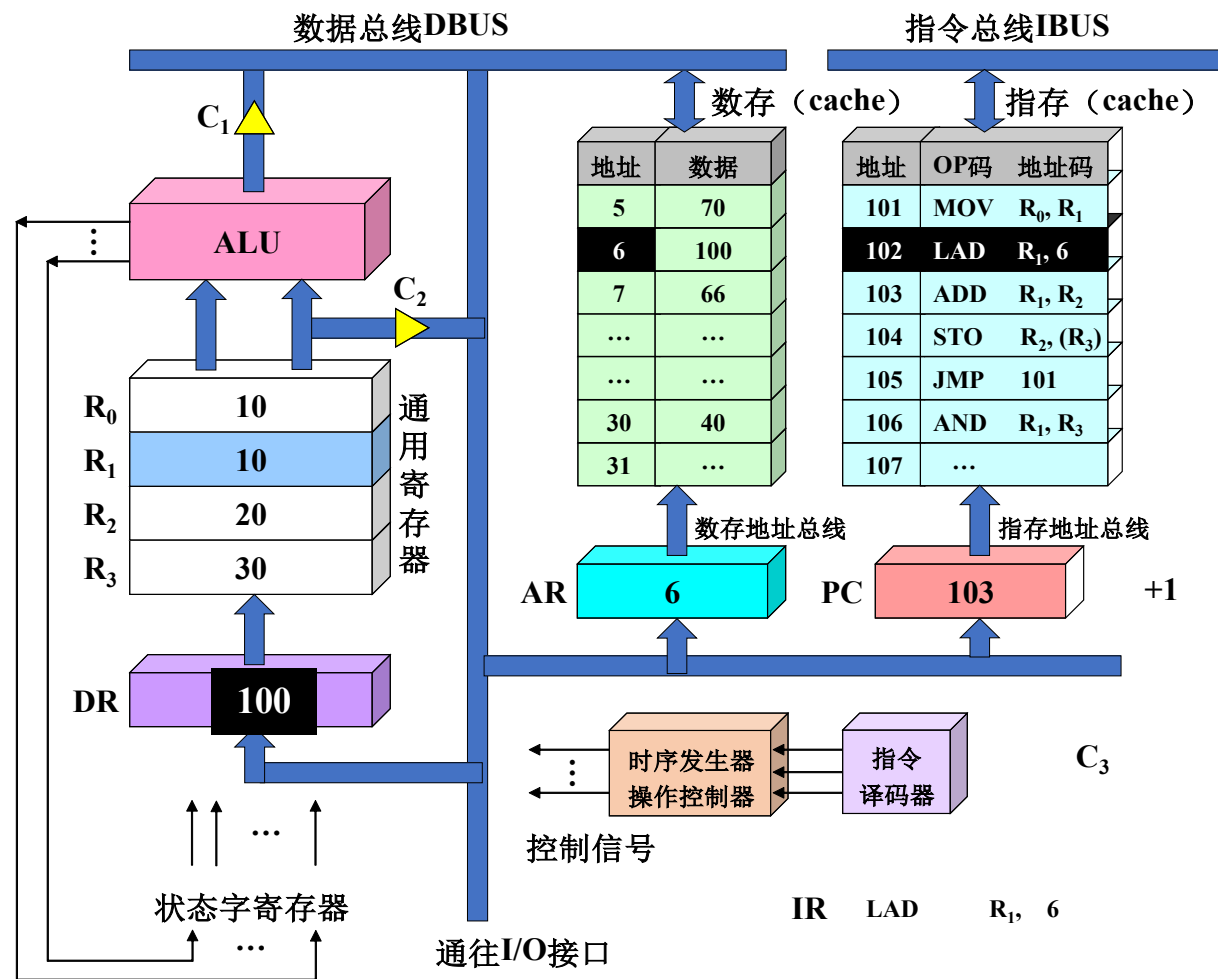
## 2. 执行周期

...

2.3. OC发出读命令，将数存6号单元中的数100读出到DBUS

2.4. OC发出命令，将DBUS上的数据100装入数据缓冲寄存器DR

2.5. OC发出命令，将DR中的数100装入通用寄存器R1，覆盖掉R1中原来的数10。



# 早期IAS机设计

## 2. 执行周期

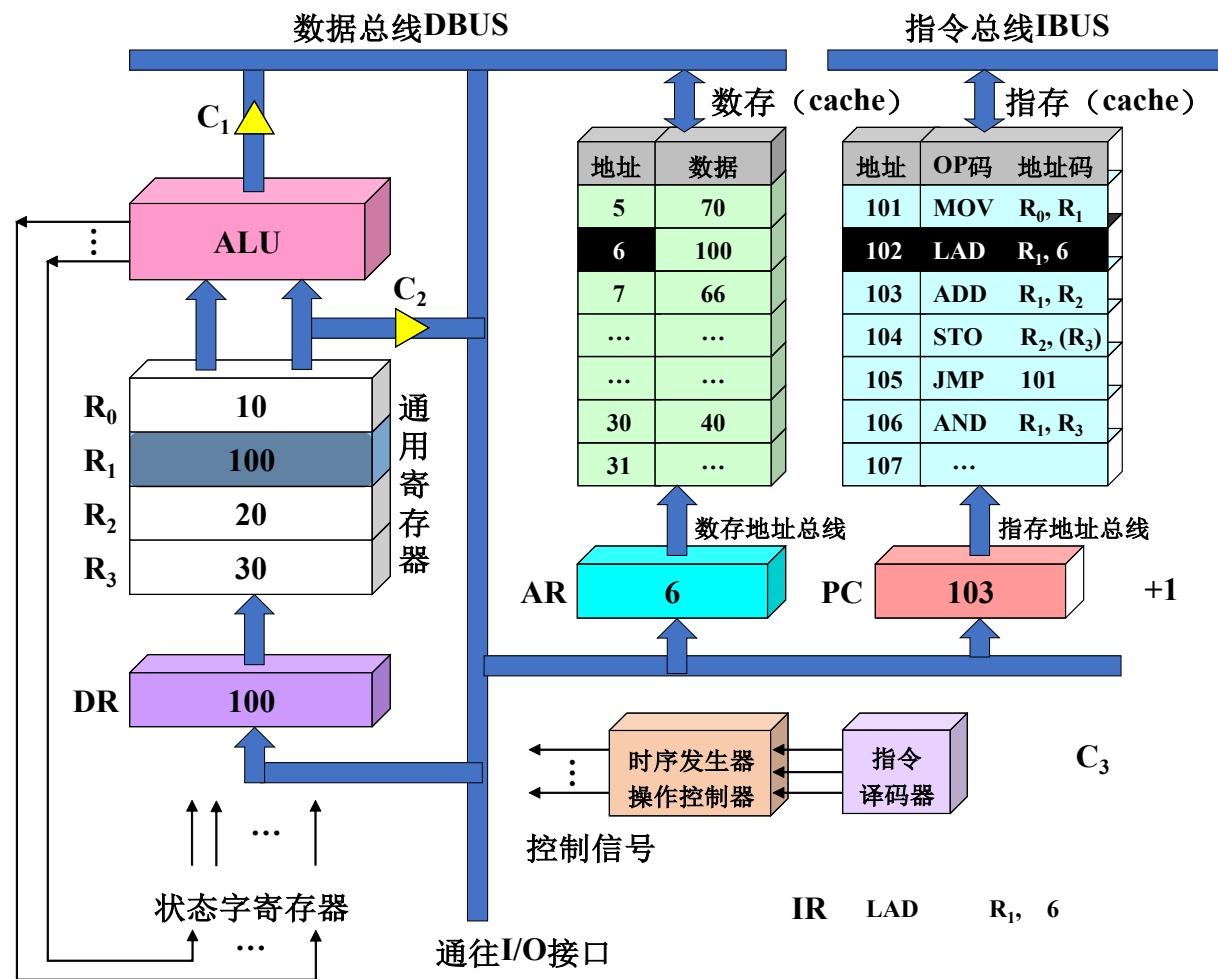
...

2.3. OC发出读命令，将数存6号单元中的数100读出到DBUS

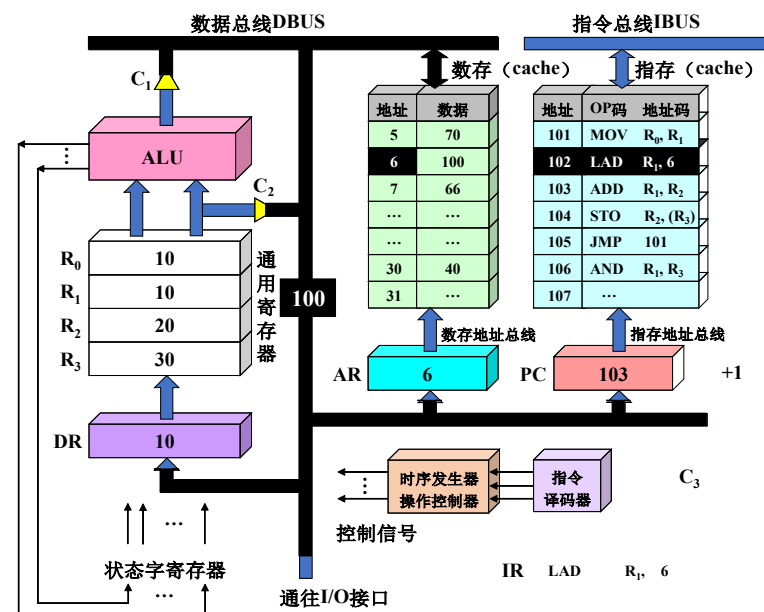
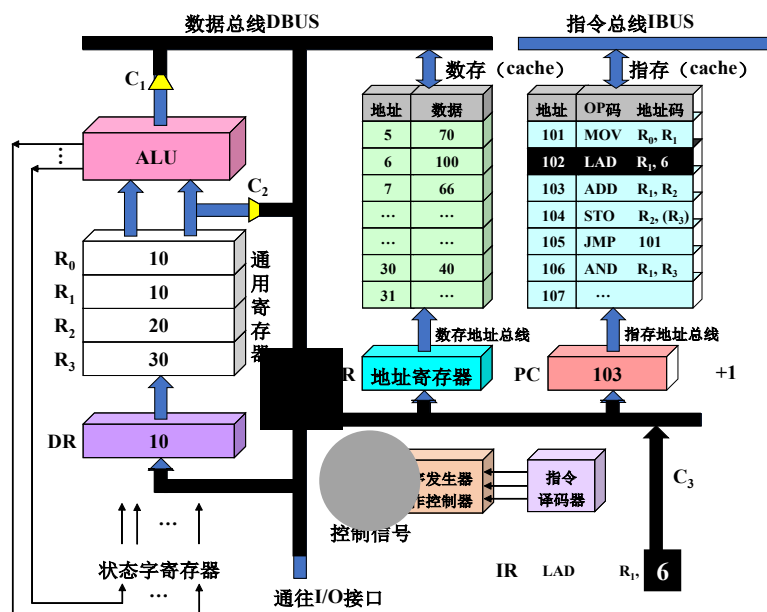
2.4. OC发出命令，将DBUS上的数据100装入数据缓冲寄存器DR

2.5. OC发出命令，将DR中的数100装入通用寄存器R1，覆盖掉R1中原来的数10。

LAD指令执行周期结束



# 早期IAS机设计



**数据总线DBUS上分时进行了地址传送和数据传送，所以执行需要2个CPU周期。**

# 早期IAS机设计

- **ADD指令的指令周期**

- RR型指令，在运算器中用两个寄存器R1和R2的数据进行加法运算
- 需要两个CPU周期
  - 取指
  - 执行

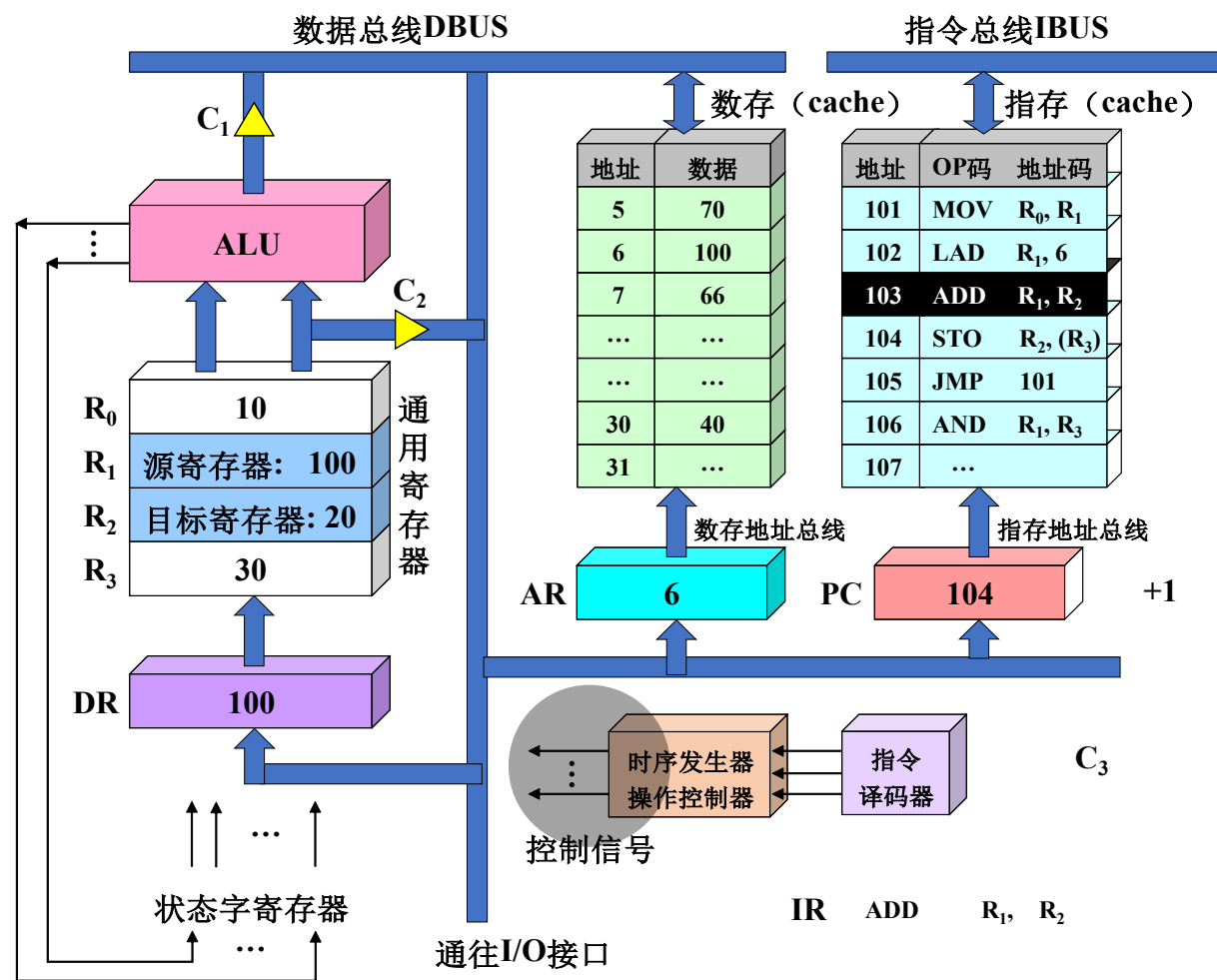
## 1. 取指周期

CPU动作同MOV、LAD指令。PC提供指令地址103，按此地址从指存中读出“ADD R1, R2”指令放入IR；然后PC+1，使PC内容变为104，为取下一条STO指令做准备。

# 早期IAS机设计

## 2. 执行周期

2.1. 操作控制器OC送出控制命令  
到通用寄存器,  
选择R1做源寄存器,  
R2做目标寄存器



# 早期IAS机设计

## 2. 执行周期

### 2.1. 操作控制器OC送出控制命令

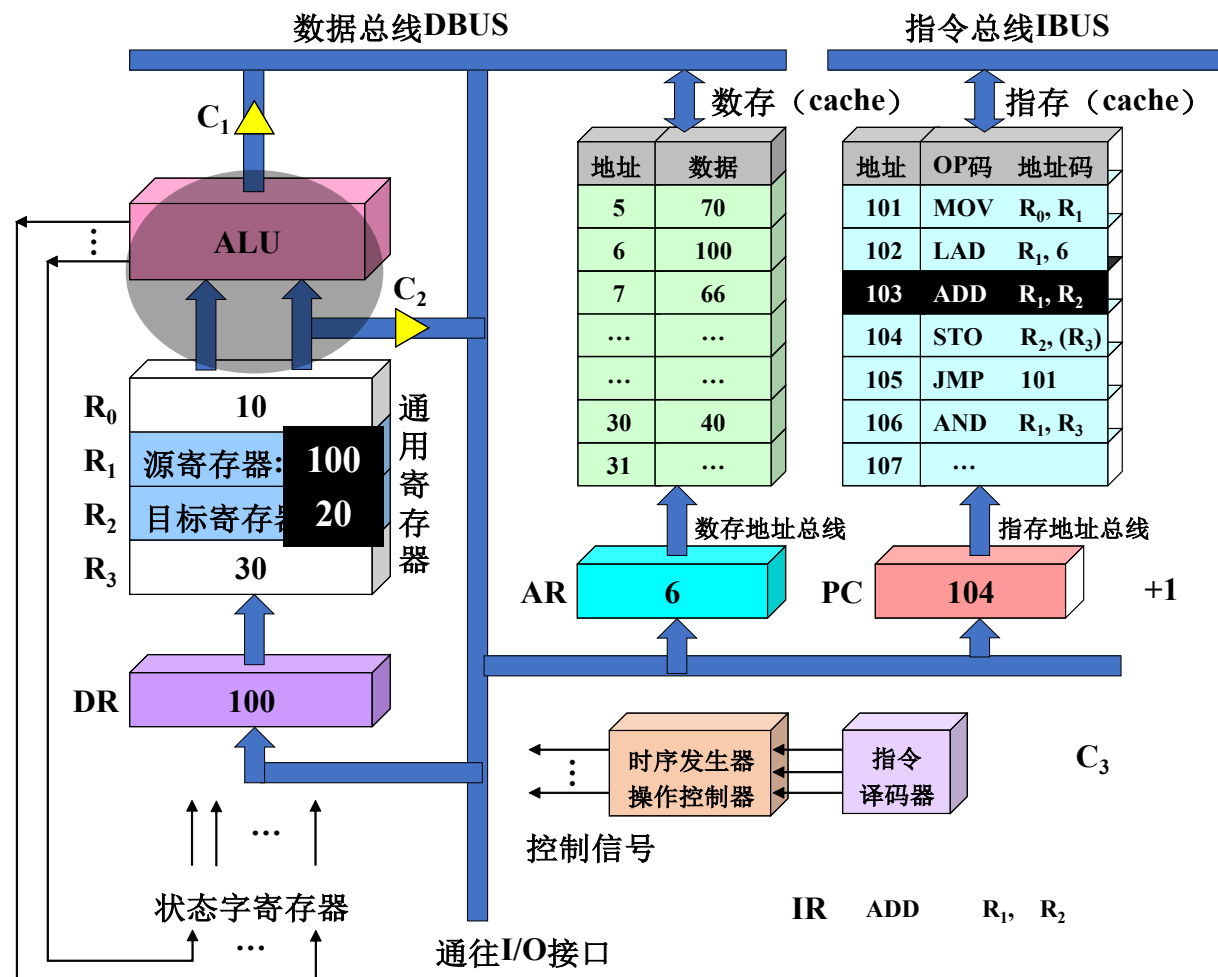
到通用寄存器,

选择R1做源寄存器,

R2做目标寄存器

### 2.2. OC送出控制命令到ALU,

指定ALU做R1和R2的加法运算



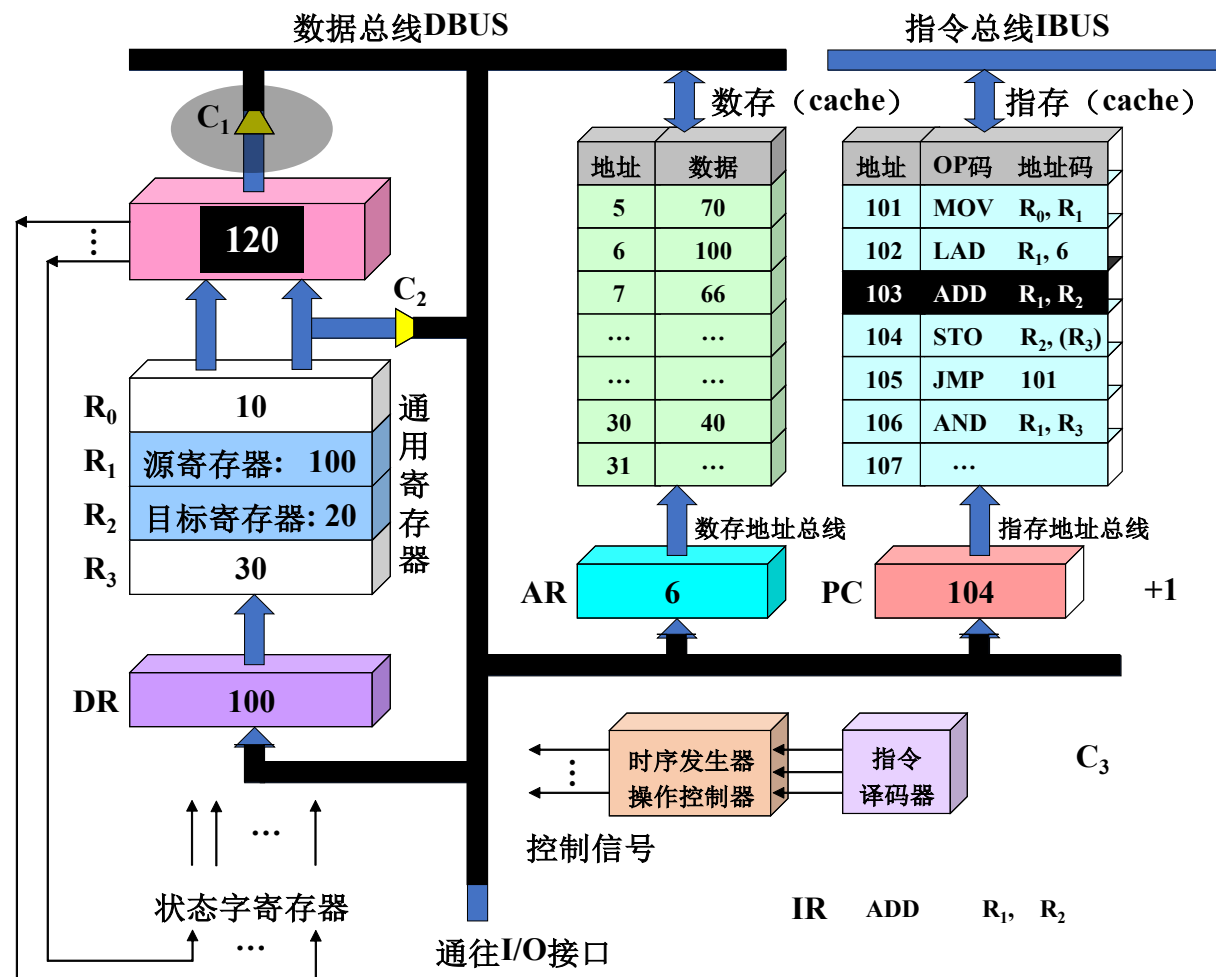
# 早期IAS机设计

## 2. 执行周期

...

2.2. OC送出控制命令到ALU,  
指定ALU做R1和R2的加法运算

2.3. OC送出控制命令,  
打开ALU三态门,  
运算结果120放到DBUS



# 早期IAS机设计

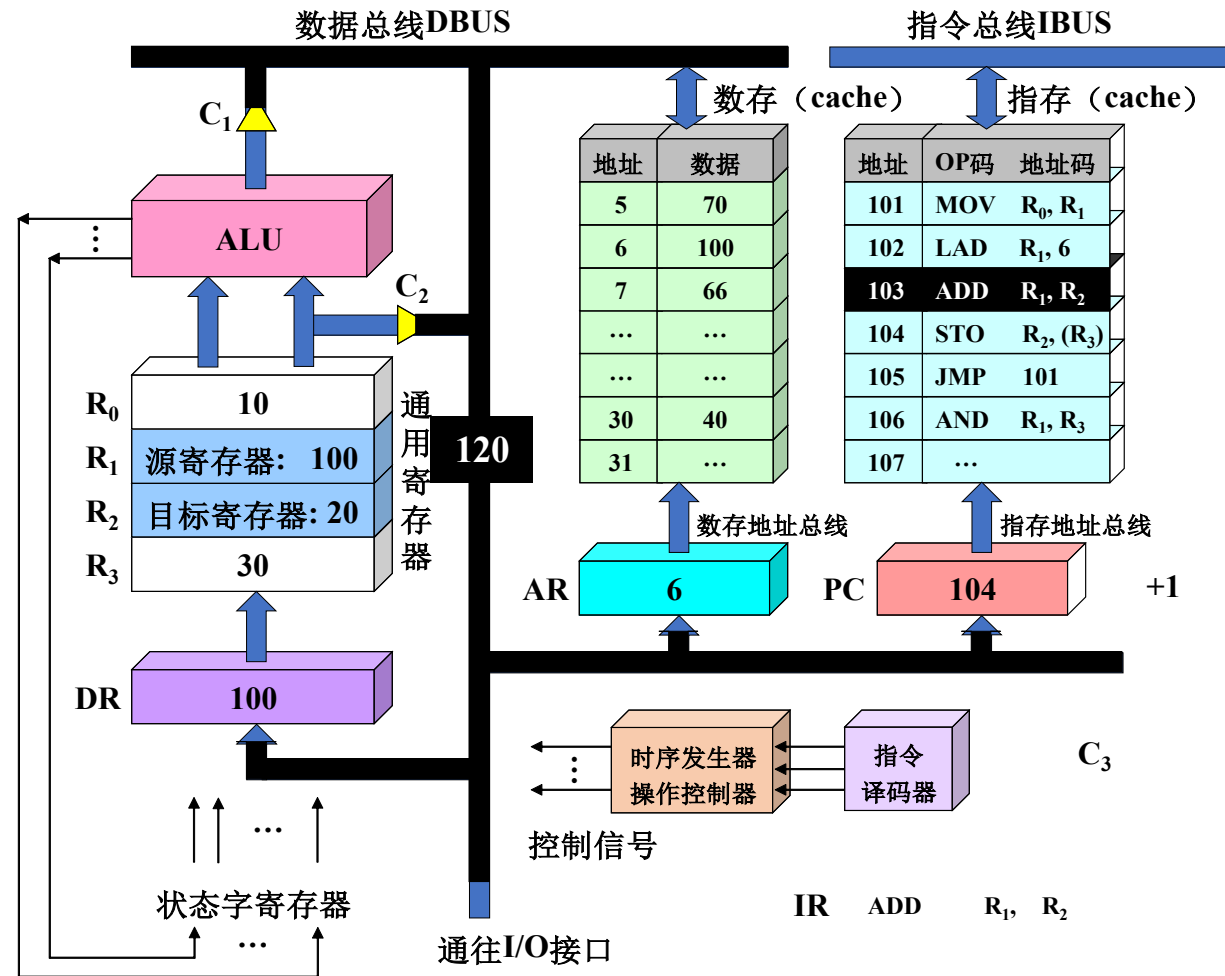
## 2. 执行周期

...

2.2. OC送出控制命令到ALU,  
指定ALU做R1和R2的加法运算

2.3. OC送出控制命令,  
打开ALU三态门,  
运算结果120放到DBUS

2.4. OC送出控制命令, 将DBUS上的  
数据送入数据缓冲寄存器DR



# 早期IAS机设计

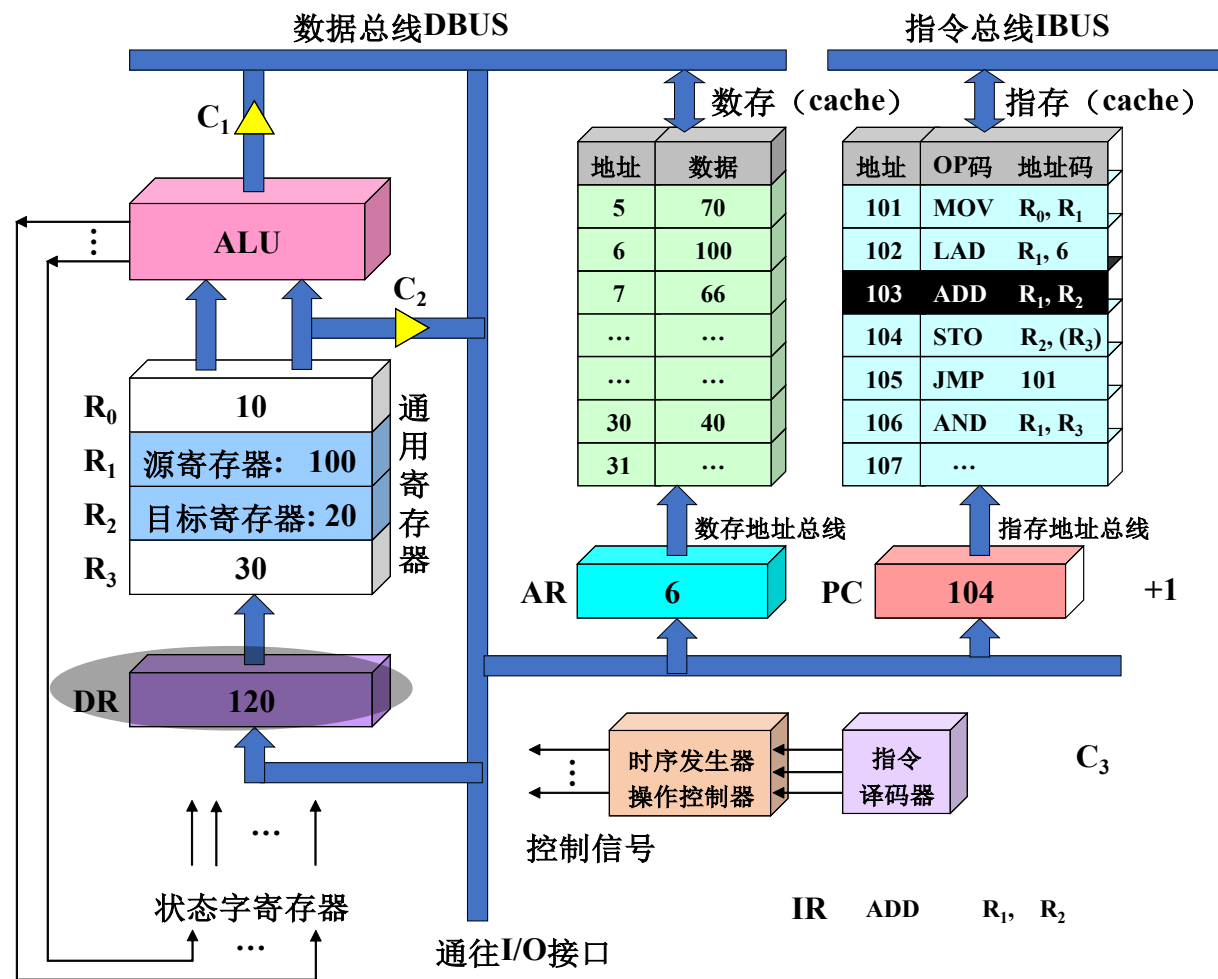
## 2. 执行周期

...

2.2. OC送出控制命令到ALU,  
指定ALU做R1和R2的加法运算

2.3. OC送出控制命令,  
打开ALU三态门,  
运算结果120放到DBUS

2.4. OC送出控制命令, 将DBUS上的  
数据送入数据缓冲寄存器DR



# 早期IAS机设计

## 2. 执行周期

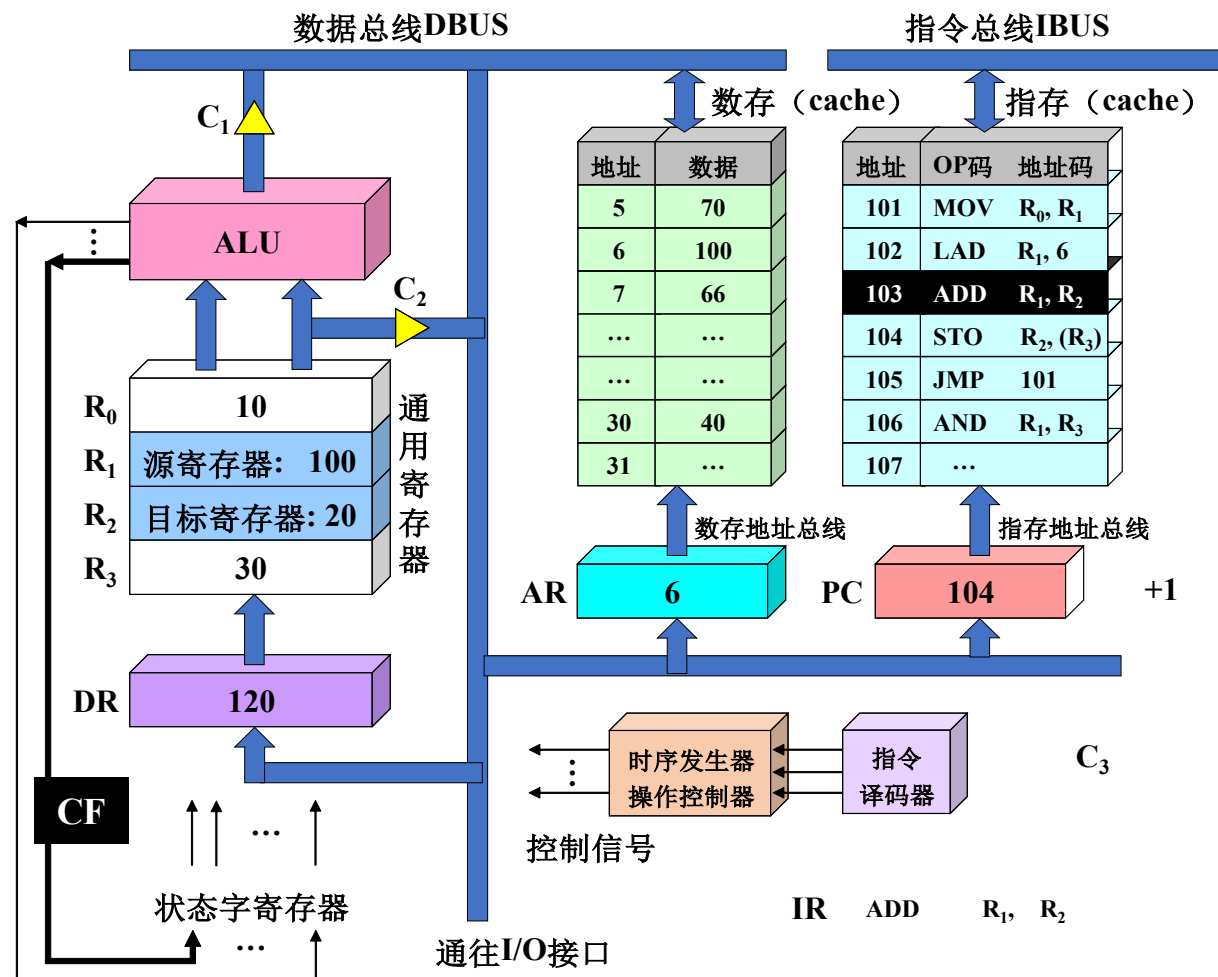
...

2.2. OC送出控制命令到ALU,  
指定ALU做R1和R2的加法运算

2.3. OC送出控制命令,  
打开ALU三态门,  
运算结果120放到DBUS

2.4. OC送出控制命令, 将DBUS上的  
数据送入数据缓冲寄存器DR

将ALU产生的进位信号保存在  
状态字寄存器PSW



# 早期IAS机设计

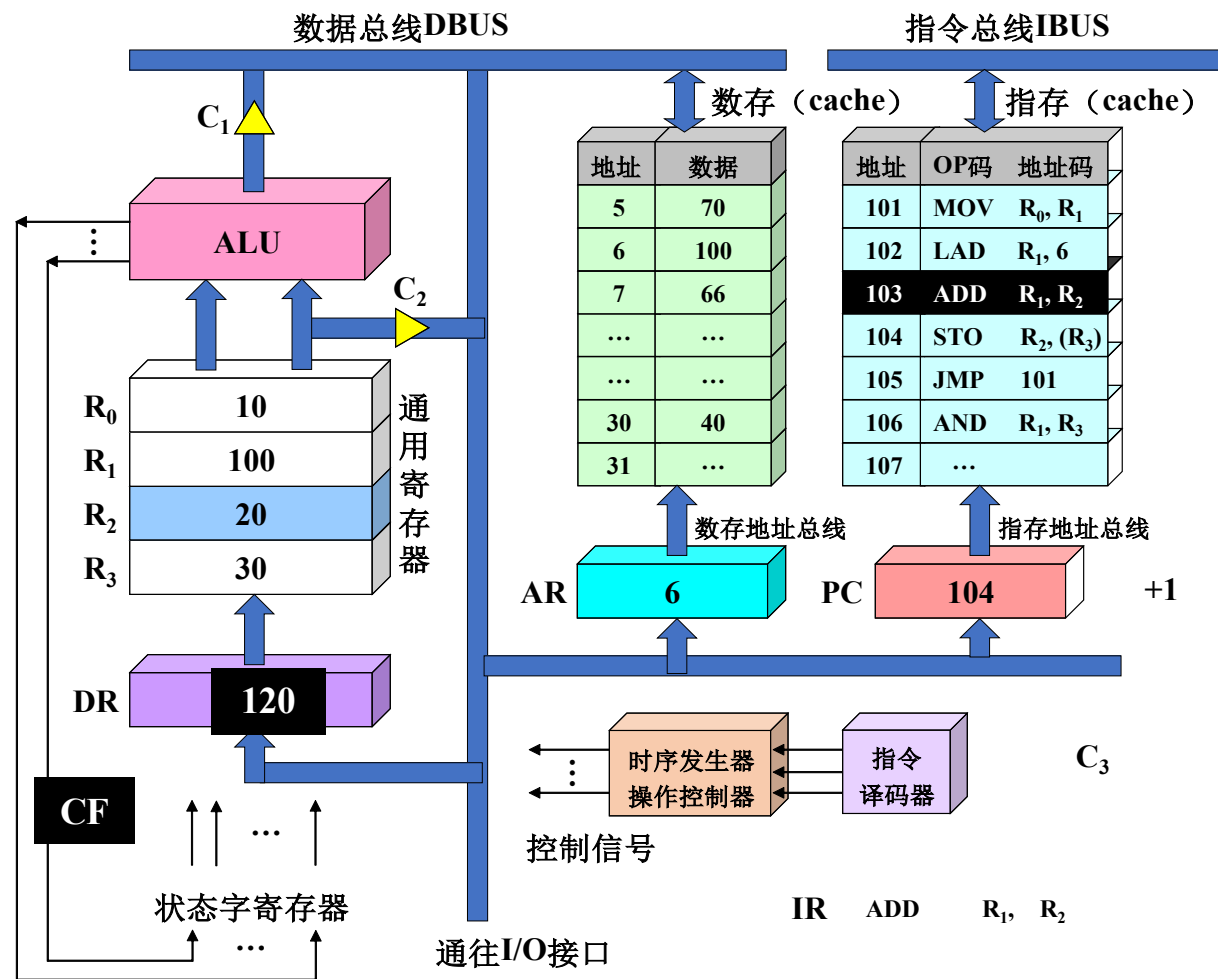
## 2. 执行周期

...

2.4. OC送出控制命令，将DBUS上的数据送入数据缓冲寄存器DR

将ALU产生的进位信号保存在状态字寄存器PSW

2.5. OC送出控制命令，  
将DR (120) 装入R2，  
R2中原来的内容20被冲掉



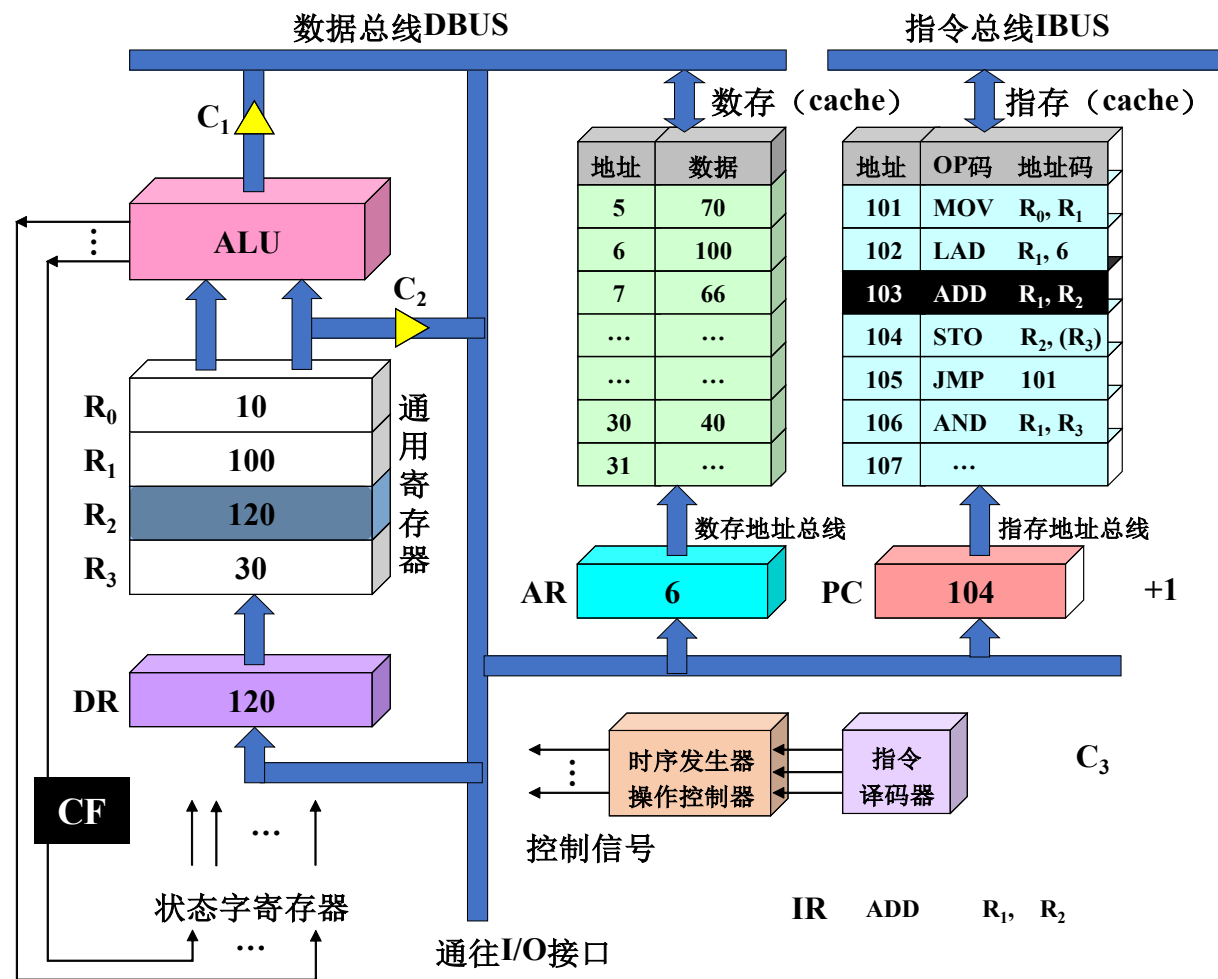
# 早期IAS机设计

## 2. 执行周期

...

2.5. OC送出控制命令,  
将DR (120) 装入R2,  
R2中原来的内容20被冲掉

ADD指令执行周期结束



# 早期IAS机设计

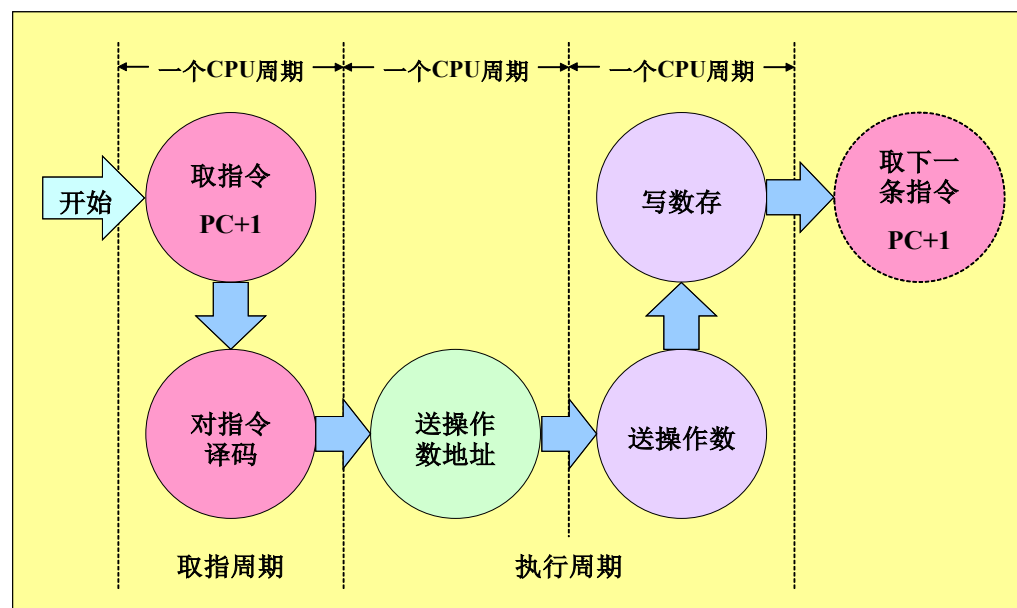
## • STO指令的指令周期

- RS型指令，需要3个CPU周期

- 取指：先访问指存取出STO指令
- 执行：计算有效地址
- 执行：访问数存写数据

### 1. 取指周期

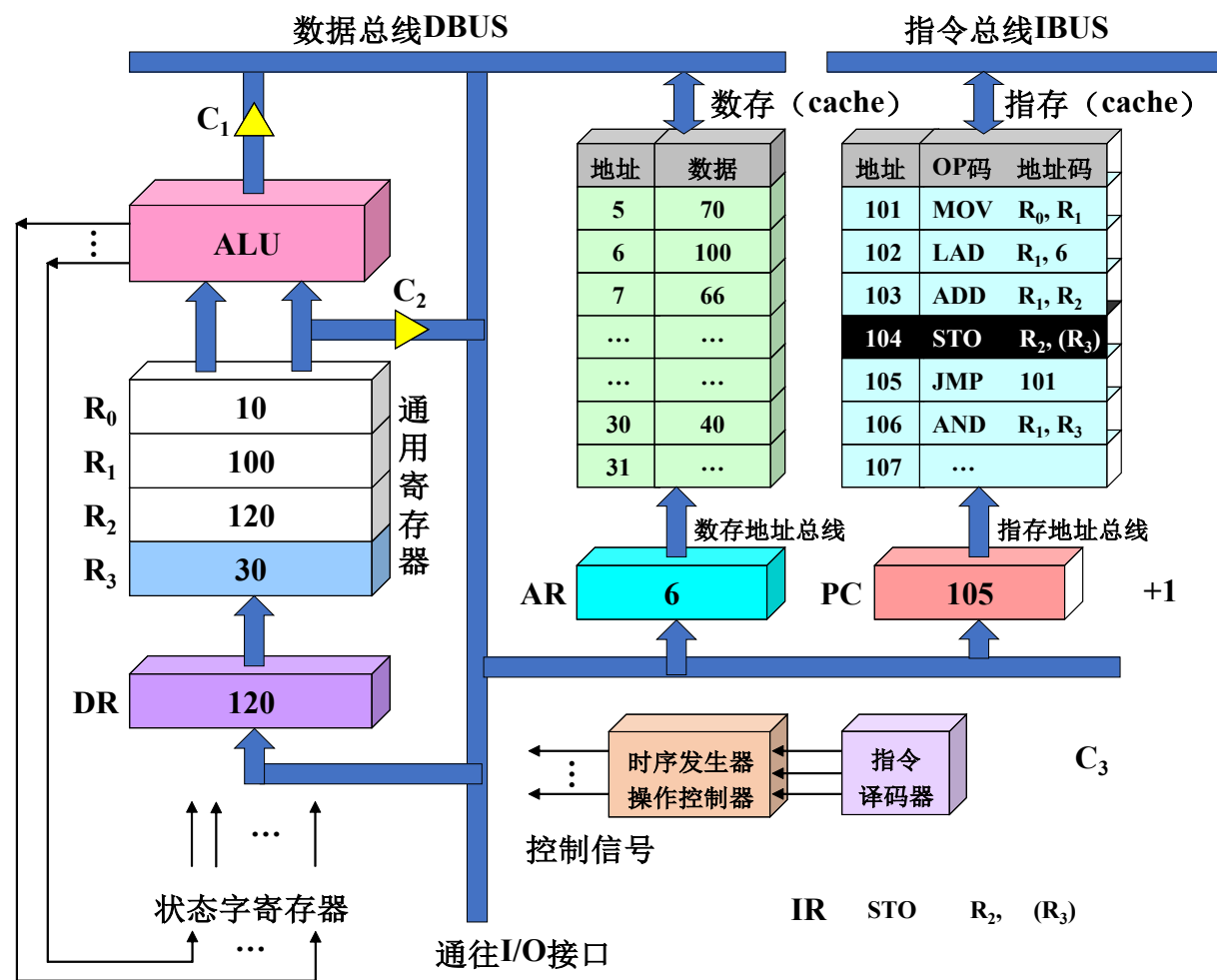
同MOV、LAD、ADD指令



# 早期IAS机设计

## 2. 执行周期

2.1. 操作控制器OC送出控制命令  
到通用寄存器,  
选择(R3)=30做  
数据存储器访问地址

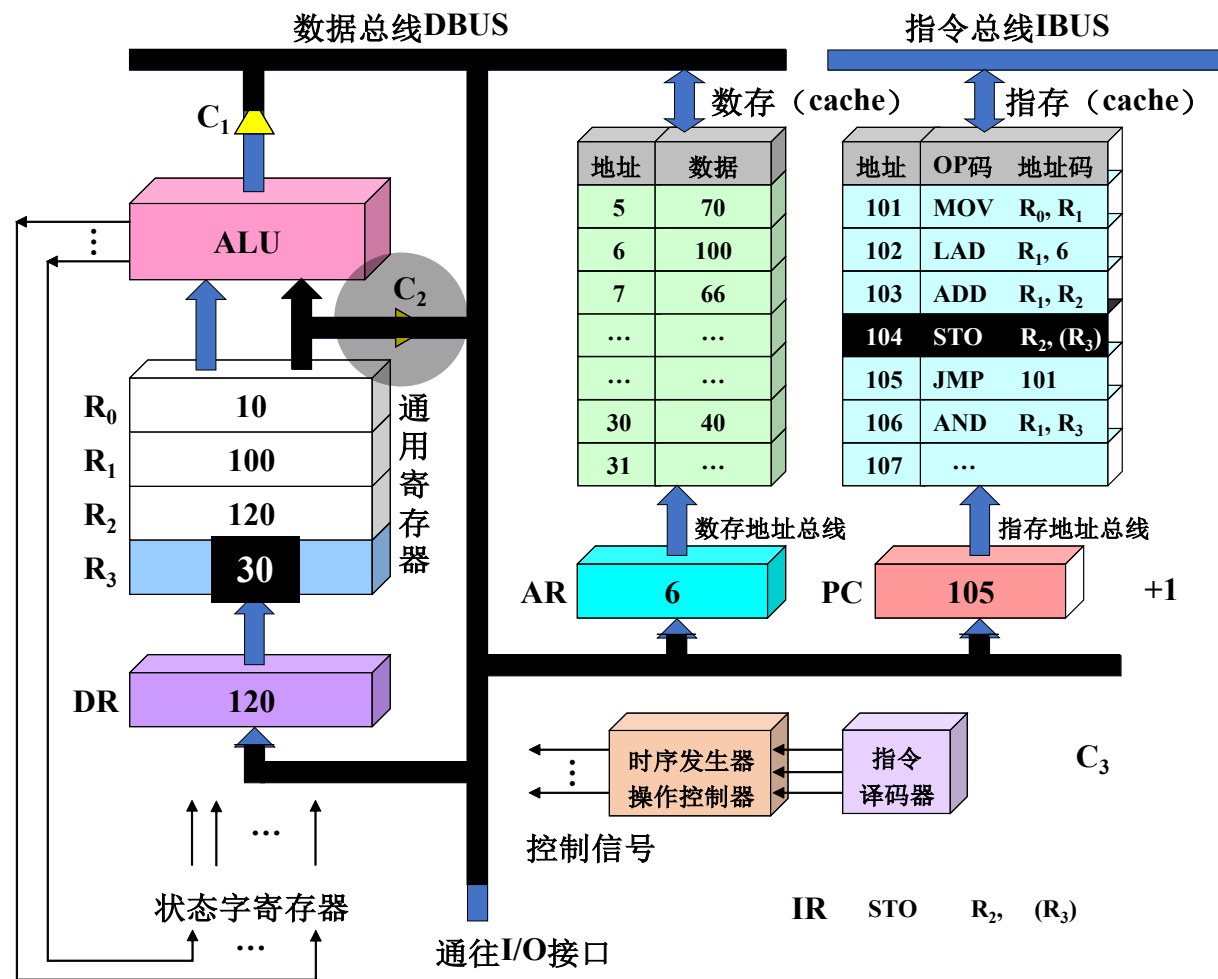


# 早期IAS机设计

## 2. 执行周期

2.1. 操作控制器OC送出控制命令  
到通用寄存器,  
选择(R3)=30做  
数据存储器访问地址

2.2. OC发出操作命令,  
打开通用寄存器输出三态门  
(不经ALU以节省时间),  
将地址30放到DBUS上



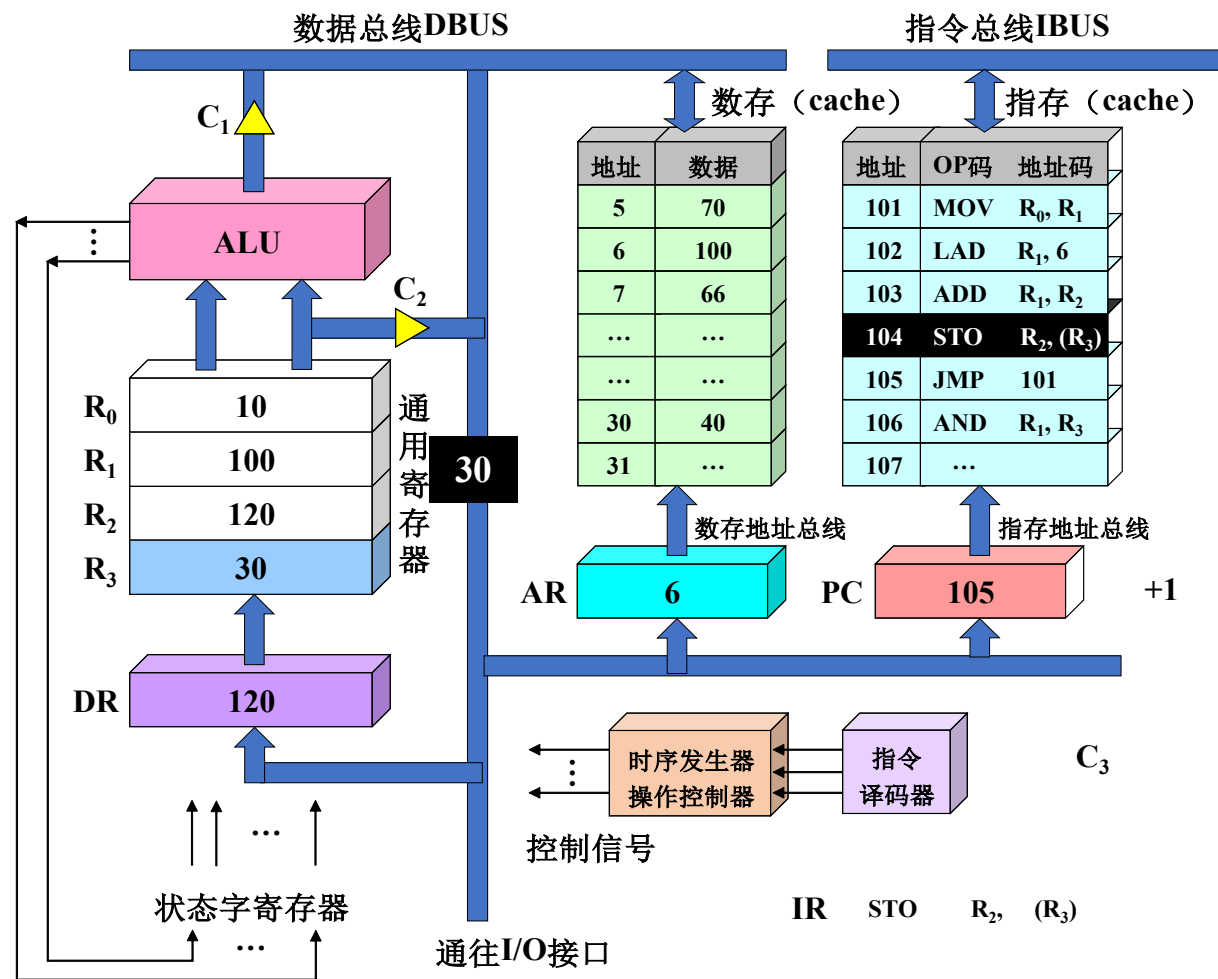
# 早期IAS机设计

## 2. 执行周期

...

2.2. OC发出操作命令,  
打开通用寄存器输出三态门  
(不经ALU以节省时间),  
将地址30放到DBUS上

2.3. OC发出操作命令,  
将地址30送入AR,  
并进行数存地址译码;



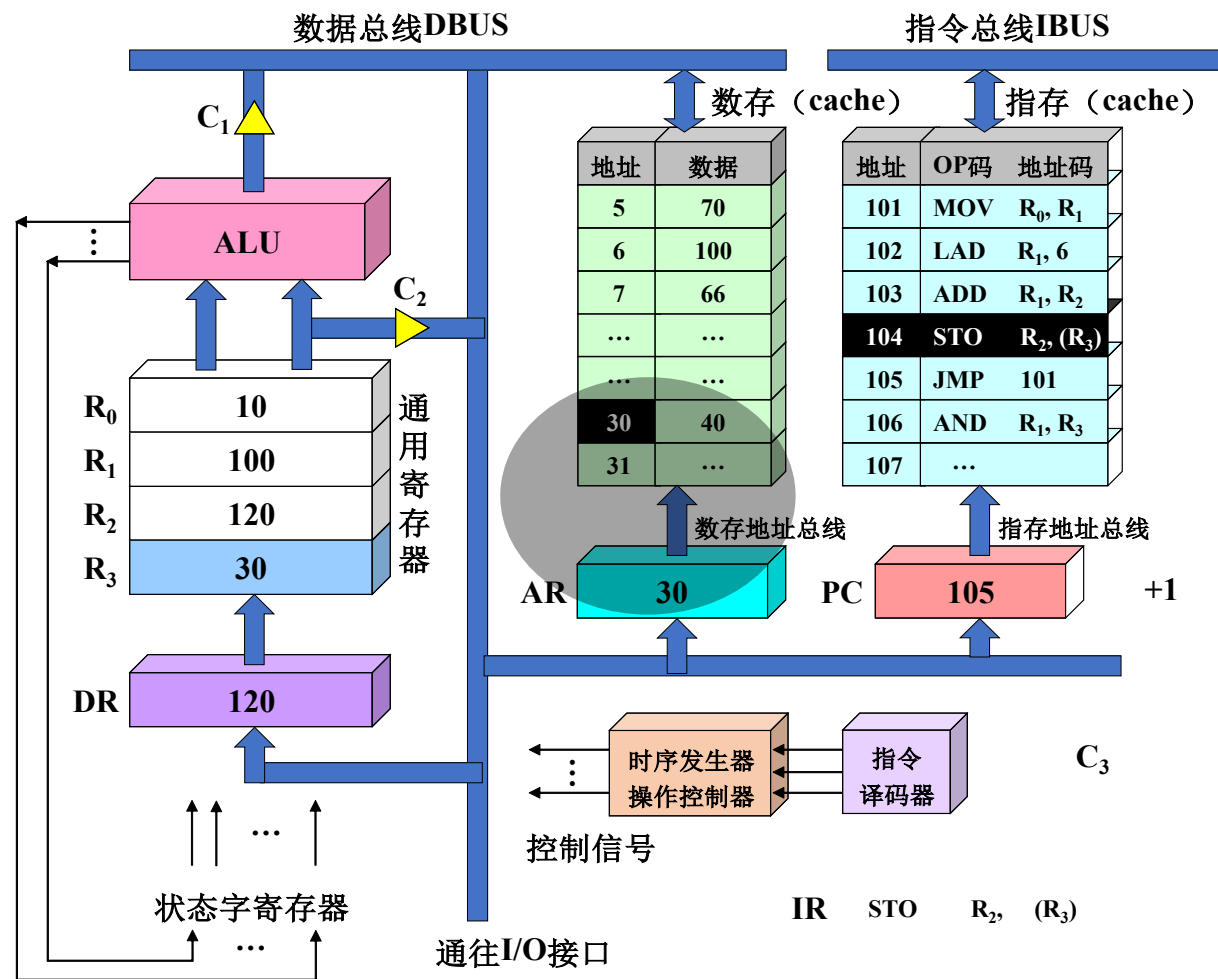
# 早期IAS机设计

## 2. 执行周期

...

2.2. OC发出操作命令，  
打开通用寄存器输出三态门  
(不经ALU以节省时间)，  
将地址30放到DBUS上

2.3. OC发出操作命令，  
将地址30送入AR，  
并进行数存地址译码；



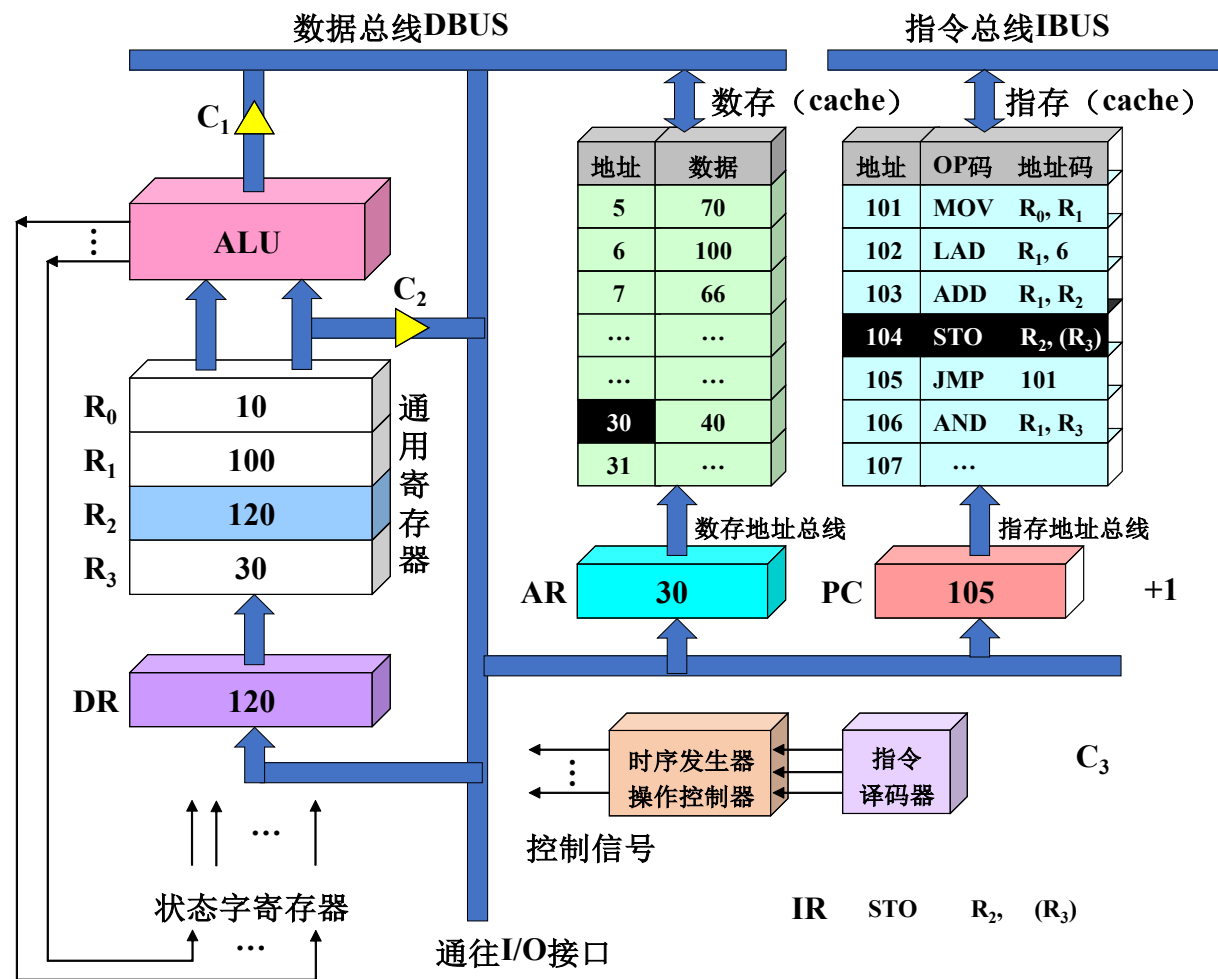
# 早期IAS机设计

## 2. 执行周期

...

2.3. OC发出操作命令，  
将地址30送入AR，  
并进行数存地址译码

2.4. OC发出操作命令到通用寄存器，  
选择(R2)=120，  
作为数存的写入数据



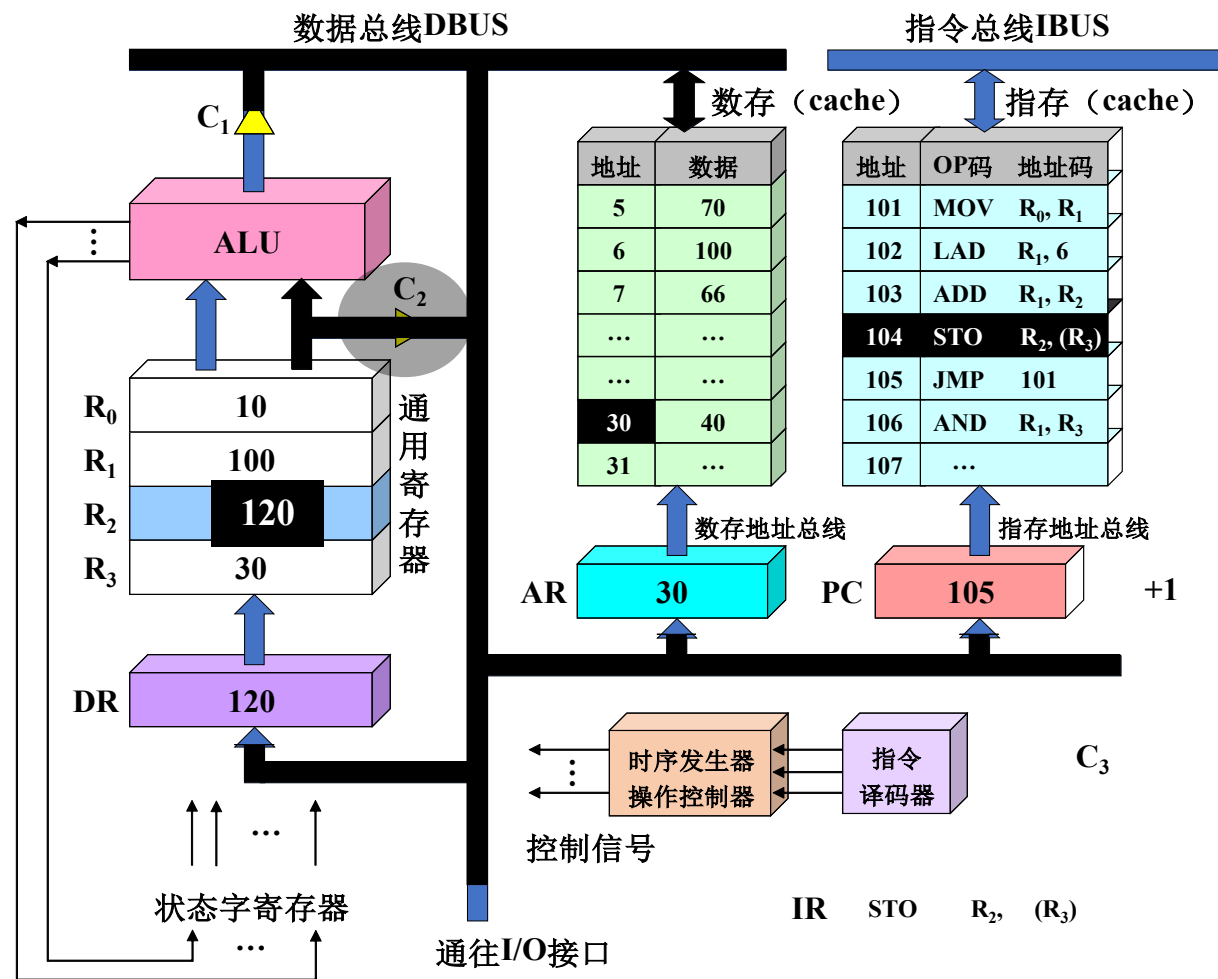
# 早期IAS机设计

## 2. 执行周期

...

2.4. OC发出操作命令到通用寄存器,  
选择(R2)=120,  
作为数存的写入数据

2.5. OC发出操作命令,  
打开通用寄存器输出三态门,  
将数据120放到DBUS上



# 早期IAS机设计

## 2. 执行周期

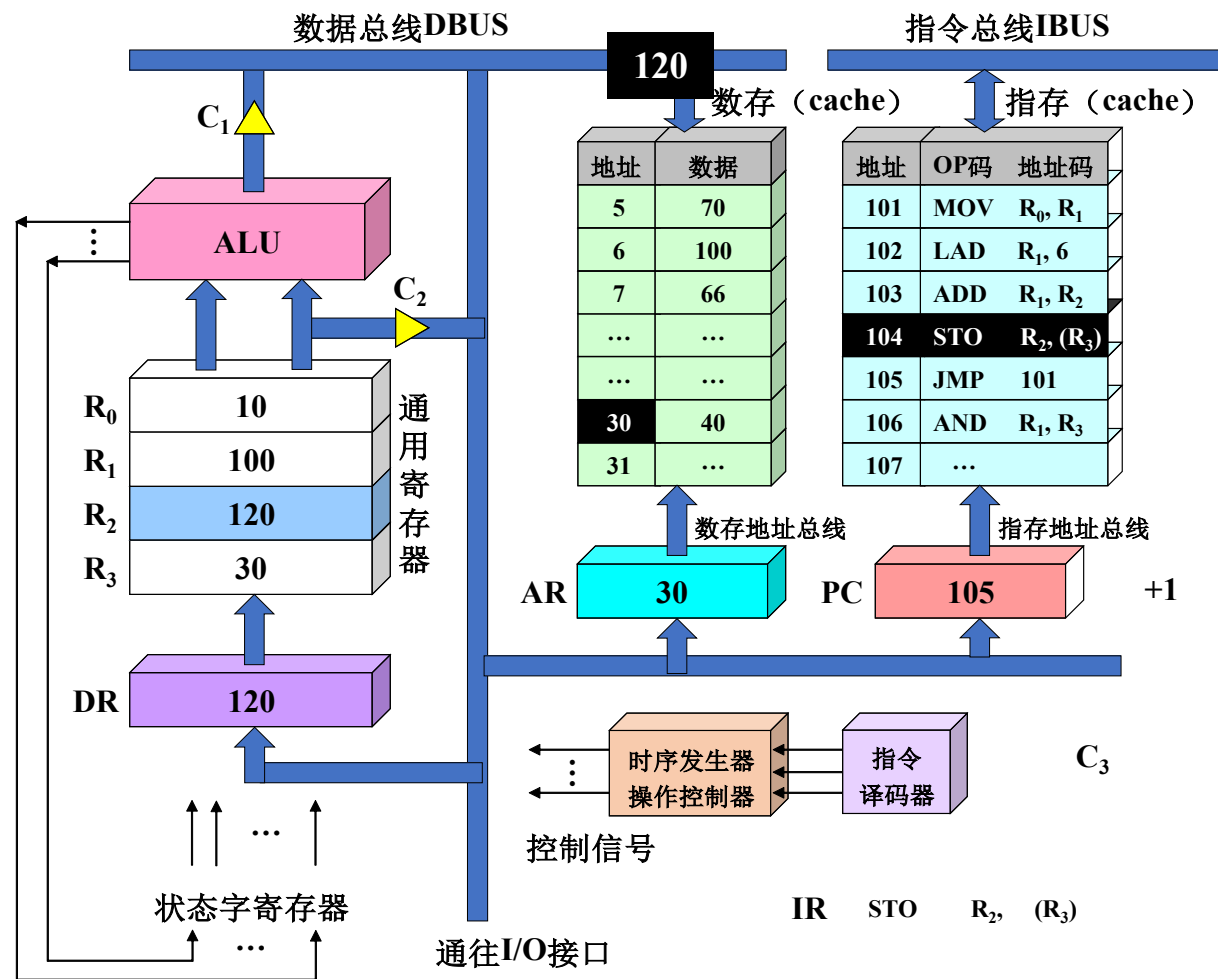
...

### 2.5. OC发出操作命令,

打开通用寄存器输出三态门,  
将数据120放到DBUS上

### 2.6. OC发出操作命令,

将数据120写入数存30号单元,  
30号单元中原先的数据40被覆盖



# 早期IAS机设计

## 2. 执行周期

...

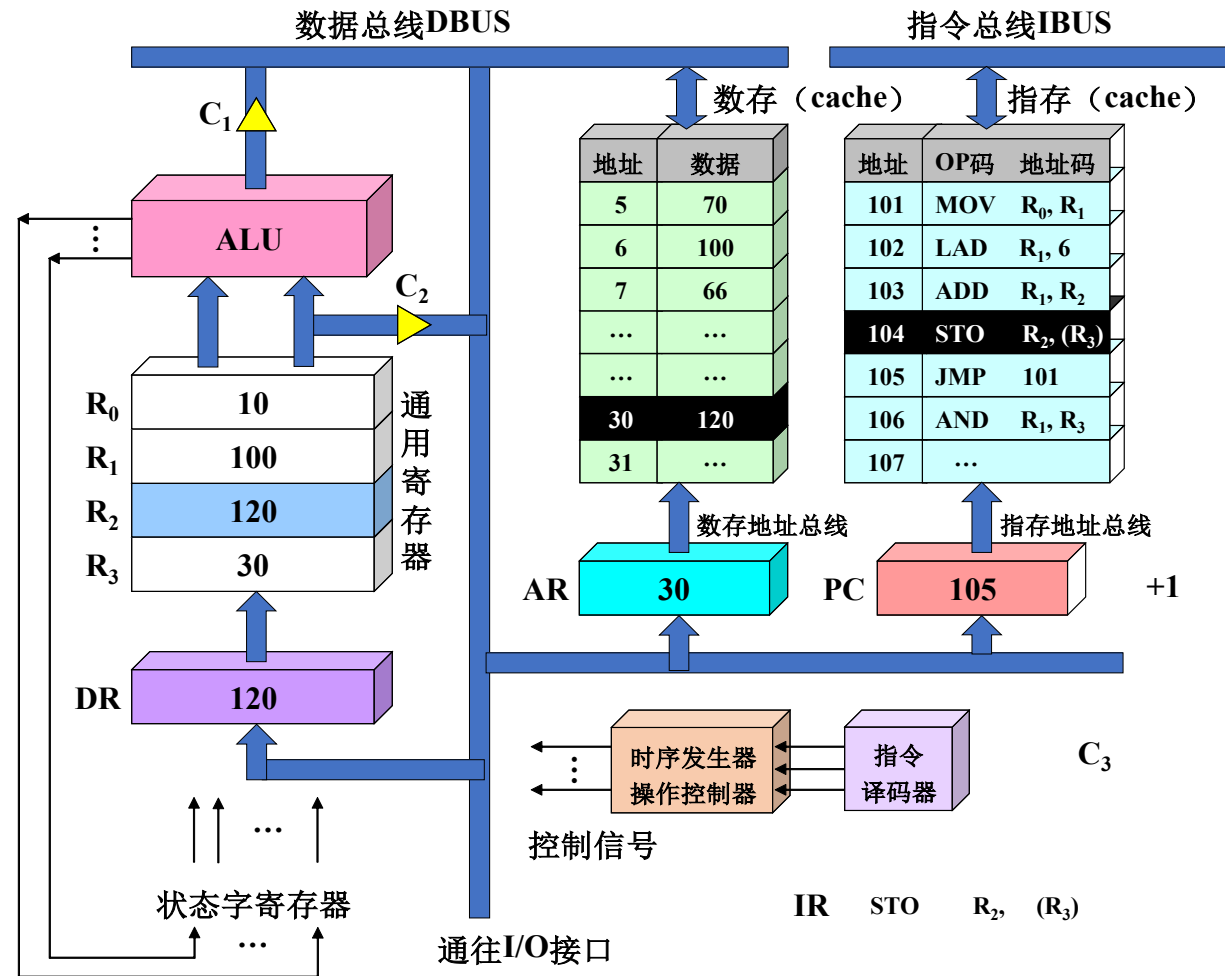
### 2.5. OC发出操作命令,

打开通用寄存器输出三态门,  
将数据120放到DBUS上

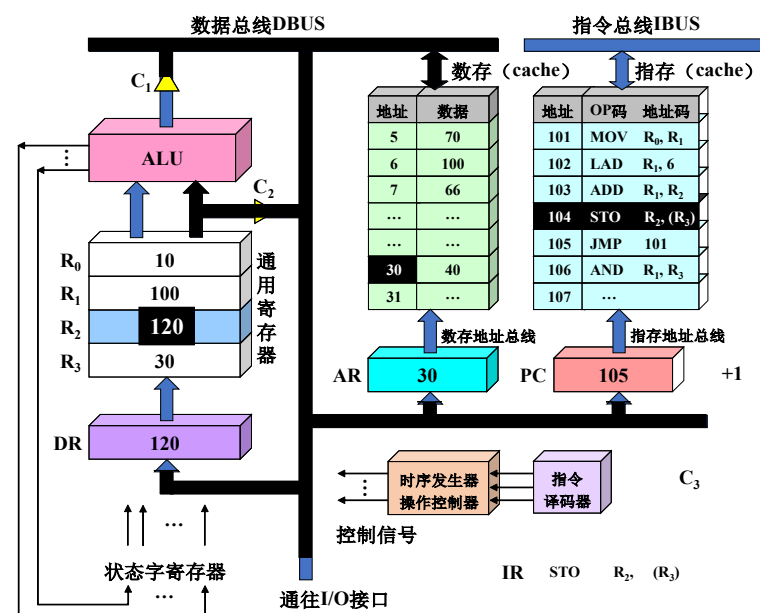
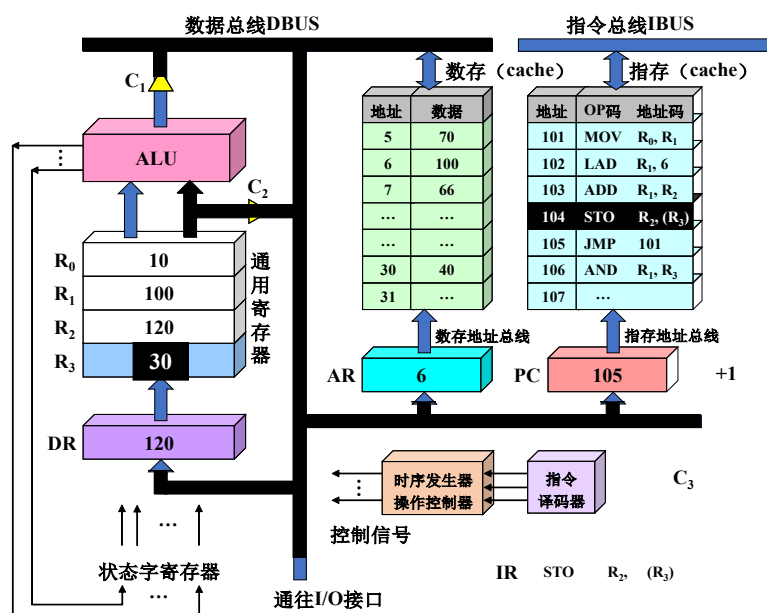
### 2.6. OC发出操作命令,

将数据120写入数存30号单元,  
30号单元中原先的数据40被覆盖

STO指令执行周期结束



# 早期IAS机设计



**DBUS是单总线结构，先送地址（30），后送数据（120），必须分时传送。**  
**因此，执行周期需要2个CPU周期。**

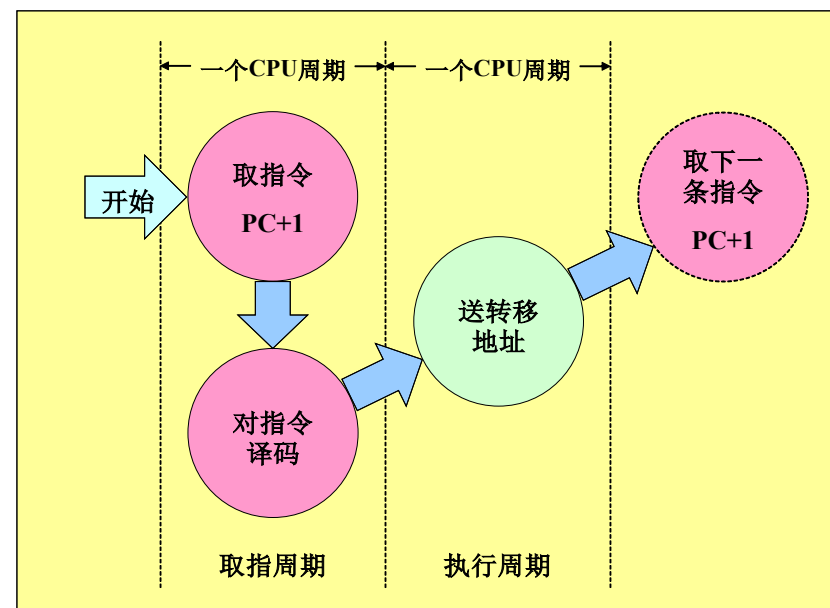
# 早期IAS机设计

## • JMP指令的指令周期

- 无条件转移指令，用来改变程序的执行顺序
- 指令周期为2个CPU周期。

### 1. 取指周期

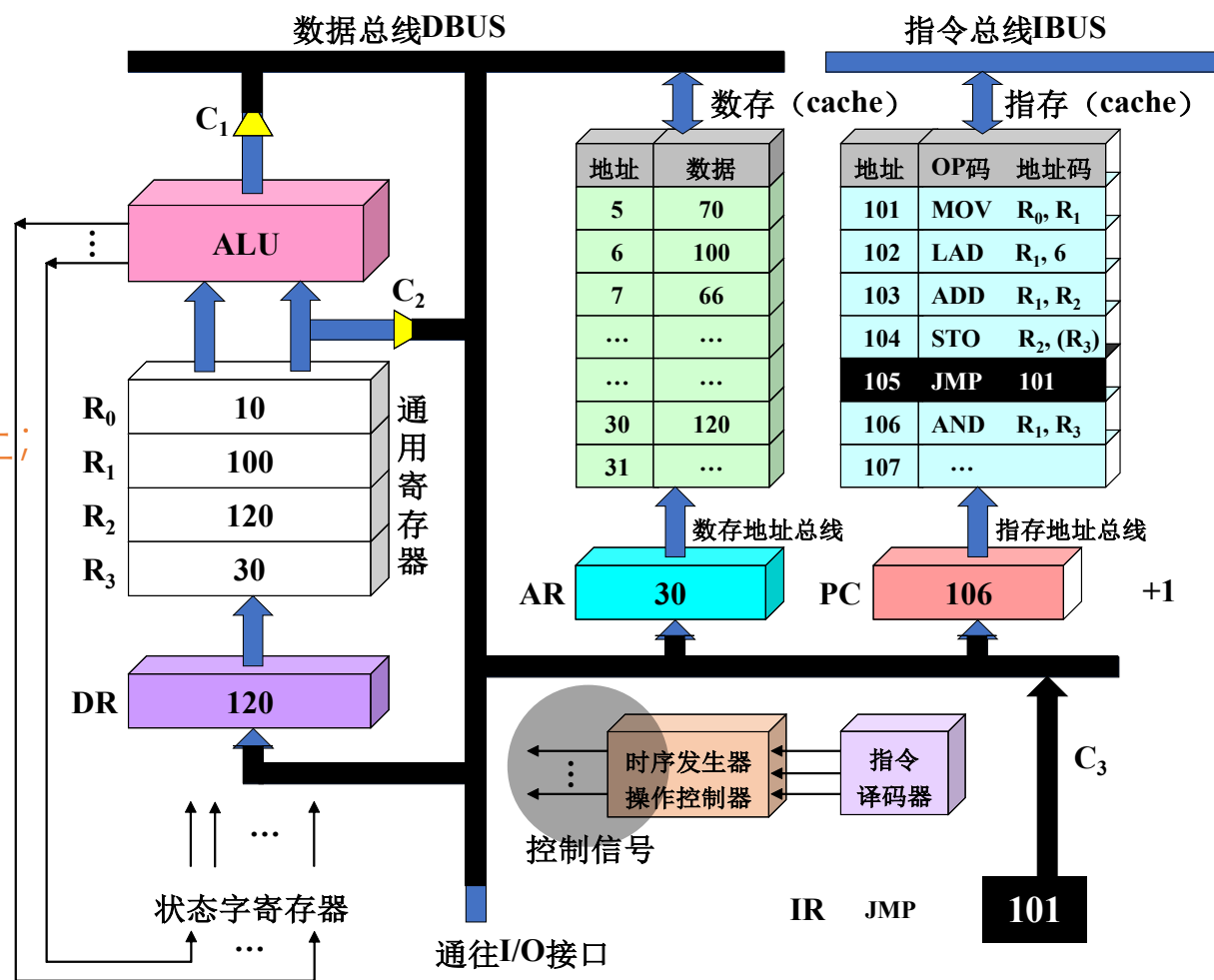
- 同MOV、LAD、ADD、STO指令



# 早期IAS机设计

## 2. 执行周期

2.1. 操作控制器OC送出控制命令,  
打开指令寄存器IR的输出三态门;  
将IR中的地址码101发送到DBUS上;

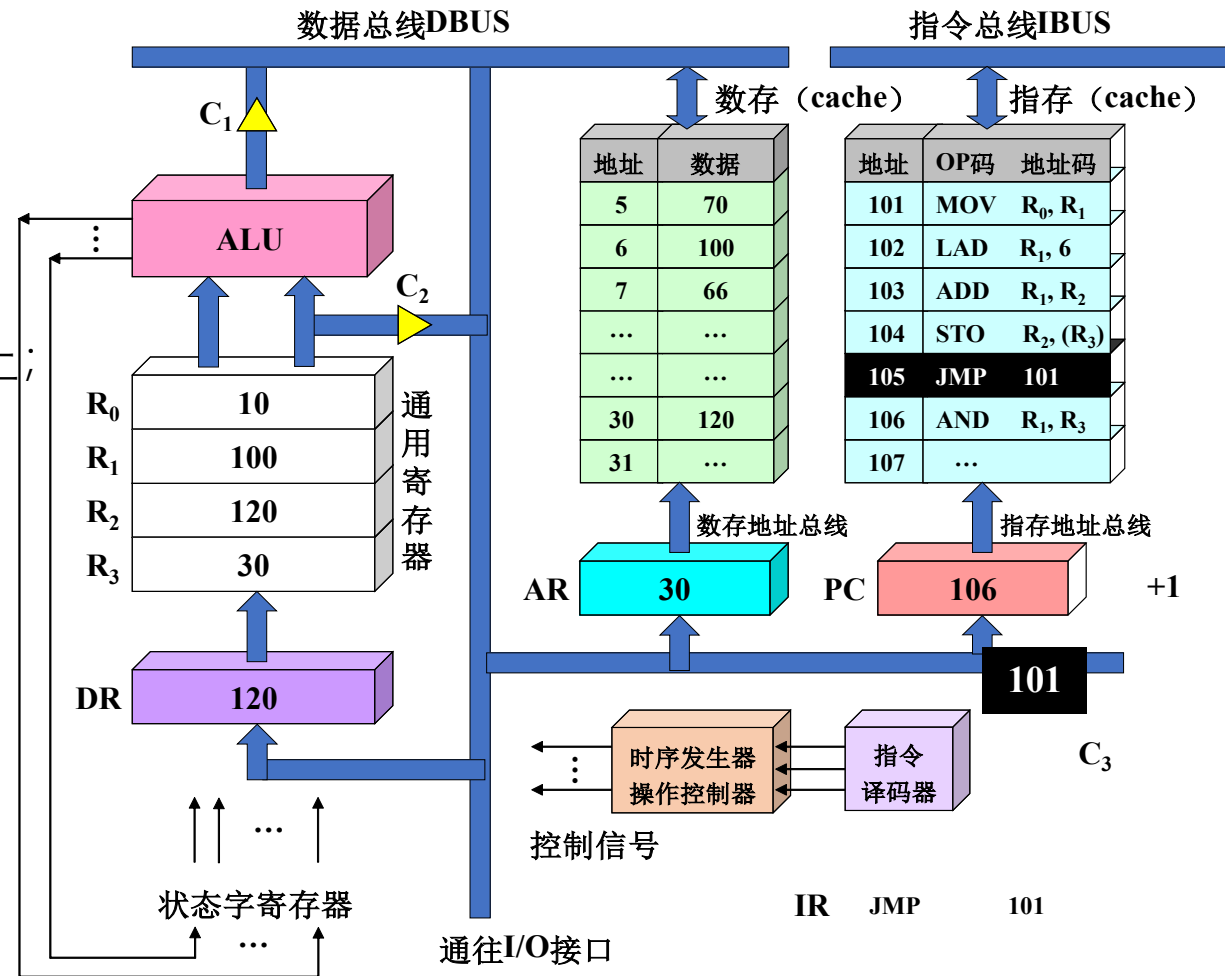


# 早期IAS机设计

## 2. 执行周期

2.1. 操作控制器OC送出控制命令,  
打开指令寄存器IR的输出三态门;  
将IR中的地址码101发送到DBUS上;

2.2. OC送出控制命令,  
将DBUS上的地址码101  
送入程序计数器PC,  
PC中原先的内容106被更换



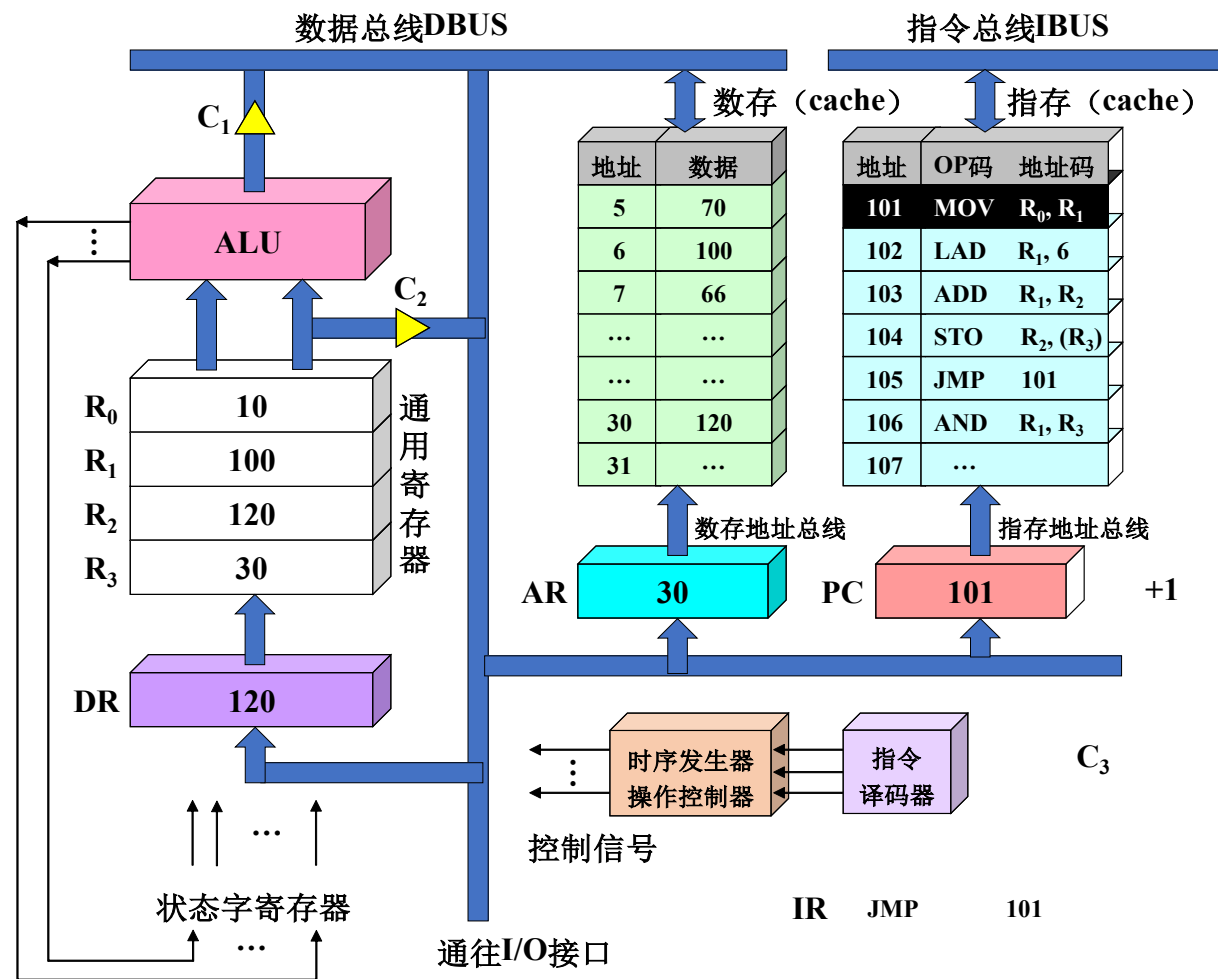
# 早期IAS机设计

## 2. 执行周期

2.1. 操作控制器OC送出控制命令,  
打开指令寄存器IR的输出三态门;  
将IR中的地址码101发送到DBUS上;

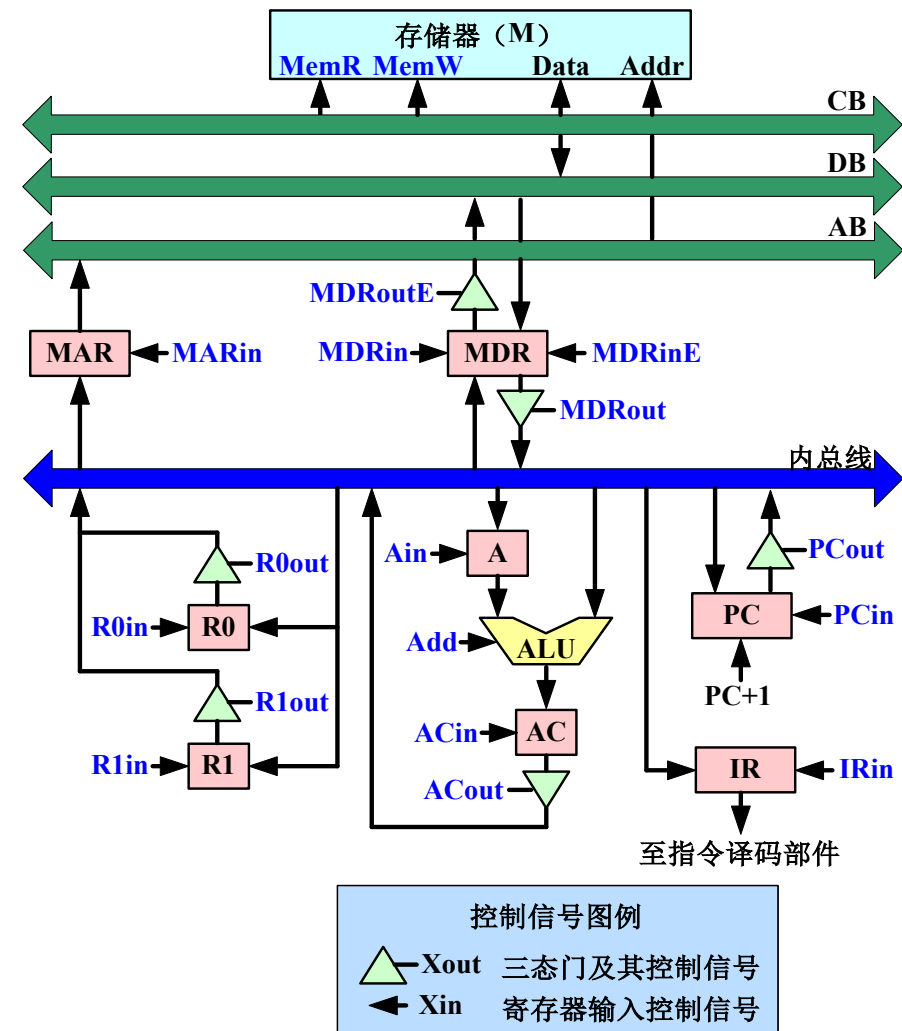
2.2. OC送出控制命令,  
将DBUS上的地址码101  
送入程序计数器PC,  
PC中原先的内容106被更换

下一条指令不是从指存106号单元取出,  
而是转移到101号单元取出  
JMP指令执行周期结束



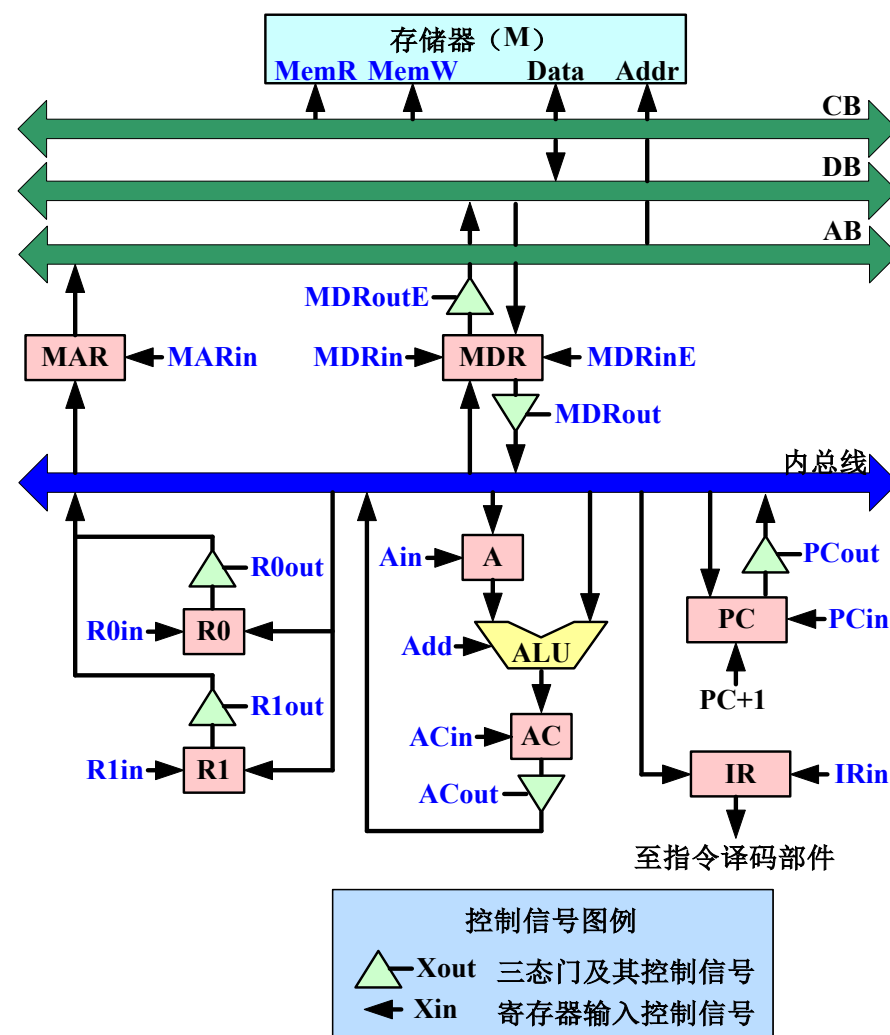
# 早期IAS机设计

- 例. 某计算机字长16位，采用16位定长指令字结构，部分数据通路结构如下图所示。
  - 图中所有控制信号为1时表示有效，为0时表示无效。例如控制信号MDRinE为1表示允许数据从DB送入MDR，MDRin为1表示允许数据从内总线送入MDR。
  - 假设MAR的输出一直处于使能状态。



# 早期IAS机设计

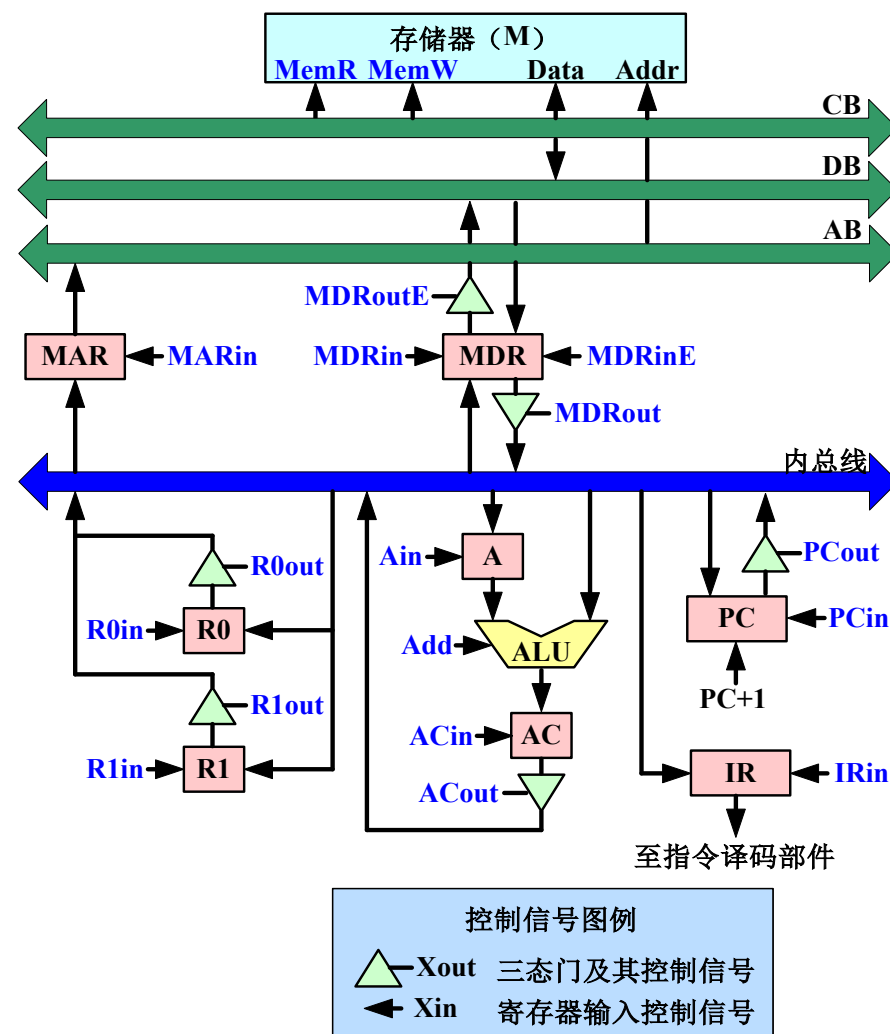
- 加法指令“ADD(R1), R0”的功能为 $(R0) + ((R1)) \rightarrow (R1)$ , 即: 将R0中的数据与R1的内容所指主存单元的数据相加, 并将结果送入R1的内容所指主存单元中保存。
- 下表给出了上述指令取指和执行阶段每个节拍 (时钟周期) 的功能和有效控制信号, 请按表中描述方式用表格列出指令执行阶段每个节拍的功能和有效控制信号。



# 早期IAS机设计

## • ADD (R1), R0

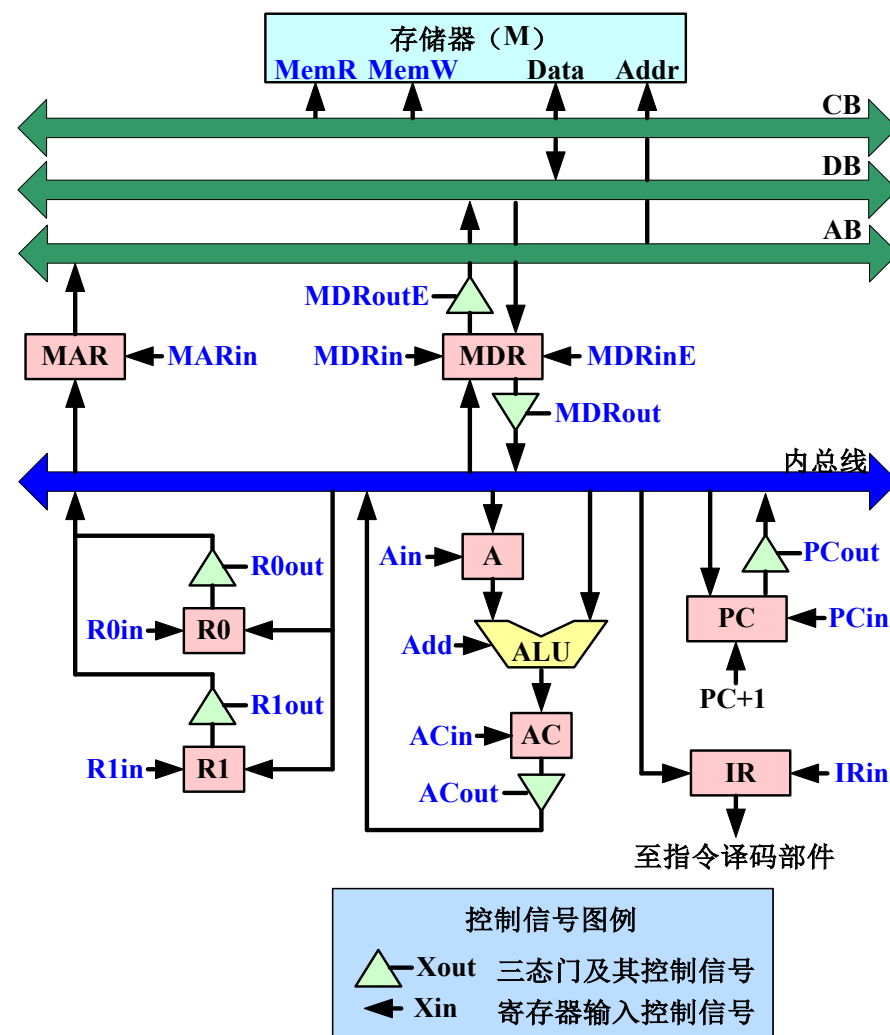
| 时钟  | 功能                                                  | 有效控制信号                                                                          |
|-----|-----------------------------------------------------|---------------------------------------------------------------------------------|
| C1  | $MAR \leftarrow (PC)$                               | PCout, MARin                                                                    |
| C2  | $MDR \leftarrow M(MAR)$<br>$PC \leftarrow (PC) + 1$ | MemR, MDRinE<br>PC+1                                                            |
| C3  | $IR \leftarrow (MDR)$                               | MDRout, IRin                                                                    |
| C4  | 指令译码                                                | 无                                                                               |
| C5  | $MAR \leftarrow (R1)$                               | R1out, MARin                                                                    |
| C6  | $MDR \leftarrow M(MAR)$                             | MemR, MDRinE                                                                    |
| C7  | $A \leftarrow (MDR)$                                | MDRout, Ain                                                                     |
| C8  | $AC \leftarrow (R0) + (A)$                          | R0out, Add, ACin                                                                |
| C9  | $MDR \leftarrow (AC)$                               | ACout, MDRin                                                                    |
| C10 | $M \leftarrow (MDR)$                                | MDRoutE, <span style="background-color: black; color: black;">XXXXXXXXXX</span> |



# 早期IAS机设计

## • ADD (R1), R0

| 时钟  | 功能                                                  | 有效控制信号               |
|-----|-----------------------------------------------------|----------------------|
| C1  | $MAR \leftarrow (PC)$                               | PCout, MARin         |
| C2  | $MDR \leftarrow M(MAR)$<br>$PC \leftarrow (PC) + 1$ | MemR, MDRinE<br>PC+1 |
| C3  | $IR \leftarrow (MDR)$                               | MDRout, IRin         |
| C4  | 指令译码                                                | 无                    |
| C5  | $MAR \leftarrow (R1)$                               | R1out, MARin         |
| C6  | $MDR \leftarrow M(MAR)$                             | MemR, MDRinE         |
| C7  | $A \leftarrow (MDR)$                                | MDRout, Ain          |
| C8  | $AC \leftarrow (R0) + (A)$                          | R0out, Add, ACin     |
| C9  | $MDR \leftarrow (AC)$                               | ACout, MDRin         |
| C10 | $M \leftarrow (MDR)$                                | MDRoutE, MemW        |

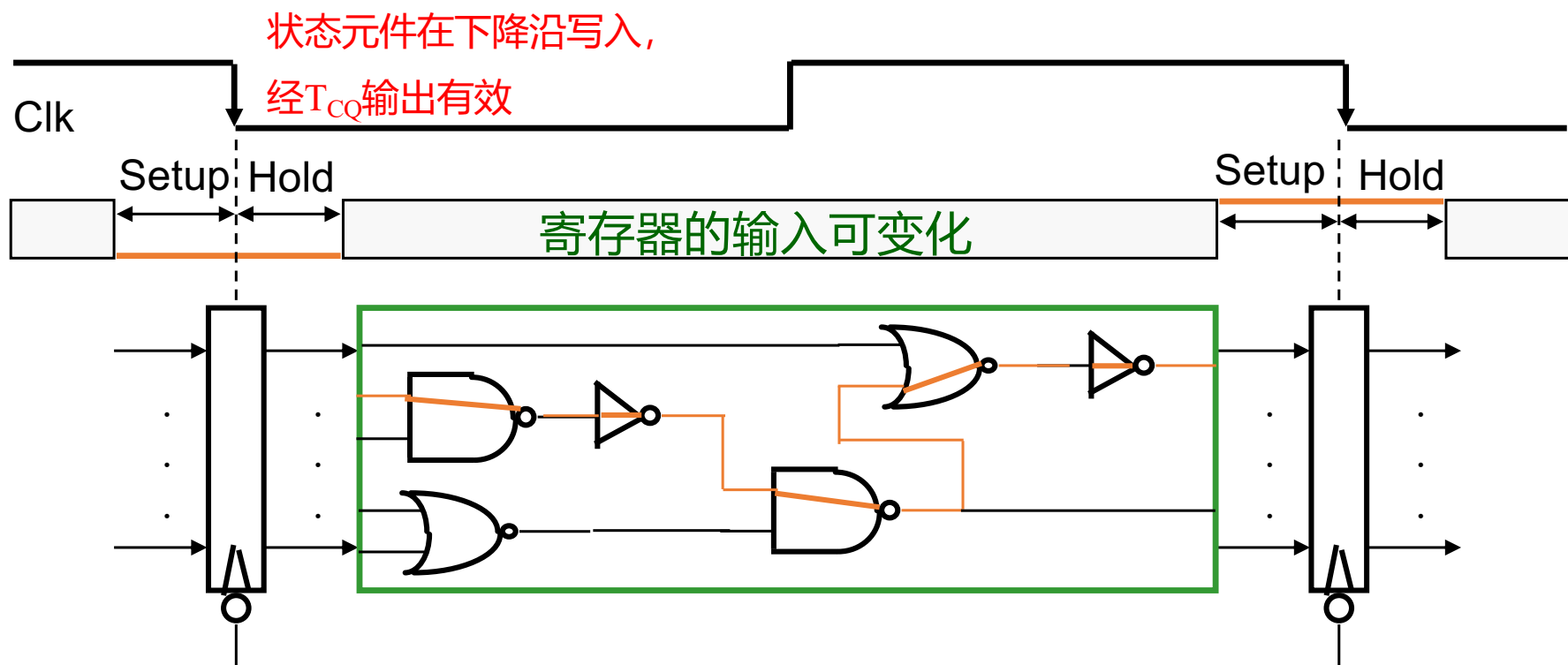


# 数据通路与时序控制

数据通路由 “...+状态元件+操作元件+  
状态元件+...” 组成

- 现代计算机已不再采用三级时序系统，机器周期的概念已逐渐消失。
- 整个数据通路中的定时信号就是时钟，一个时钟周期就是一个节拍。

只有状态元件能存储信息，所有操作元件都须从状态单元接收输入，并将输出写入状态单元中。其输入为前一时钟生成的数据，输出为当前时钟所用的数据



# 处理器设计

- 处理器设计涉及
  - 数据通路的设计
    - 数据通路中有两种元件
      - 操作元件：由组合逻辑电路实现
      - 存储（状态）元件：由时序逻辑电路实现
  - 控制单元的设计

# 设计处理器的步骤

- ISA（Y86-64）确定后，进行分散连接处理器设计的大致步骤
  - 第一步：分析每条指令的功能，并用一种描述语言来形式化地表示。
  - 第二步：根据指令的功能给出所需的元件，并考虑如何将他们互连。
  - 第三步：确定每个元件所需控制信号的取值。
  - 第四步：汇总所有指令所涉及到的控制信号，据此设计控制器电路。

# 数据通路：存储、处理、**传送**

## • (单) 总线式

- 寄存器文件、ALU、存储器通过**一条公共总线**连接
- 优点：结构简单、硬件量少、容易设计
- 缺点
  - 总线带宽有限（一次传一份数据）
  - 访问冲突（取指令、读数据、写结果用同一总线，必须排队）
  - 流水线难以展开

**8086**

## • 非总线（分散）式

- 不同功能部件有**独立的通路**
- 缺点：硬件复杂、芯片面积大
- 优点：
  - 并行性好（同时取指、访存、执行）
  - 适合支持指令流水

**早期IAS机**

指令存储器、数据存储器分离，  
取指、访存可以并行

**现代RISC CPU  
(MIPS、ARM)**